

Appunti di Basi di Dati

Emanuele Romano

Indice

1	Progettazione Concettuale	1
1.1	Progettazione Concettuale con Class Diagrams	1
1.1.1	Ruoli nelle Associazioni	2
1.1.2	Navigabilità e Direzionalità	2
1.1.3	Tipi Enumerativi (Enumerazioni)	3
1.1.4	Classi di Associazione	3
1.1.5	Grado di un'Associazione e Relazioni Ternarie	5
1.1.6	Aggregazione e Composizione	7
1.1.7	Generalizzazioni/Specializzazioni	8
1.2	La Documentazione di Supporto: I Dizionari dei Dati	9
2	Modello Relazionale e Progettazione Logica	11
2.1	Introduzione	11
2.2	Concetti Fondamentali	11
2.3	Le Chiavi	13
2.4	Traduzione delle Associazioni	14
2.4.1	Associazioni 1 a 1	14
2.4.2	Associazioni 1 a Molti (1:N)	16
2.4.3	Associazioni Molti a Molti (N:N)	17
2.5	Ristrutturazione dello Schema Concettuale	18
2.5.1	Analisi delle Chiavi: Chiavi Naturali vs Surrogate	19
2.5.2	Analisi degli Attributi Derivati e delle Ridondanze	20
2.5.3	Analisi delle Associazioni Uno a Uno: Fusione vs Separazione	21
2.5.4	Analisi degli Attributi Strutturati	22
2.5.5	Analisi degli Attributi a Valore Multiplo	24
2.6	Codifica delle Gerarchie (Ereditarietà)	25
2.7	Riepilogo: Il Ciclo di Vita della Progettazione dei Dati	26
3	Normalizzazione Relazionale	28
3.1	Ridondanze e Anomalie	28
3.2	Dipendenze Funzionali	29
3.3	La Forma Normale di Boyce-Codd (BCNF)	30
3.3.1	Decomposizioni	31
3.3.2	Qualità delle Decomposizioni	32
3.4	La Terza Forma Normale (3NF)	33
3.5	Le Forme Normali 1NF e 2NF	34
4	Introduzione a SQL e Definizione dei Metadati	36
4.1	Nozioni introduttive	36
4.2	I Domini di Base (Tipi di Dato) nello Standard SQL	37
4.3	Operatori Logici e di Confronto	39
4.4	Istruzioni DDL	40
4.4.1	Creazione di uno schema logico	40

4.4.2	Creazione di una tabella	41
4.4.3	I Vincoli Intra-attributo (o in-line)	41
4.4.4	I Vincoli di Tabella (Out-of-line)	42
4.4.5	Classificazione Concettuale dei Vincoli	44
4.4.6	Integrità Referenziale e Chiavi Esterne (FOREIGN KEY)	46
4.4.7	Identificatori Univoci: Chiavi Surrogate e il costrutto IDENTITY	48
5	SQL e Algebra Relazionale	51
5.1	Proiezioni	52
5.1.1	La Proiezione in SQL: Il comando SELECT	53
5.2	Selezioni	53
5.2.1	La Selezione in SQL: La clausola WHERE	54
5.2.2	Attributi Parziali e Logica Ternaria	55
5.3	Composizione di Proiezioni e Selezioni	56
5.3.1	La Composizione in SQL e l'Ordine Logico di Esecuzione	57
5.4	Qualificazione degli Attributi	58
5.5	Ridenominazione	58
5.5.1	La Ridenominazione in SQL: L'operatore AS	58
5.6	Operazioni Insiemistiche	59
5.6.1	Le Operazioni Insiemistiche in SQL	61
5.7	Il Prodotto Cartesiano e le Giunzioni (Join)	62
5.7.1	I Join in SQL	64
5.7.2	Il Prodotto Cartesiano in SQL: CROSS JOIN	65
5.8	Giunzioni Esterne (Outer Join)	66
5.8.1	Le Giunzioni Esterne in SQL	66
5.8.2	Il pattern "Anti-Join": Isolare le assenze	67
5.9	Raggruppamento e Funzioni di Aggregazione	67
5.9.1	Le Funzioni di Aggregazione in SQL: GROUP BY	69
5.9.2	Filtrare i Gruppi: La clausola HAVING	70
5.10	L'Ordine Logico di Esecuzione in SQL	71
5.11	Le Viste (Views): Tabelle Virtuali	72
5.12	Interrogazioni Nidificate (Subqueries)	73
5.12.1	Subqueries in WHERE (filtri dinamici)	73
5.12.2	Subqueries in FROM (Tabelle Derivate o Inline Views)	76
5.12.3	Subqueries in SELECT (Colonne Calcolate)	77
5.13	Viste vs Interrogazioni Nidificate	78
5.14	Ordinamento dei Risultati: La clausola ORDER BY	79
5.15	Espressioni e Funzioni Predefinite nella SELECT	80
5.15.1	Uso di Espressioni Costanti	81
5.15.2	Panoramica delle Funzioni Predefinite Standard	81
5.16	Manipolazione dei Dati (DML): INSERT, UPDATE, DELETE	82
5.16.1	L'istruzione INSERT	82
5.16.2	L'istruzione UPDATE	83
5.16.3	L'istruzione DELETE	83

6	SQL Avanzato in PostgreSQL	85
6.1	Controllo del Result Set: Limit, Offset e Nulls	85
6.1.1	Taglio e Paginazione: <code>LIMIT</code> e <code>OFFSET</code>	85
6.1.2	Gestione delle Incognite nell'Ordinamento: <code>NULLS FIRST / LAST</code>	86
6.2	DML Avanzato in PostgreSQL: <code>RETURNING</code> e <code>UPSERT</code>	86
6.3	Gestione Dati Semi-Strutturati: Il tipo <code>JSONB</code>	87
6.4	Common Table Expressions (CTE): Modularità e Ricorsione	88
6.4.1	La clausola <code>WITH</code> (Sintassi e semantica)	88
6.4.2	CTE Multiple e Concatenate (Data Pipelining)	89
6.4.3	Cenni alle CTE Ricorsive (<code>WITH RECURSIVE</code>)	90
6.5	Logica Condizionale e Gestione delle Incognite	92
6.5.1	Il costrutto <code>CASE WHEN</code> (Semplice vs Ricercato)	93
6.5.2	Pivoting dei Dati (Trasposizione)	95
6.5.3	Funzioni di coalescenza: gestione dei valori <code>NULL</code>	96
6.6	Funzioni Analitiche (Window Functions)	97
6.6.1	L'Ordinamento interno alla Finestra (<code>ORDER BY</code>)	98
6.6.2	Il Framing (<code>ROWS BETWEEN / RANGE BETWEEN</code>)	99
6.6.3	Funzioni di Ranking (<code>ROW_NUMBER()</code> , <code>RANK()</code> , <code>DENSE_RANK()</code>)	101
6.6.4	Funzioni Posizionali (<code>LEAD()</code> e <code>LAG()</code>)	103
7	Ottimizzazione, Prestazioni, Scalabilità	105
7.1	L'Anatomia dello Storage	105
7.2	Strutture Dati per l'Indicizzazione	106
7.2.1	Il motore universale relazionale: Il B-Tree (Balanced Tree)	106
7.2.2	Velocità puntuale: Gli Indici Hash	108
7.3	Strategie di Indicizzazione: Clustered vs Non-Clustered Index	109
7.3.1	Forzatura dell'Architettura: Il comando <code>CLUSTER</code> e il De-Clustering	111
7.4	Uso degli Indici	112
7.4.1	Indicizzazione Automatica vs Manuale	112
7.4.2	Indici Composti e la Regola del Prefisso Sinistro	114
7.4.3	Cheat Sheet: Quando creare un Indice (e quando evitarlo)	115
7.5	Ispezione Fisica delle Query: I Piani di Esecuzione (PostgreSQL)	116
7.5.1	Gli strumenti di diagnostica: <code>EXPLAIN</code> vs <code>EXPLAIN ANALYZE</code>	116
7.5.2	Leggere un Piano di Esecuzione e identificare i colli di bottiglia	116
8	Gestione delle Transazioni e Concorrenza	118
8.1	Fondamenti della Transazione e Proprietà ACID	118
8.1.1	Cos'è una transazione: il concetto di "Tutto o Niente" (Commit e Rollback)	118
8.1.2	Le 4 Proprietà ACID: Il Contratto del Database	119
8.1.3	Il limite del singolo statement: perché una singola query non basta	120
8.2	Le Anomalie della Concorrenza	121
8.2.1	Lost Update (L'Aggiornamento Perduto)	122
8.2.2	Dirty Read (Lettura Sporca)	122
8.2.3	Non-Repeatable Read (Lettura Non Ripetibile)	123
8.2.4	Phantom Read (Lettura Fantasma)	124
8.3	I Livelli di Isolamento	125
8.4	Lock Pessimistico vs Lock Ottimistico	127
8.4.1	Lock Pessimistico: La Forza Bruta (<code>FOR UPDATE</code>)	127

8.4.2	Lock Ottimistico: L'Astuzia del Versioning (Lo Standard)	127
8.4.3	Implementazione in JPA/Hibernate: L'annotazione @Version	128
8.5	La Pratica in Spring Boot: L'annotazione @Transactional	129
8.5.1	Il pattern Proxy di Spring: cosa succede "sotto il cofano"	130
8.5.2	La trappola del Rollback: Eccezioni Checked vs Unchecked	131
8.5.3	Propagazione delle Transazioni: REQUIRED vs REQUIRES_NEW	132

Progettazione Concettuale

1.1 Progettazione Concettuale con Class Diagrams

La progettazione concettuale è la fase in cui si modella il *cosa* deve essere rappresentato nella base di dati, ignorando i dettagli implementativi del DBMS. Sebbene la letteratura classica utilizzi il modello Entity-Relationship (ER), l'approccio moderno predilige l'uso dei **Class Diagrams UML**, di cui di seguito richiameremo i concetti fondamentali. Si assumerà che il lettore abbia già familiarità con i diagrammi UML in un contesto orientato agli oggetti, dunque ne faremo dei richiami piuttosto sommari e ci concentreremo sulle specificità della modellazione per basi di dati.

Il punto di partenza è un testo in linguaggio naturale (specifiche). La costruzione del diagramma segue una strategia di mapping semantico:

- **Sostantivi → Classi (Entità):** Rappresentano oggetti con esistenza autonoma (es. *Studente*, *Corso*).
- **Verbi → Associazioni (Relazioni):** Descrivono i legami logici tra le classi (es. *Iscrive*, *Sostiene*).
- **Aggettivi o Specifiche → Attributi:** Proprietà atomiche che descrivono una classe (es. *Nome*, *Matricola*).

Gli elementi costitutivi di un Class Diagram sono:

Classe: Rappresentata da un rettangolo. Nella progettazione concettuale ci si concentra su *Nome* e *Attributi*. Le *Operazioni* (metodi) vengono solitamente omesse.

Associazione: Una linea che connette due classi. Indica che esiste una correlazione logica tra le istanze delle classi coinvolte.

Cardinalità: Indicano il numero minimo e massimo di istanze di una classe che possono essere legate a una singola istanza della classe opposta.

Tabella 1.1: Sintassi delle Cardinalità UML

Sintassi	Significato
1	Esattamente uno (Obbligatorio)
0..1	Zero o uno
* o 0..*	Zero o molti
1..*	Uno o molti (Almeno uno)

Un attributo si dice **totale** se ha cardinalità minima strettamente maggiore di zero, **parziale** altrimenti: quindi il primo deve sempre avere un valore, mentre per uno parziale il valore è opzionale.

Un attributo si dice **singolo** se ha cardinalità massima uguale a uno, **multiplo** altrimenti.

Un attributo è **atomico** se il database non ne percepisce la struttura interna. Gli attributi atomici possono essere:

- **Primitivi:** Rappresentano valori elementari non scomponibili.

- *Numerici*: `int`, `float`, `decimal`.
- *Alfanumerici*: `string`, `char`.
- *Logici*: `boolean`.
- **Predefiniti Strutturati**: Tipi di dato complessi forniti dal sistema e trattati come atomici dal DBMS.
 - *Temporal*: `date`, `time`, `timestamp`.
 - *Oggetti Binari*: `blob` (immagini, file), `clob` (testi estesi).

Un errore comune è modellare come attributo ciò che dovrebbe essere una classe. La regola empirica è:

1. Se l'informazione è un valore semplice (es. *Età*), è un **Attributo**.
2. Se l'informazione ha proprietà proprie o deve essere legata ad altre entità (es. *Indirizzo* inteso come via, civico e CAP), deve essere una **Classe** autonoma.



L'insieme degli attributi deve permettere di identificare univocamente ogni istanza della classe.

Osservazione 1.1. Sebbene in Java usi `-` (`private`) e `+` (`public`), nei diagrammi per database la visibilità è meno rilevante, ma mantenere il `-` per gli attributi aiuta a ricordare l'incapsulamento dei dati.

1.1.1 Ruoli nelle Associazioni

Un **ruolo** identifica la funzione specifica che un'istanza di una classe svolge all'interno di un'associazione. Viene indicato alle estremità della linea di associazione.

- **Utilità**: I ruoli sono essenziali per chiarire il significato del legame, specialmente quando l'associazione è *riflessiva* (connette una classe a sé stessa) o quando esistono più associazioni tra le stesse classi.
- **Naming Convention**: Solitamente si utilizza un sostantivo che descrive la responsabilità (es. *responsabile*, *supervisore*, *partecipante*).
- **Corrispondenza OOP**: Nella programmazione ad oggetti, il nome del ruolo coincide spesso con il nome dell'attributo (o del campo) che contiene il riferimento all'oggetto correlato.

Esempio 1.1 (Associazione Riflessiva o Ricorsiva). Si consideri la classe **Persona** e l'associazione *Genitorialità*. Senza ruoli, il legame sarebbe ambiguo. Assegnando i ruoli **genitore** e **figlio** alle due estremità, la semantica del modello diventa univoca.

Osservazione 1.2. Spesso si tende a omettere il nome dell'associazione (il verbo al centro della linea) se i ruoli sono già sufficientemente esplicativi. In molti schemi professionali vengono mostrati solo i ruoli, perché sono più vicini alla struttura logica finale del database.

1.1.2 Navigabilità e Direzionalità

Una differenza cruciale tra la modellazione OOP e quella per basi di dati riguarda la **navigabilità** delle associazioni.

- **In Java (OOP)**: Le associazioni sono spesso *unidirezionali* per favorire il disaccoppiamento tra le classi. La direzione indica quale oggetto mantiene il riferimento (puntatore) verso l'altro.

- **Nelle Basi di Dati:** Le associazioni sono intrinsecamente *bidirezionali*. Una relazione tra due entità rappresenta un fatto logico che può essere interrogato partendo da entrambi i lati (es. tramite operatori di *Join* in SQL).

Dunque nel class diagram concettuale per database, si evitano solitamente le frecce di navigabilità. Una linea semplice indica che l'informazione è disponibile per l'interrogazione in modo simmetrico. La "direzionalità" fisica verrà stabilita solo nella fase di progettazione logica attraverso il posizionamento delle chiavi esterne.

1.1.3 Tipi Enumerativi (Enumerazioni)

Quando il dominio di un attributo è costituito da un insieme di valori costante, discreto e predefinito, si utilizza il costrutto della **Enumerazione**. Si tratta di un tipo di dato speciale che permette a un attributo di assumere solo uno dei valori elencati in una lista chiusa.

In UML, le enumerazioni sono rappresentate come classi con lo stereotipo «enumeration». Non hanno attributi propri, ma contengono un elenco di valori letterali che rappresentano le possibili istanze del tipo enumerativo.



A differenza dei diagrammi in OOP, le enumerazioni **non sono classi**, ma semplicemente un tipo di dato speciale; **non hanno associazioni con le classi**.

1.1.4 Classi di Associazione

Nella modellazione concettuale, capita spesso che un'associazione tra due classi possieda delle proprietà proprie. Se un attributo non descrive intrinsecamente né la classe A né la classe B, ma esiste solo in funzione della loro relazione, ci troviamo di fronte alla necessità di una **Classe di Associazione**.

Si consideri, ad esempio, il caso classico di un sistema universitario: uno **Studente** sostiene un **Corso**. Vogliamo memorizzare il *voto* conseguito.

- Il *voto* non è un attributo dello **Studente** (egli ha voti diversi per corsi diversi).
- Il *voto* non è un attributo del **Corso** (il corso ha voti diversi per studenti diversi).

In un diagramma standard, saremmo costretti a omettere il voto o a inserirlo forzatamente in una delle due classi, creando un errore concettuale e ridondanza.

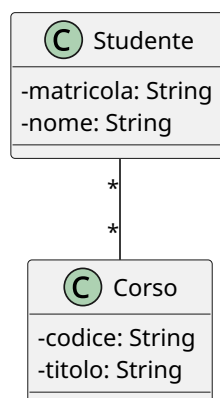


Figura 1.1: Associazione Studente-Corso.

L'UML risolve il problema permettendo a un'associazione di avere una propria classe collegata. Questa classe ha un doppio valore: è sia un legame logico tra le istanze, sia un contenitore di attributi.

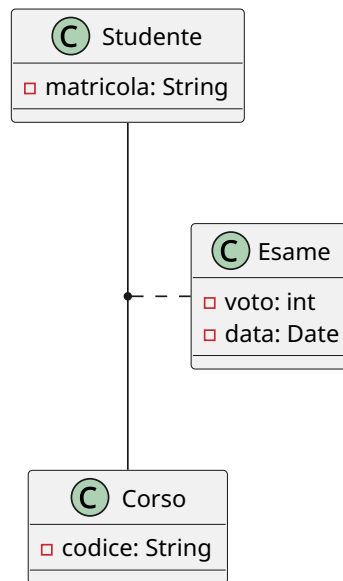


Figura 1.2: Studente-Corso con classe di associazione Esame.

Sintatticamente, si rappresenta con un rettangolo di classe collegato alla linea di associazione tramite una **linea tratteggiata**. Essa eredita implicitamente le "chiavi" di entrambe le classi coinvolte.

In molte fasi di progettazione logica, o per chiarezza di implementazione, la classe di associazione viene "esplicitata" (o reificata). La classe di associazione diventa una classe intermedia a tutti gli effetti, e l'associazione multi-a-molti originale si trasforma in due associazioni uno-a-molti.

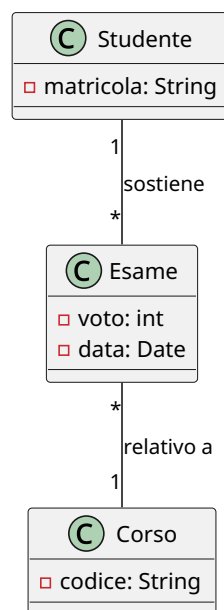


Figura 1.3: Reificazione della classe di associazione Esame.

L'esplicitazione è spesso necessaria perché molti strumenti di modellazione o linguaggi di programmazione non supportano nativamente il costrutto della classe di associazione. In questo caso, la classe intermedia (es. **Esame**) funge da "ponte" tra le due entità principali.

Tip

La classe di associazione è significativa solo su associazioni *multi-a-multi*, ma occasionalmente si usa anche per altri tipi di associazioni.

1.1.5 Grado di un'Associazione e Relazioni Ternarie

Il **grado** di un'associazione è definito come il numero di classi coinvolte nel legame logico.

- **Associazioni Binarie (Grado 2):** Collegano due classi. Costituiscono la quasi totalità delle relazioni in un database.
- **Associazioni N-arie (Grado $N \geq 3$):** Collegano tre o più classi contemporaneamente. Il caso più comune è l'associazione **ternaria**.

Un'associazione ternaria si rende necessaria quando il legame logico esiste solo se si considerano tre entità simultaneamente; la scomposizione in relazioni binarie separate causerebbe una perdita irreparabile di informazione semantica. In UML si rappresenta graficamente interponendo un **rombo** tra le linee che collegano le tre classi.

Si consideri, ad esempio, uno scenario di logistica industriale in cui sono coinvolte tre classi: **Fornitore**, **Prodotto** e **Progetto** (cantiere di destinazione). L'atto del rifornimento è un evento atomico: il *Fornitore A* fornisce il *Prodotto X* specificamente per il *Progetto Y*. Se provassimo a mappare questo dominio con tre relazioni binarie (*Fornitore-Prodotto*, *Prodotto-Progetto*, *Fornitore-Progetto*), sapremmo quali prodotti vende un fornitore e quali prodotti usa un progetto, ma non sapremmo mai **quale specifico fornitore ha portato quel determinato prodotto a quel singolo progetto**.

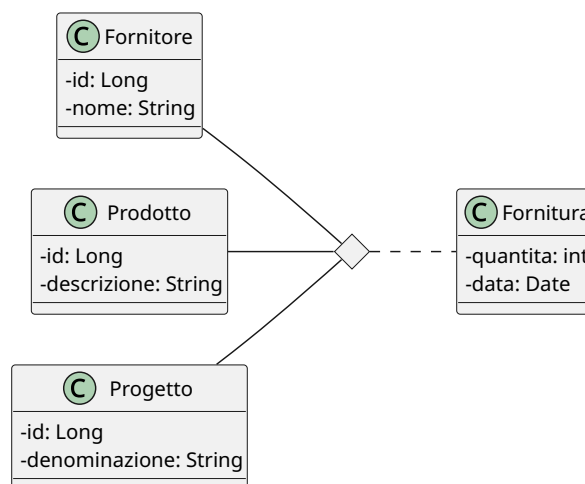


Figura 1.4: Esempio associazione ternaria con classe di associazione.

Interpretazione di una associazione n-aria

Per comprendere a fondo l'associazione ternaria, si può pensare che una delle tre classi non si colleghi singolarmente alle altre due, ma alla loro **coppia**. Nel

🔦 nostro esempio, il **Progetto** non è legato separatamente a un **Prodotto** e a un **Fornitore**, ma è associato stabilmente alla coppia (*Prodotto*, *Fornitore*), intesa come specifica combinazione. Analogamente per le altre due classi.

Poiché i DBMS relazionali e i framework ORM (come JPA/Hibernate) non supportano le relazioni native a tre vie, si deve **esplicitare** (o reificare) l'associazione prima dell'implementazione.

La reificazione trasforma il rombo dell'associazione in una classe vera e propria, che chiameremo **AssegnazioneFornitura**. L'associazione ternaria originale viene così frantumata in tre associazioni binarie distinte di tipo **1-a-Molti**:

- Da **Fornitore** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)
- Da **Prodotto** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)
- Da **Progetto** a **AssegnazioneFornitura** (Cardinalità $1 \rightarrow *$)

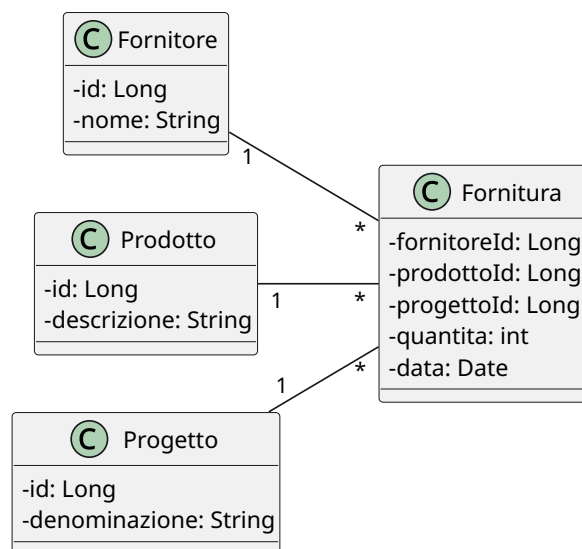


Figura 1.5: Reificazione dell'associazione ternaria.

Nella realtà, gli eventi n-ari catturano quasi sempre proprietà quantitative. Riprendendo l'esempio precedente, l'atto della fornitura genera intrinsecamente una *quantità* di pezzi consegnati e una *data* di consegna. Questi attributi non appartengono a nessuna delle tre classi isolate, ma alla loro combinazione. In UML si modella una **Classe di Associazione** (es. **Fornitura**) collegata tramite una linea tratteggiata al rombo dell'associazione ternaria.

Il processo di esplicitazione in questo scenario è immediato: la classe di associazione **Fornitura** viene promossa a classe centrale del cluster, inglobando gli attributi **quantità** e **data**. Le tre classi originarie si collegano a essa tramite relazioni binarie dirette.

⚠ La trappola delle cardinalità ternarie

Nelle associazioni ternarie UML, la cardinalità posta vicino a una classe (es. **Progetto**) indica quante istanze di quella classe possono combinarsi con una **coppia fissa** di istanze delle altre due classi (**Fornitore** e **Prodotto**). Questa logica è controintuitiva e differisce dai diagrammi ER classici (notazione di Chen), inducendo spesso i progettisti in errore. Sostituire subito la ternaria con una classe



esplicitata azzera ogni ambiguità interpretativa.

1.1.6 Aggregazione e Composizione

Nello sviluppo dei Class Diagrams, le associazioni standard indicano un legame logico simmetrico. Tuttavia, per modellare scenari in cui esiste un rapporto di possesso o di subordinazione strutturale, l'UML introduce le **relazioni tutto-parte** (part-whole relationships). Queste si dividono in due varianti fondamentali, distinte in base al vincolo sul ciclo di vita degli oggetti coinvolti: l'aggregazione e la composizione.

L'**aggregazione** rappresenta una relazione in cui un oggetto “Tutto” (aggregante) contiene o colleziona uno o più oggetti “Parte” (aggregati), ma questi ultimi mantengono un'esistenza autonoma all'interno del dominio. Si tratta di un legame “*has-a*” ad accoppiamento debole.

- **Sintassi UML:** Si rappresenta graficamente con una linea che termina con un **rombo vuoto** (bianco) posizionato sul lato della classe “Tutto”.
- **Ciclo di vita:** La distruzione dell'oggetto aggregante **non** comporta la distruzione degli oggetti aggregati.

Esempio 1.2 (Università e Professore). Si consideri la classe **Università** (il Tutto) e la classe **Professore** (la Parte). Un'università raccoglie un insieme di docenti. Tuttavia, se l'università dovesse cessare la sua attività, i singoli professori non verrebbero eliminati dal sistema: essi continuerebbero ad esistere come entità autonome e potrebbero essere associati ad altri atenei.

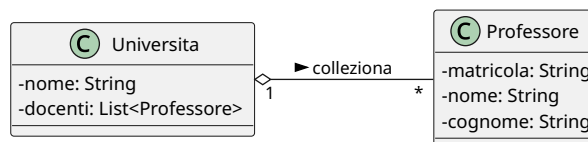


Figura 1.6: Esempio di aggregazione

La **composizione** è una forma di aggregazione più stringente, esclusiva e gerarchica. In questo scenario, l'oggetto “Parte” è una componente indissolubile del “Tutto”. Una parte può appartenere a un solo tutto alla volta e non ha alcuna autonomia esistenziale.

- **Sintassi UML:** Si rappresenta con una linea che termina con un **rombo pieno** (nero) posizionato sul lato della classe “Tutto”.
- **Ciclo di vita:** Il ciclo di vita delle parti è totalmente subordinato a quello del tutto. Di conseguenza, la distruzione dell'oggetto composto (Tutto) determina l'istanza di **distruzione automatica** di tutte le sue parti. La cardinalità dal lato del tutto è rigorosamente 1 o 0..1.

Esempio 1.3 (Ordine ed Elemento d'Ordine). Si consideri un sistema di e-commerce con le classi **Ordine** (il Tutto) e **RigaOrdine** (la Parte, che descrive il prodotto e la quantità). Una riga d'ordine ha significato logico solo se contestualizzata all'interno dello specifico acquisto. Se l'utente decide di eliminare l'intero **Ordine**, il sistema deve rimuovere a cascata tutte le relative istanze di **RigaOrdine**.

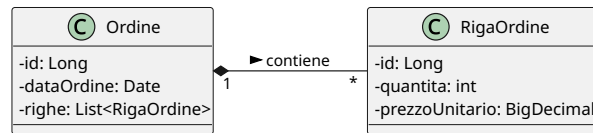


Figura 1.7: Esempio di composizione

🔍 L'aggregazione non ha alcuna necessità tecnica stringente: una semplice associazione fa esattamente lo stesso identico lavoro. A livello di tabelle, sia un'associazione generica sia un'aggregazione si tradurranno nello stesso modo, la differenza è puramente semantica e documentale, non implementativa. Alcuni autorevoli ingegneri del software (tra cui Martin Fowler, uno dei padri dell'architettura software moderna) sostengono da anni che l'aggregazione UML sia fundamentalmente inutile perché la linea di demarcazione con l'associazione standard è troppo sfumata e crea più ambiguità che benefici. Fowler la definisce ironicamente "un effetto placebo grafico".

Il caso della composizione è invece più significativo, perché il diagramma fornisce un'informazione in più: la parte non può esistere senza il tutto.

1.1.7 Generalizzazioni/Specializzazioni

L'ereditarietà modella una relazione di tipo “IS-A” (è-un) tra una classe più generale, detta **Generalizzazione** (o Superclasse, per analogia con i contesti OOP), e una o più classi più specifiche, dette **Specializzazioni** (o Sottoclassi).

Le specializzazioni ereditano automaticamente tutti gli attributi, i ruoli e le associazioni della generalizzazione, senza necessità di ridefinirli, e possono estendere il modello introducendo attributi o associazioni propri.

Sintassi UML: Si rappresenta graficamente con una linea che termina con una **freccia con la punta triangolare vuota** rivolta verso la generalizzazione.

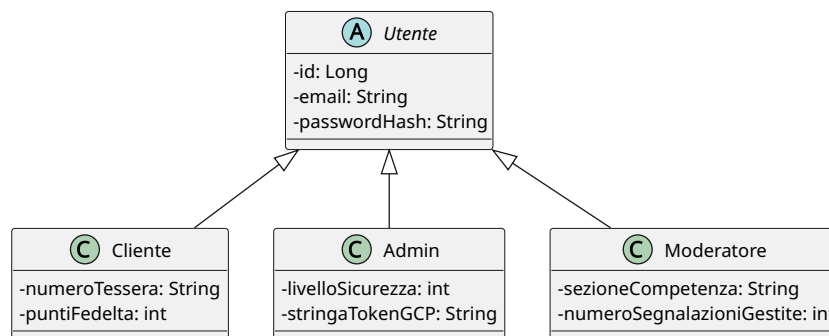


Figura 1.8: Esempio di ereditarietà

È fondamentale comprendere che **l'ereditarietà non esiste nel modello relazionale delle basi di dati** (o almeno, non secondo il modello relazionale dei dati che noi studieremo). I database relazionali non possiedono strutture gerarchiche native; essi conoscono esclusivamente il concetto di tabella (relazione), che è per sua natura una struttura “piatta” e bidimensionale.

La generalizzazione appartiene quindi in modo esclusivo al livello della **progettazione concettuale**. In questa fase, lo scopo del Class Diagram è modellare il dominio reale nel modo più fedele e naturale possibile, catturandone la semantica e i vincoli logici

(ovvero i rapporti di sotto-tipo/sopra-tipo) ed eliminando qualsiasi vincolo tecnologico. Il problema di come “appiattare” questa gerarchia per adattarla a un database relazionale sarà l’obiettivo centrale della successiva fase di progettazione logica.

Una generalizzazione può essere classificata combinando due tipi di vincoli ortogonali, utili a descrivere la natura delle istanze nel dominio reale:

1. **Completezza (Totale vs Parziale):**

- **Totale (Complete):** Ogni istanza della superclasse deve obbligatoriamente appartenere ad almeno una delle subclassi. Non esistono elementi “generici” (corrisponde al concetto di *Classe Astratta* in OOP).
- **Parziale (Incomplete):** Un’istanza della superclasse può esistere senza dover per forza appartenere a una delle subclassi catalogate.

2. **Disgiunzione (Disgiunta vs Sovrapposta):**

- **Disgiunta (Disjoint):** Un’istanza della superclasse può appartenere al massimo a **una sola** delle subclassi. Le sottoclassi sono mutuamente esclusive.
- **Sovrapposta (Overlapping):** Una singola istanza della superclasse può appartenere contemporaneamente a più subclassi distinte.

Ci sono dunque quattro combinazioni possibili. Nel diagramma UML, i vincoli della generalizzazione si indicano inserendo un’etichetta testuale racchiusa tra parentesi graffe (*Generalization Set*) in prossimità delle linee di ereditarietà, specificando la coppia di proprietà (es: {complete, overlapping}).

Esempio 1.4 (Esempi di Vincoli). Si considerino i seguenti scenari aziendali:

- **Totale / Disgiunta:** La superclasse *ContoBancario* ha come subclassi *ContoCorrente* e *ContoDeposito*. Un conto deve essere obbligatoriamente di un tipo o dell’altro, e non può essere entrambe le cose contemporaneamente.
- **Parziale / Sovrapposta:** La superclasse *Persona* ha come subclassi *Studente* e *Dipendente*. All’interno dell’anagrafica universitaria, una persona può essere solo un cittadino esterno (parziale), oppure può essere contemporaneamente uno studente che lavora per l’ateneo (sovrapposta).

1.2 La Documentazione di Supporto: I Dizionari dei Dati

Il Class Diagram, pur essendo uno strumento visivo formidabile, non è espressivamente sufficiente a catturare l’intera semantica di un dominio applicativo. Elementi come il significato preciso di un attributo, il contesto d’uso di un’associazione o le regole aziendali complesse (*Business Rules*) non possono essere espressi tramite nodi e archi.

La progettazione concettuale si completa quindi attraverso i **Dizionari dei Dati**, estensioni testuali strutturate che definiscono formalmente ogni componente dello schema concettuale. Questi documenti rappresentano la specifica dei requisiti su cui si baserà lo sviluppo dello strato di persistenza e della logica di business.

I dizionari dei dati sono:

- **Dizionario delle Classi:** descrive in modo univoco ogni entità del sistema. Si presenta come una tabella in cui ogni riga rappresenta una classe. Per ogni classe vengono forniti il nome (prima colonna), una descrizione in linguaggio naturale per evitare ambiguità terminologiche (seconda colonna), nome, tipo e descrizione di ciascun attributo (terza colonna).
- **Dizionario delle Associazioni:** si presenta come una tabella analoga alla precedente, in cui ogni riga rappresenta un’associazione. Per ogni associazione vengono

forniti il nome (prima colonna), descrizione (seconda colonna), le classi coinvolte con ruolo, molteplicità e descrizione (terza colonna).

- **Dizionario dei Vincoli:** è il documento più critico. I **Vincoli di Integrità Esogeni** (o regole di business) sono restrizioni sul dominio dei dati che **non possono** essere espresse graficamente nel Class Diagram tramite la normale sintassi UML. Mentre una cardinalità $1..*$ esprime un vincolo strutturale grafico, una regola del tipo “*Un cliente non può effettuare un ordine se il suo account è sospeso*” non ha un simbolo UML dedicato. Viene quindi codificata nel dizionario dei vincoli in linguaggio naturale o tramite espressioni logico-matematiche formali (come l’OCL - *Object Constraint Language*). Si presenta come una tabella a due colonne in cui ogni riga rappresenta un vincolo, nella prima colonna si inserisce il nome del vincolo oppure un suo identificatore univoco (es. BR - Business Rule), nella seconda colonna la sua descrizione in linguaggio naturale.

Esempio 1.5 (Catalogo dei Vincoli d’Integrità).

BR01 (Coerenza Temporale): La data di spedizione di un `Ordine` deve essere strettamente successiva o uguale alla sua data di creazione.

BR02 (Integrità Finanziaria): L’attributo `totale` di un `Ordine` deve essere matematicamente pari alla sommatoria del prodotto tra `quantita` e `prezzoUnitario` di tutte le istanze di `RigaOrdine` a esso collegate.

BR03 (Regola di Business): Un `Utente` con ruolo `Admin` non può associare a sé stesso un’istanza di `Ordine` in veste di `acquirente`.



Nel dizionario dei vincoli non vanno inseriti vincoli che sono già espressi nel class diagram.

I vincoli devono garantire la qualità interna dei dati nella base di dati.

Modello Relazionale e Progettazione Logica

2.1 Introduzione

Il passaggio dalla progettazione concettuale alla progettazione logica rappresenta la transizione dal *cosa* deve essere rappresentato al *come* organizzare concretamente i dati all'interno di un sistema di gestione di basi di dati (DBMS). Questo processo richiede l'adozione di uno specifico **modello dei dati**.

Sebbene la storia dell'informatica abbia visto l'evoluzione di diversi paradigmi — come il modello gerarchico, quello reticolare e quello orientato agli oggetti —, la nostra trattazione si concentrerà sul **modello relazionale**, che costituisce lo standard industriale prevalente. Vale la pena notare che i DBMS moderni più diffusi (come PostgreSQL e Oracle) adottano un'evoluzione di questo approccio, denominata **modello relazionale a oggetti**, la quale estende le fondamenta del modello relazionale puro integrando costrutti tipici del paradigma OOP.

Uno dei principali vantaggi nell'aver adottato i **Class Diagrams UML** nella fase concettuale risiede nella loro assoluta indipendenza tecnologica: essi si adattano teoricamente a qualunque modello logico finale. Di contro, questa spiccata espressività introduce un inevitabile **disallineamento semantico** (*impedance mismatch*) quando l'obiettivo è la traduzione in uno schema relazionale puro.

Nel passaggio alla progettazione logica relazionale, le principali sfide strutturali da affrontare saranno dettate dai rigidi vincoli di atomicità e piattezza del dato tipici delle tabelle:

- **Attributi strutturati:** Il modello relazionale esige che i valori degli attributi siano atomici e non strutturati. Non è quindi possibile nidificare record, oggetti o tipi di dato complessi all'interno di una colonna.
- **Attributi a valore multiplo (multivalore):** Ogni cella di una tabella relazionale può contenere esclusivamente un singolo valore atomico, vietando l'uso nativo di collezioni, array, liste o set all'interno di un singolo attributo.
- **Assenza di ereditarietà:** Il modello relazionale puro non supporta nativamente le gerarchie di generalizzazione-specializzazione, costringendo il progettista ad applicare strategie di “appiattimento” o di scomposizione strutturale.

2.2 Concetti Fondamentali

Il **modello relazionale** si basa sul concetto matematico di relazione. Questo modello rappresenta la struttura logica e piatta su cui verranno mappati i dati dell'applicazione. Per formalizzare rigorosamente il modello, è necessario definire i suoi tre componenti atomici: i domini, lo schema e la relazione stessa.

Un **dominio** (indicato con D) è un insieme di valori atomici, ovvero non ulteriormente scomponibili. Ogni dominio è caratterizzato da un nome e da un tipo di dato (es: `int`, `varchar`, `date`).

Osservazione 2.1 (Il Tipo di Dato come Dominio Implicito). Nella transizione dalla teoria all'ingegneria del software, il **tipo di dato** (es. `int`, `varchar`, `date`) definisce l'insieme universo di partenza, agendo come **dominio massimo implicito** dell'attributo.

Eventuali restrizioni logiche (come un *range* di valori validi per un voto universitario o un'età) si configurano come un **sottoinsieme** di tale dominio base. Vedremo in seguito come vengono implementate limitazioni specifiche.

Uno **schema relazionale** (o schema di tabella) definisce la struttura logica dei dati. Esso è costituito da un nome R e da un insieme di attributi $A_1, A_2, \dots, A_n$, a ciascuno dei quali è associato un dominio D_i , che denoteremo anche con $dom(A_i)$.

Lo schema viene denotato formalmente come:

$$R(A_1 : D_1, A_2 : D_2, \dots, A_n : D_n)$$

Lo schema relazionale corrisponde dunque esattamente al concetto di **Classe**. Esso definisce la struttura dei dati, i nomi dei campi e i rispettivi tipi, agendo come un “blueprint” statico.

Matematicamente, date le premesse precedenti, definiamo il dominio totale dello schema come il prodotto cartesiano dei domini dei singoli attributi:

$$D_{tot} = D_1 \times D_2 \times \dots \times D_n$$

Una **relazione** r sullo schema R è un sottoinsieme finito del prodotto cartesiano dei domini degli attributi:

$$r \subseteq D_1 \times D_2 \times \dots \times D_n$$

Gli elementi che compongono questo sottoinsieme sono detti **tuple** (o *n*-uple). Una *tuple* è un vettore di valori $(v_1, v_2, \dots, v_n)$ tale che ogni $v_i \in D_i$.



La corrispondenza con OOP è la seguente:

- Una **Tupla** corrisponde al singolo **Oggetto** (una istanza della classe contenente dati reali).
- La **Relazione** corrisponde all'insieme di tutti gli oggetti di quella classe **attualmente persistiti** nel database in un determinato istante temporale (lo stato corrente della tabella).

Esempio 2.1 (Formalizzazione di un'Anagrafica). Definiamo lo schema relazionale per la classe **Studente**:

$$STUDENTE(\text{matricola} : \text{String}, \text{nome} : \text{String}, \text{età} : \text{Integer})$$

Il prodotto cartesiano di questi tre domini genererebbe infinite combinazioni (comprese matricole inesistenti accoppiate a nomi casuali ed età assurde).

La **relazione** $r(STUDENTE)$ conterrà invece solo le tuple degli studenti reali inseriti nel sistema:

$$r = \{('654321', 'Giovanni', 22), ('123456', 'Mario', 24)\}$$

Osservazione 2.2. È significativo evidenziare che una relazione è *un insieme*, dunque non ammette duplicati e non rappresenta una struttura ordinata.

In sintesi:

Nel passaggio dalla progettazione concettuale alla modellazione logica, i concetti del paradigma orientato agli oggetti (*OOP*) e del Class Diagram si traducono come segue:

- **Classi** → **Schemi Relazionali**: Le classi si traducono in schemi di relazione, identificati da un nome (corrispondente al nome della classe) e da un insieme di attributi.
- **Attributi** → **Domini**: A ciascun attributo dello schema è associato un **dominio**, ovvero l'insieme di tutti i possibili valori atomici che quell'attributo può assumere (equivalente al tipo di dato in programmazione).
- **Istanze** → **Relazione (Stato)**: Una **relazione** è un sottoinsieme discreto e finito del dominio totale (il prodotto cartesiano di tutti i singoli domini). Essa rappresenta l'insieme delle tuple (istanze o oggetti) effettivamente memorizzate e persistite nel sistema in un dato momento temporale.

2.3 Le Chiavi

Fino ad ora abbiamo analizzato come tradurre classi e attributi del Class Diagram nel modello relazionale, lasciando in sospeso la traduzione delle associazioni. Per poter modellare le relazioni tra tabelle, è prima necessario introdurre il concetto di chiave.

Sia $R(A_1, A_2, \dots, A_n)$ uno schema di relazione. Un sottoinsieme di attributi $SK \subseteq \{A_1, A_2, \dots, A_n\}$ si definisce **superchiave** se i suoi valori identificano in modo univoco ogni singola tupla all'interno di qualsiasi istanza valida della relazione.

Esempio 2.2 (Superchiavi di un'Anagrafica). Si consideri lo schema:

STUDENTE(CF, matricola, nome, cognome, dataNascita)

I singoletti $\{CF\}$ e $\{matricola\}$ sono superchiavi. Di conseguenza, **qualsiasi sottoinsieme che li contenga** (es: $\{CF, nome\}$ o $\{matricola, cognome\}$) è a sua volta una superchiave.

Osservazione 2.3 (Ipotesi di Esistenza). Nella fase di progettazione concettuale si assume l'ipotesi che ogni istanza di una classe sia distinguibile dalle altre. Questa proprietà ci garantisce che **esiste sempre almeno una superchiave** per ogni schema relazionale: nella peggiore delle ipotesi (assenza di identificatori naturali), la superchiave coinciderà con l'insieme di tutti gli attributi dello schema.

Un sottoinsieme di attributi K è una **chiave candidata** se è una **superchiave minimale**. Dire che una superchiave è *minimale* significa che è **irriducibile**: eliminando un qualsiasi attributo da K , l'insieme rimanente perde la proprietà di univocità e cessa di essere una superchiave.

Questo **non** significa che debba avere il numero minimo di attributi in assoluto (cardinalità minima). Se nello schema coesistono una superchiave irriducibile di cardinalità 1 e una di cardinalità 2, sono entrambe, con pari dignità, chiavi candidate.

È evidente che una superchiave composta da un unico attributo (un singoletto) è di fatto e necessariamente una chiave candidata.

Esempio 2.3 (Chiave Candidata Composta). Si consideri lo schema che mappa la classe degli esami universitari:

ESAME(matricola, corso, voto, data)

In questo scenario, il sottoinsieme {matricola, corso} rappresenta una chiave candidata (di fatto, è l'unica chiave candidata in questo caso). Essa è minimale perché se eliminassimo **corso**, la sola **matricola** non garantirebbe l'univocità (uno studente può sostenere più esami); viceversa, eliminando **matricola**, il solo **corso** conterrebbe i voti di tutti gli studenti.

Poiché in uno schema relazionale possono coesistere più chiavi candidate, il progettista deve sceglierne una da eleggere come identificatore ufficiale delle tuple. La chiave candidata scelta prende il nome di **Chiave Primaria** (*Primary Key* - PK).

La selezione della chiave primaria è una decisione puramente ingegneristica e architeturale. Nello schema relazionale scritto, la chiave primaria si indica **sottolineando** gli attributi che la compongono.

Esempio 2.4 (Scelta della PK). Riprendendo lo schema **STUDENTE**, le chiavi candidate sono CF e **matricola**. Sceglieremo come chiave primaria la **matricola**:

STUDENTE(CF, matricola, nome, cognome)

La **Chiave Esterna** (*Foreign Key* - FK) è lo strumento fondamentale del modello relazionale per tradurre le associazioni.

Siano R_1 e R_2 due schemi relazionali (non necessariamente distinti). Un insieme di attributi FK di R_1 è una chiave esterna che punta a R_2 se:

1. Gli attributi in FK sono in corrispondenza biunivoca con gli attributi che compongono la **chiave primaria** (PK) di R_2 .
2. I domini degli attributi corrispondenti sono identici ($dom(FK_i) = dom(PK_i)$).

Il vincolo impone che per ogni tupla t_1 nell'istanza corrente di R_1 , il valore di $t_1[FK]$ deve essere nullo (NULL), oppure coincidere esattamente con il valore $t_2[PK]$ di **una tupla** t_2 **attualmente esistente** nell'istanza di R_2 .

Nello schema testuale, la presenza di una chiave esterna si denota convenzionalmente con una **doppia sottolineatura**.

2.4 Traduzione delle Associazioni

2.4.1 Associazioni 1 a 1

Un'associazione **1 a 1** tra due classi A e B indica che a un'istanza di A corrisponde al massimo un'istanza di B , e viceversa. Nel modello relazionale, questo legame si traduce introducendo una **Chiave Esterna (FK)** in uno dei due schemi che punta alla Chiave Primaria (PK) dell'altro.

Esempio 2.5 (Associazione esattamente 1 a 1). Si consideri un'associazione in cui la partecipazione è rigida e obbligatoria per entrambe le classi ($1..1 \leftrightarrow 1..1$), ad esempio tra l'entità **STUDENTE**(matricola, nome, cognome) e il suo **FASCICOLO**(codice, dataCreazione). Ogni studente possiede esattamente un fascicolo e ogni fascicolo appartiene esattamente a uno studente. Traduciamo il modello aggiungendo la chiave esterna nello schema del fascicolo:

- STUDENTE(matricola, nome, cognome)
- FASCICOLO(codice, dataCreazione, matricolaStudente)

Affinché la relazione rimanga rigorosamente 1 a 1, non è sufficiente definire la chiave esterna. Se ci limitassimo a questo, la tabella che ospita la FK potrebbe avere più tuple che puntano alla stessa PK della tabella bersaglio, trasformando di fatto la relazione in una 1 a N.

Riprendendo l'esempio precedente, se l'attributo **studente** nello schema **FASCICOLO** fosse definito unicamente come chiave esterna, nulla impedirebbe al database di registrare due tuple distinte di fascicoli (es. codice 'F01' e 'F02') aventi entrambe il valore **studente** = '12345'. Questo corromperebbe la semantica concettuale, asserendo che la matricola 12345 possiede *molti* fascicoli.

Per garantire la biunivocità dell'associazione, è obbligatorio imporre un **vincolo di univocità** sugli attributi che compongono la chiave esterna. Matematicamente, se lo schema R_1 contiene la chiave esterna FK verso R_2 , deve valere che per ogni coppia di tuple distinte $t_x, t_y \in r(R_1)$:

$$t_x[FK] \neq t_y[FK] \quad (\text{escludendo i valori NULL})$$

Q Perché usare una sola Chiave Esterna?

Vista la forte simmetria di un'associazione 1 a 1, potrebbe sorgere spontaneo il dubbio: perché non posizionare una chiave esterna in *entrambi* gli schemi (es. aggiungere **fascicolo** dentro **STUDENTE** e **studente** dentro **FASCICOLO**)? La teoria relazionale esclude questa pratica per tre ragioni fondamentali:

1. **Ridondanza e Rischio di Anomalie:** Il legame logico verrebbe salvato due volte. Se per errore un sistema aggiornasse un lato senza allineare l'altro (es. lo **Studente** punta al **Fascicolo A**, ma il **Fascicolo A** punta a un altro **Studente**), il database entrerebbe in uno stato incoerente.
2. **Bidirezionalità Intrinseca:** Come già stabilito, la presenza di una singola chiave esterna è matematicamente sufficiente per stabilire un'uguaglianza logica tra le tuple. Il motore del database può navigare il legame in entrambe le direzioni senza bisogno di puntatori doppi.
3. **Dipendenza Circolare (Deadlock di Inserimento):** Poiché in questo esempio la partecipazione è obbligatoria da ambo i lati, le rispettive chiavi esterne dovrebbero avere un vincolo NOT NULL. Tuttavia, questo creerebbe un paradosso logico: non si potrebbe inserire uno studente nel database perché manca il suo fascicolo, e non si potrebbe inserire il fascicolo perché manca lo studente a cui associarlo. Inserire la chiave esterna in un solo schema spezza questo circolo vizioso.

Osservazione 2.4. In alcuni casi estremi di forte coesione, le due tabelle possono persino essere fuse in un unico schema relazionale. Approfondiremo questo aspetto nel prossimo paragrafo.

Nell'esempio precedente si aveva una situazione di relazione esattamente 1 a 1 e la scelta dello schema in cui inserire la chiave esterna era indifferente. In altri casi, la scelta di quale delle due tabelle debba ospitare la FK dipende dalle **cardinalità minime** (l'opzionalità o obbligatorietà dell'associazione): se la classe A partecipa in modo opzionale (0..1) e la classe B in modo obbligatorio (1..1), la chiave esterna **deve essere inserita nello schema B**. In questo modo, la FK in B potrà essere dichiarata NOT NULL (vincolo di esistenza garantito). Se la mettessimo in A , saremmo costretti ad ammettere valori NULL per le istanze di A che non partecipano all'associazione, sprecando spazio e riducendo l'eleganza relazionale.

Esempio 2.6 (Direzione di un Dipartimento). Si consideri l'associazione 1 a 1 tra `Professore` (0..1) e `Dipartimento` (1..1): un professore può dirigere al massimo un dipartimento, ma un dipartimento deve avere esattamente un direttore.

Seguendo la regola, posizioniamo la chiave esterna nel `Dipartimento`:

- `PROFESSORE`(codiceDocente, nome, cognome)
- `DIPARTIMENTO`(codice, nome, codiceDirettore)

Vincoli su `DIPARTIMENTO.codiceDirettore`:

- **Foreign Key:** punta a `PROFESSORE.codiceDocente`
- **UNIQUE:** per garantire che lo stesso professore non diriga due dipartimenti (vincolo 1:1).
- **NOT NULL:** per garantire che ogni dipartimento abbia un direttore (cardinalità minima 1).

Q Controllo Applicativo vs Vincolo Strutturale (UNIQUE)

In alcuni contesti didattici o in framework di sviluppo specifici, è possibile omettere la dichiarazione esplicita del vincolo **UNIQUE** per le associazioni 1 a 1.

In questi scenari, si assume un'ipotesi di **coerenza applicativa**: si dà per scontato che sia la logica del software sovrastante (es. i controlli nel codice backend) a impedire l'inserimento di tuple multiple, in accordo con la natura del dominio ("non può mai capitare che l'entità sia associata a molti").

Tuttavia, dal punto di vista della solidità ingegneristica e dell'integrità fisica dei dati (*defensive programming*), l'omissione del vincolo **UNIQUE** rende il database strutturalmente identico a un'associazione 1 a Molti. Fissare il vincolo a livello di DBMS rimane la pratica raccomandata per garantire che i dati non vengano mai corrotti, indipendentemente da eventuali bug del software applicativo.

2.4.2 Associazioni 1 a Molti (1:N)

Nel modello relazionale, questa associazione si traduce secondo una regola fissa e inderogabile, dettata dal vincolo di atomicità degli attributi (assenza di attributi multivalore): la **Chiave Esterna (FK) deve essere sempre inserita nello schema dal lato "Molti" (B)**, e punterà alla Chiave Primaria (PK) dello schema dal lato "Uno" (A).

A differenza dell'associazione 1 a 1, in questo caso **non** si deve applicare alcun vincolo di univocità (**UNIQUE**) sulla chiave esterna. È infatti intrinseco nella natura dell'associazione 1:N che più tuple della relazione *B* possano possedere lo stesso valore per l'attributo FK, puntando così alla medesima istanza di *A*.

Esempio 2.7 (Impiegati in un Dipartimento). Si consideri l'associazione tra `Dipartimento` (1) e `Impiegato` (N). Un dipartimento ospita molti impiegati, ma un impiegato afferisce a un solo dipartimento. La chiave esterna viene posizionata nello schema `IMPIEGATO`:

- `DIPARTIMENTO`(codice, nome, sede)
- `IMPIEGATO`(matricola, nome, cognome, codiceDipartimento)

Osservazione 2.5 (Monodirezionalità Apparente). Anche in questo caso, la posizione fisica della chiave esterna genera una *monodirezionalità apparente*. Guardando lo schema, sembra che un impiegato "conosca" il proprio dipartimento, ma che il dipartimento sia ignaro dei propri impiegati (poiché non ha attributi che li referenziano).

Come già chiarito, questa è solo un'illusione strutturale: l'informazione è conservata intatta dalla logica dei valori. Il database è sempre in grado di ricavare l'elenco degli

impiegati di uno specifico dipartimento, semplicemente interrogando la tabella **IMPIEGATO** e filtrando le tuple che possiedono quel determinato `codiceDipartimento`.

2.4.3 Associazioni Molti a Molti (N:N)

Nei casi precedenti (1:1 e 1:N), la traduzione avveniva inserendo una chiave esterna direttamente in uno dei due schemi coinvolti. Se provassimo ad applicare questa logica a un'associazione N:N, violeremmo il vincolo fondamentale del modello relazionale:

- Inserire la chiave esterna di **CORSO** nello schema **STUDENTE** costringerebbe una tupla studente a contenere un attributo **multivalore** (una lista di codici di corsi).
- Viceversa, inserire la chiave esterna di **STUDENTE** in **CORSO** creerebbe una lista di matricole dentro la tupla del corso.

Poiché il modello relazionale ammette esclusivamente valori atomici, l'uso di chiavi esterne dirette è matematicamente impossibile.

Per tradurre un'associazione Molti a Molti, la teoria relazionale impone la creazione di un **terzo schema di relazione autonomo**, comunemente chiamato *tabella di giunzione*, *tabella ponte* o *relazione associativa*.

Questa nuova tabella conterrà esclusivamente (almeno nella sua forma base) le chiavi esterne che puntano alle chiavi primarie dei due schemi originari. La **Chiave Primaria** di questa nuova tabella sarà composta dall'unione delle due chiavi esterne (chiave primaria composta).

Esempio 2.8 (Studenti e Corsi). Si consideri l'associazione N:N tra **STUDENTE** e **CORSO**. Creiamo la tabella ponte **FREQUENZA**:

- **STUDENTE**(matricola, nome)
- **CORSO**(codice, titolo, cfu)
- **FREQUENZA**(matricolaStudente, codiceCorso)

Vincoli sulla tabella **FREQUENZA**:

- **Primary Key:** È composta dalla coppia (matricolaStudente, codiceCorso). Questo garantisce che uno studente non possa essere iscritto due volte allo stesso identico corso.
- **Foreign Key 1:** matricolaStudente punta a **STUDENTE**.matricola.
- **Foreign Key 2:** codiceCorso punta a **CORSO**.codice.

Osservazione 2.6 (Delegazione della Complessità). Dal punto di vista della progettazione (struttura statica), il pattern della tabella di giunzione rende la risoluzione delle associazioni N:N un processo banale e standardizzato. La reale complessità viene delegata alla fase operativa (interrogazione dei dati). Per estrarre, ad esempio, i nomi degli studenti e i titoli dei corsi da loro frequentati, il DBMS dovrà eseguire computazionalmente onerosi **JOIN multipli**, attraversando la tabella ponte per ricongiungere i dati disaccoppiati.

È chiaro che questo metodo per rendere le associazioni è del tutto generale e, in teoria, potrebbe essere usato per tutti i tipi di associazione; tuttavia, si tratta ovviamente di una cattiva pratica in virtù del costo computazionale che è stato appena discusso.

Classi di Associazione

Nel modellamento concettuale tramite Class Diagram UML, capita frequentemente che un'associazione molti a molti possieda a sua volta degli attributi (formalizzati tramite una

Classe di Associazione). Un esempio tipico è la relazione di superamento di un esame tra uno studente e un corso, la quale porta con sé attributi come il *voto*, la *data* e la *lode*.

La traduzione di questo costrutto nel modello relazionale segue fedelmente la regola dello schema ponte vista in precedenza. Gli attributi propri dell'associazione vengono semplicemente inseriti come **colonne aggiuntive all'interno della tabella di giunzione**.

Esempio 2.9 (Mappatura della Classe di Associazione Esame). Estendiamo l'esempio precedente trasformando lo schema ponte FREQUENZA in uno schema ESAME che ospiti anche i dati del superamento del corso:

- STUDENTE(matricola, nome, cognome)
- CORSO(codice, titolo, cfu)
- ESAME(matricolaStudente, codiceCorso, voto, data, lode)

Nello schema ESAME, la chiave primaria rimane la coppia composta (matricolaStudente, codiceCorso), il che modella il vincolo per cui uno studente può registrare al massimo un voto per un determinato corso. Gli attributi *voto*, *data* e *lode* diventano attributi non chiave della tabella di giunzione.

Associazioni n-arie

La logica della tabella di giunzione, introdotta per le associazioni binarie Molti a Molti, scala in modo naturale e identico per le **associazioni n-arie** (con $n > 2$).

Quando nel diagramma concettuale un'associazione coinvolge tre o più classi (ad esempio un'associazione ternaria), la traduzione nel modello relazionale richiede la creazione di un unico **schema ponte** dedicato. Questo schema assolverà il compito di legare tutte le entità simultaneamente e conterrà:

- Una **Chiave Esterna (FK)** per ciascuna delle n entità partecipanti, che punta alla rispettiva chiave primaria.
- Una **Chiave Primaria (PK) composta** dall'unione di tutte le chiavi esterne coinvolte (salvo casi particolari in cui vincoli di cardinalità massima pari a 1 su specifici rami consentano di escludere alcune FK dalla chiave primaria).
- Gli eventuali attributi dell'associazione (o della Classe di Associazione), inseriti semplicemente come colonne non chiave.

Esempio 2.10 (Associazione Ternaria di Fornitura). Si consideri un'associazione ternaria che modella la fornitura di materiali aziendali: un determinato **Fornitore** vende uno specifico **Prodotto** destinato a un particolare **Progetto**, in una certa **quantità**.

La traduzione genera tre schemi base e un quarto schema ponte per l'associazione ternaria:

- FORNITORE(partitaIva, nomeAzienda)
- PRODOTTO(codiceProdotto, descrizione)
- PROGETTO(idProgetto, budget)
- FORNITURA(partitaIva, codiceProdotto, idProgetto, quantità)

Nello schema FORNITURA, la chiave primaria è la tripla di attributi (partitaIva, codiceProdotto, idProgetto). L'attributo *quantità* è un attributo dell'associazione che dipende funzionalmente dall'intera tripla, in quanto esprime quanti pezzi di quel prodotto specifico sono stati comprati da quel fornitore per quel preciso progetto.

2.5 Ristrutturazione dello Schema Concettuale

Il Class Diagram realizzato durante la progettazione concettuale ha l'unico scopo di modellare la semantica del dominio applicativo, ignorando i limiti tecnologici. Tuttavia,

avvicinandosi alla progettazione logica, l'adozione dello specifico modello relazionale fa emergere vincoli strutturali e di performance che rendono il diagramma originario inefficiente o inapplicabile.

Prima di procedere alla traduzione in tabelle, è quindi necessaria una fase di **Ristrutturazione del Class Diagram**, durante la quale il modello viene riadattato e "sporcato" con dettagli implementativi. Questa fase si articola in diverse analisi sequenziali.

2.5.1 Analisi delle Chiavi: Chiavi Naturali vs Surrogate

La prima operazione consiste nel verificare gli identificatori (chiavi primarie) scelti per le classi. Nel modello concettuale, è prassi utilizzare come identificatori degli attributi già esistenti nel dominio della realtà, noti come **Chiavi Naturali**. Tuttavia, capita spesso che per identificare univocamente un'istanza nel mondo reale sia necessario unire molti attributi, creando una **chiave primaria composta** molto ingombrante.

L'uso di chiavi primarie formate da tre, quattro o più attributi genera due enormi problemi nel database relazionale:

1. **Propagazione delle Foreign Key:** Poiché le associazioni si traducono copiando la chiave primaria nella tabella collegata, una chiave di 4 attributi costringerà tutte le tabelle figlie ad avere 4 colonne aggiuntive solo per mantenere il legame. Questo allarga a dismisura le tabelle e spreca memoria.
2. **Costo Computazionale:** Gli indici sulle chiavi composite sono lenti da scorrere e pesanti da aggiornare, rendendo i JOIN estremamente inefficienti.

Per risolvere questo problema, si modifica il Class Diagram introducendo un nuovo attributo artificiale, privo di alcun significato nel dominio di business, chiamato **Chiave Surrogata** (o *Sintetica*). Generalmente si tratta di un numero intero auto-incrementante (es. `id`, `codice`) o di un UUID alfanumerico. Questo attributo viene eletto a nuova Chiave Primaria, sostituendo la chiave naturale composta.

Esempio 2.11 (Introduzione di una Chiave Surrogata). Si consideri la classe **Stanza** di un polo universitario. Nel mondo reale, una stanza è identificata univocamente dalla combinazione di tre attributi: `edificio`, `piano` e `numeroStanza`.

`STANZA(edificio, piano, numeroStanza, capienza)`

Se un corso dovesse essere assegnato a una stanza, lo schema **CORSO** dovrebbe ereditare tutte e tre le colonne come Foreign Key.

Ristrutturiamo lo schema introducendo una chiave surrogata `idStanza`:

`STANZA(idStanza, edificio, piano, numeroStanza, capienza)`

Ora, le tabelle collegate (come **CORSO**) dovranno importare solamente la colonna `idStanza` (un semplice intero) per stabilire l'associazione, ottimizzando drasticamente le prestazioni del database.

Attenzione alla perdita di integrità

Quando si sostituisce una chiave naturale con una chiave surrogata, il database userà il nuovo `id` per distinguere le righe. Questo crea un potenziale rischio: il database permetterà di inserire due righe con `id` diversi ma con gli **stessi identici dati naturali** (es. due stanze diverse assegnate allo stesso edificio, stesso piano e



stesso numero). Per evitare dati duplicati, è **obbligatorio** imporre un vincolo di univocità (UNIQUE) sull'insieme degli attributi che formavano l'originaria chiave naturale composita.

2.5.2 Analisi degli Attributi Derivati e delle Ridondanze

Durante la progettazione concettuale, è comune inserire attributi il cui valore non è indipendente, ma può essere ricavato matematicamente o logicamente da altri dati presenti nel sistema. Nel Class Diagram UML, questa caratteristica viene esplicitata anteponendo una barra (slash) al nome dell'attributo (es. /età).

Nel passaggio al Class Diagram revisionato (e successivamente allo schema logico), il progettista deve compiere una scelta architeturale precisa: l'attributo derivato verrà **effettivamente memorizzato** (storicizzato) nel database o verrà eliminato dallo schema e **calcolato a runtime** ogni volta che viene richiesto? Se si decide di non memorizzarlo, l'attributo scompare del tutto dallo schema revisionato.

Esempio 2.12 (Età vs Data di Nascita). Si consideri la classe *Persona* dotata degli attributi *dataDiNascita* ed /età. L'età è banalmente derivabile per differenza tra la data odierna e la data di nascita.

La scelta tra memorizzazione e calcolo on-the-fly comporta un compromesso tra prestazioni di lettura, spazio occupato e coerenza dei dati.

Scenario A: Memorizzazione (Storicizzazione del dato)

- **Vantaggi:** Tempi di risposta immediati durante le interrogazioni (query). Il DBMS si limita a leggere il dato senza sprecare cicli di CPU per eseguire calcoli, aspetto cruciale se il dato è richiesto frequentissimamente.
- **Svantaggi:**
 - Spreco di spazio su disco (memoria).
 - Rischio di *inconsistenza*: se la *dataDiNascita* viene corretta ma ci si dimentica di aggiornare l'età, il database conterrà informazioni contraddittorie.
 - Oneri di aggiornamento: dati altamente volatili (come l'età, che cambia ogni anno) richiedono procedure o *trigger* che aggiornino continuamente il database, generando un carico di scrittura pesante.

Scenario B: Calcolo a Runtime (Nessuna memorizzazione)

- **Vantaggi:** Nessun rischio di inconsistenza (il calcolo parte sempre dal "dato sorgente" aggiornato), risparmio di memoria e zero manutenzione per l'aggiornamento.
- **Svantaggi:** Costo computazionale pagato al momento dell'interrogazione. Se il calcolo è complesso e viene richiesto da milioni di utenti, il server potrebbe rallentare drasticamente.



Principi Generali per gli Attributi Derivati

Nell'ingegneria dei dati, la scelta si basa sull'analisi del carico di lavoro previsto e sulla volatilità del dato:

1. **Dati volatili e calcoli leggeri (Preferire il Calcolo):** Se il dato derivato cambia spesso nel tempo e l'operazione matematica è banale, l'attributo si elimina e si calcola al volo.
2. **Dati stabili e calcoli pesanti (Preferire la Memorizzazione):** Se il



dato originario, una volta inserito, non cambia più (es. il totale di una **Fattura** emessa e consolidata, ricavato dalla somma delle sue **Righe**), conviene storicizzare il totale. Il costo di aggiornamento è zero (la fattura non cambierà) e si evitano pesanti JOIN e somme ad ogni lettura.

Analisi delle Associazioni Ridondanti

Il concetto di ridondanza si estende oltre i singoli attributi e coinvolge l'intera topologia del Class Diagram. In uno schema complesso, potremmo accorgerci dell'esistenza di **associazioni derivabili** per transitività.

Ad esempio, se un **Impiegato** è associato a un **Dipartimento**, e un **Dipartimento** è associato a una **Città**, un'eventuale associazione diretta tra **Impiegato** e **Città** è da considerarsi un ciclo ridondante, poiché la città dell'impiegato è già ricavabile navigando il percorso **Impiegato** → **Dipartimento** → **Città**.

Come per gli attributi, il progettista deve decidere se mantenere l'associazione diretta (tramite l'aggiunta di una chiave esterna ridondante) o se farla collassare e affidarsi alla navigazione del grafo.



Principi Generali per le Associazioni Ridondanti

La gestione dei cicli si basa sul compromesso tra prestazioni di *routing* e integrità strutturale:

1. **Evitare anomalie strutturali (Preferire l'eliminazione):** La regola generale impone di rimuovere le associazioni ridondanti. Mantenere un legame diretto crea il rischio di inconsistenze: potremmo, per errore, assegnare l'Impiegato al Dipartimento di Milano, ma associarlo direttamente alla Città di Roma.
2. **Ottimizzazione estrema (Mantenere il legame diretto):** Si deroga alla regola precedente solo in sistemi ad altissimo traffico (*read-heavy*) in cui il percorso di navigazione è molto lungo (es. richiede di attraversare 4 o 5 tabelle con relative operazioni di JOIN). In quel caso, si introduce l'associazione ridondante come scorciatoia prestazionale, assumendosi l'onere (spesso tramite *stored procedures*) di mantenerla sincronizzata con il percorso principale.

2.5.3 Analisi delle Associazioni Uno a Uno: Fusione vs Separazione

Durante la progettazione concettuale, due entità legate da un'associazione rigorosamente biunivoca ($1..1 \leftrightarrow 1..1$) mantengono identità separate per ragioni semantiche. Poiché i loro cicli di vita coincidono (nascono e muoiono insieme), nella fase di ristrutturazione logica si presenta l'opportunità di **fondere le due classi in un unico schema relazionale**, consolidando i loro attributi ed eliminando la necessità di una chiave esterna.

Questa decisione architetturale comporta un delicato trade-off tra la velocità di lettura (assenza di JOIN) e l'occupazione di memoria (pesantezza della tabella), oltre a sollevare questioni di sicurezza dei dati.

Esempio 2.13 (Fusione vs Separazione: Paziente e Cartella Clinica). Si consideri l'associazione obbligatoria ($1..1 \leftrightarrow 1..1$) tra un **Paziente** e la sua **CartellaClinica**. Ogni paziente ha una sola cartella clinica, e ogni cartella appartiene a un solo paziente.

Scenario A: Fusione (Unico Schema) Se fondessimo le due entità, otterremmo un'unica tabella massiccia:

PAZIENTE(codiceFiscale, nome, telefono, gruppoSanguigno, anamnesiCompleta)

- *Vantaggio*: Nessun JOIN necessario per leggere i dati completi.
- *Svantaggio*: L'attributo **anamnesiCompleta** potrebbe essere un testo lunghissimo (BLOB o TEXT). Ogni volta che un impiegato dell'accettazione interroga il database solo per cercare il numero di **telefono** del paziente, il motore del database è costretto a caricare in memoria dal disco fisso anche i megabyte di testo dell'anamnesi, sprecando enormi risorse di I/O. Inoltre, si espongono dati sensibili a personale non medico.

Scenario B: Separazione (Due Schemi) Mantenendo gli schemi separati, applichiamo un *partizionamento verticale* logico:

- PAZIENTE(codiceFiscale, nome, telefono)
- CARTELLA_CLINICA(codiceCartella, gruppoSanguigno, anamnesi, cfPaziente)

Vantaggi:

1. **Performance**: La tabella PAZIENTE è "leggera" e stretta. Le query anagrafiche quotidiane saranno fulminee e il database potrà tenere l'intera tabella nella cache RAM.
2. **Sicurezza**: Si possono impostare permessi di accesso (GRANT) a livello di tabella, permettendo alla segreteria di leggere PAZIENTE e impedendo l'accesso a CARTELLA_CLINICA, riservato ai soli medici.

Svantaggio: Quando un medico avrà bisogno del nome del paziente e della sua anamnesi contemporaneamente, il sistema dovrà pagare il costo computazionale di un JOIN.



La Regola Pratica

La fusione in un unico schema è raccomandata quando gli attributi della seconda classe sono pochi, di piccole dimensioni (es. interi, date, stringhe brevi) e vengono interrogati quasi sempre insieme a quelli della classe principale (alta coesione). La separazione è invece mandatoria in presenza di attributi "pesanti" (es. file, immagini, testi lunghi) o sottoposti a stringenti policy di privacy.

2.5.4 Analisi degli Attributi Strutturati

Nel diagramma delle classi originario, è frequente l'utilizzo di **attributi strutturati** (o *composti*), ovvero attributi che raggruppano al loro interno campi di natura eterogenea. L'esempio classico è l'attributo **residenza** assegnato a una classe **Persona**, il quale è concettualmente composto da **via**, **civico**, **cap** e **città**.

Il Modello Relazionale, tuttavia, impone il rispetto della **Prima Forma Normale (1NF)**, la quale stabilisce che il dominio di ogni attributo debba essere **atomico** (indivisibile). I database relazionali puri non supportano l'inserimento di "sotto-strutture" o "oggetti" all'interno di una singola cella.

Per risolvere questa incompatibilità durante la fase di ristrutturazione, il progettista deve eliminare l'attributo strutturato scegliendo una delle seguenti tre strategie:

1. Appiattimento (Flattening) degli attributi

L'attributo strutturato viene rimosso e sostituito dall'esplicitazione di tutti i suoi sotto-campi direttamente come colonne della classe principale.

- **Schema:** PERSONA(cf, nome, via, civico, cap, città)
- **Vantaggi:** Non richiede JOIN. Permette di effettuare ricerche efficienti sui singoli sotto-campi (es. "trova tutte le persone che abitano a Roma").
- **Svantaggi:** "Allarga" la tabella principale, rischiando di riempirla di valori NULL qualora l'attributo strutturato originario fosse opzionale (0..1).

2. Accorpamento in formato Stringa (Concatenazione)

I sotto-campi vengono fusi in un'unica stringa testuale, inserita in un solo attributo atomico.

- **Schema:** PERSONA(cf, nome, indirizzoCompleto)
- **Vantaggi:** Estrema semplicità e compattezza dello schema.
- **Svantaggi:** Rende difficilissime o ineseguibili le query analitiche (es. contare le persone per CAP richiederebbe complesse e lente operazioni di *parsing* testuale tramite LIKE o *Regex*).

3. Creazione di un'Entità Autonoma

Si estrapola l'attributo strutturato promuovendolo a nuova **Classe autonoma**, collegata a quella originale tramite un'associazione.

- **Schema:** PERSONA(..., idResidenza) → RESIDENZA(id, via, civico, città)
- **Vantaggi:** Massima pulizia formale. Permette di riutilizzare l'entità (es. se cinque membri di una famiglia condividono la stessa residenza, il dato non viene duplicato 5 volte, ma inserito una volta sola e linkato).
- **Svantaggi:** Introduce un costo computazionale (JOIN) per ogni lettura completa e rende più complessa la fase di inserimento dei dati.

Principi Generali di Scelta

La decisione tra queste tre soluzioni è dettata dall'utilizzo pratico che l'applicazione farà del dato:

1. **Se il dato è un blocco opaco (Preferire l'Accorpamento testuale):** Se l'indirizzo serve esclusivamente per essere stampato su un'etichetta di spedizione e al sistema non interesserà mai fare statistiche per città o per CAP, una stringa formattata è la scelta più leggera e sensata.
2. **Se i campi hanno dignità di interrogazione (Preferire l'Appiattimento):** Se il software prevede filtri di ricerca (es. un menù a tendina per filtrare gli utenti per Provincia), i campi devono essere separati per poter essere indicizzati singolarmente.
3. **Se l'attributo ha vita propria o cardinalità multipla (Preferire l'Entità Autonoma):** Se l'indirizzo deve essere storicizzato (tenere traccia dei vecchi indirizzi), se una persona ha molteplici indirizzi (casa, lavoro, fatturazione), o se lo stesso indirizzo raggruppa logicamente più utenti (come in un'azienda), l'attributo strutturato **deve** diventare una tabella separata.

2.5.5 Analisi degli Attributi a Valore Multiplo

Nel Class Diagram UML è perfettamente lecito assegnare a un'entità un attributo con cardinalità multipla, ovvero una lista o un array di valori (es. `telefoni [1..*]` per la classe `Persona`). Tuttavia, il modello relazionale ammette unicamente **valori singoli e scalari** per ogni cella. L'inserimento di vettori o array non è nativamente supportato dalle regole dell'algebra relazionale.

Il tentativo di risolvere il problema "appiattendo" l'attributo in una serie di colonne fisse (`telefono1`, `telefono2`, ...) rappresenta una pessima pratica ingegneristica: spreca enormi quantità di memoria (generando celle NULL per gli utenti con un solo recapito), limita la scalabilità (non permette l'inserimento di un numero di recapiti superiore alle colonne predisposte) e rende le query di ricerca estremamente inefficienti.

Per normalizzare lo schema, il progettista deve eliminare l'attributo multivalore ricorrendo a una delle seguenti soluzioni:

1. Accorpamento testuale (Formattazione Custom)

Tutti i valori dell'array vengono concatenati e salvati in un'unica stringa di testo, separati da un carattere speciale (es. virgola o punto e virgola).

- **Schema:** `PERSONA(cf, nome, listaTelefoni)`
- *Valore di esempio:* "3331234567; 069876543"
- **Vantaggi:** Non modifica la struttura del database e non richiede tabelle aggiuntive.
- **Svantaggi:** Impedisce al database di effettuare controlli sui singoli valori (es. non si può usare un vincolo `UNIQUE` per impedire che due utenti inseriscano lo stesso numero). Rendere aggiornabile o ricercabile un singolo numero richiede complesse operazioni di manipolazione delle stringhe.

2. Creazione di una Classe Esterna (Associazione 1:N)

Si estrapola l'attributo trasformandolo in un'entità autonoma debole, legata all'entità principale da un'associazione uno a molti. È la soluzione relazionale per eccellenza.

- **Schema:**
 - `PERSONA(cf, nome)`
 - `TELEFONO(numero, cfPersona)`
- **Vantaggi:** Scalabilità infinita (una persona può avere da zero a mille telefoni senza sprecare byte). Le query sono pulite e veloci, e si possono imporre vincoli di integrità sui singoli numeri.
- **Svantaggi:** Richiede un'operazione di `JOIN` per estrarre l'elenco completo dei telefoni associati a una persona.

Principi Generali di Scelta

La scelta della tecnica di risoluzione è quasi sempre a favore dell'entità esterna, tranne in casi specifici di puro logging:

1. **Dati operativi e ricercabili (Regola Aurea → Entità Esterna):** Se il dato ha valore di business, se il sistema dovrà mai cercare "a chi appartiene questo numero", se l'utente deve poter modificare o cancellare un singolo valore dalla sua lista, la creazione di una tabella esterna 1:N è l'unica via ingegneristicamente solida.



2. **Dati descrittivi morti (Eccezione → Accorpamento Testuale):** Si opta per la stringa formattata solo per dati passivi, che vengono scritti una volta e letti in blocco unicamente per stamparli a schermo. Ad esempio: un attributo multivalore `note_aggiuntive` o `tag_ricerca` in sistemi molto semplici.

2.6 Codifica delle Gerarchie (Ereditarietà)

Il modello relazionale è intrinsecamente "piatto" e non supporta nativamente il concetto orientato agli oggetti di ereditarietà (superclasse e sottoclassi). Pertanto, quando nel Class Diagram si incontra una gerarchia di generalizzazione/specializzazione, è necessario "smontarla" trasformandola in tabelle ordinarie.

L'ingegneria del software offre tre strategie principali per affrontare questa traduzione, ciascuna con precisi impatti su prestazioni, memoria e integrità dei dati.

1. Partizionamento Orizzontale (Accorpamento verso il basso) Questa soluzione prevede l'eliminazione della superclasse e la creazione di una tabella per ciascuna sottoclasse. Ogni tabella figlia "eredita" fisicamente copiando al proprio interno tutti gli attributi della generalizzazione.

- **Schema logico:** `PERSONA(nome, cognome)` scompare. Si creano `STUDENTE(id, nome, cognome, matricola)` e `DOCENTE(id, nome, cognome, stipendio)`.
- **Vantaggi:** Interrogare una specifica sottoclasse è estremamente veloce (nessun JOIN).
- **Svantaggi e Limiti:** Questa strada è matematicamente percorribile **solo se la generalizzazione è totale** (ovvero se non esistono istanze della superclasse che non appartengano a nessuna sottoclasse). Inoltre, se si vuole fare una query su tutte le persone (indipendentemente dal tipo), il database deve eseguire una pesante operazione di UNION tra le tabelle.

2. Partizionamento Singolo (Accorpamento verso l'alto) Questa soluzione, all'opposto, prevede l'eliminazione delle sottoclassi. Si crea un'unica, grande tabella (la superclasse) che ingloba tutti gli attributi di tutte le specializzazioni, introducendo un attributo **discriminante** (un enumeratore) per capire di che "tipo" sia la riga.

- **Schema logico:** `PERSONA(id, tipoPersona, nome, cognome, matricola, stipendio)`
- **Vantaggi:** È sempre applicabile ed è la soluzione più performante in assoluto (niente JOIN, niente UNION). Molto amata nello sviluppo web moderno per la sua velocità in lettura.
- **Svantaggi (Problema dei vincoli persi):** Genera tabelle "sparse", piene di valori NULL (uno studente avrà lo stipendio NULL). Soprattutto, fa perdere il rigore delle associazioni originarie.

Esempio 2.14 (Recupero dei vincoli di consistenza). Nel Class Diagram, supponiamo che l'associazione *"Insegna"* leghi la classe `Corso` ESCLUSIVAMENTE alla sottoclasse `Docente`. Utilizzando l'accorpamento verso l'alto, la tabella `CORSO` dovrà avere una chiave esterna che punta all'unica tabella rimasta: `PERSONA`. Questo permette a livello strutturale l'assurdità di far insegnare un corso a uno `Studente`. Per recuperare l'informazione persa, si **deve** imporre un vincolo applicativo o un *CHECK constraint* SQL che assicuri che la chiave esterna punti solo a righe in cui l'attributo discriminante `tipoPersona = 'Docente'`.

3. Partizionamento Verticale (Traduzione in Associazioni 1:1) Questa strategia preserva la struttura del diagramma originale. Si crea una tabella per la superclasse (con i soli attributi comuni) e una tabella per ogni sottoclasse (con i soli attributi specifici). Le tabelle figlie vengono legate alla tabella padre tramite un'associazione 1:1 rigorosa, utilizzando la tecnica della **Chiave Primaria Condivisa**.

- **Schema logico:**
 - PERSONA(idPersona, nome, cognome)
 - STUDENTE(idPersona, matricola)
 - DOCENTE(idPersona, stipendio)
- **Vantaggi:** Rispetta la terza forma normale, non spreca spazio con i NULL e mantiene perfettamente validi i vincoli delle associazioni. Nel caso di **ereditarietà disgiunta**, è necessario aggiungere un *trigger* che impedisca allo stesso *idPersona* di comparire sia in STUDENTE che in DOCENTE.
- **Svantaggi:** È la soluzione più costosa a livello computazionale. L'estrazione dei dati completi di un utente richiederà sempre un JOIN tra la tabella specifica e la superclasse.



Principi Generali di Scelta dell'Ereditarietà

Non esiste una soluzione perfetta a priori, ma la pratica ingegneristica suggerisce queste euristiche:

1. **Usa l'accorpamento in alto (Singola Tabella):** Quando le sottoclassi differiscono per pochissimi attributi e non hanno relazioni specifiche pesanti. Il piccolo spreco di spazio dovuto ai NULL è ampiamente ripagato dalla velocità estrema delle query.
2. **Usa l'accorpamento in basso (Tabelle Separate):** Quando la generalizzazione è totale, le query non cercano quasi mai la "superclasse" generica, e le sottoclassi sono entità molto ricche e complesse di per sé.
3. **Usa le associazioni 1:1 (Tabella per Classe):** Quando l'integrità dei dati è la priorità assoluta, le sottoclassi hanno tantissime associazioni specifiche con altre tabelle del database e lo spazio su disco deve essere ottimizzato al massimo.

2.7 Riepilogo: Il Ciclo di Vita della Progettazione dei Dati

Prima di introdurre il linguaggio di interrogazione e definizione dei dati, è utile schematizzare l'intera metodologia di progettazione di basi di dati affrontata finora. Il processo ingegneristico segue un flusso di raffinamento progressivo (dal livello astratto al livello implementativo), suddiviso in fasi distinte e sequenziali. Il quarto punto rappresenta un'anticipazione di quanto vedremo nel prossimo capitolo.

1. Progettazione Concettuale (Modellazione del Dominio)

- *Obiettivo:* Rappresentare la semantica della realtà di interesse in modo puro e indipendente dalla tecnologia.
- *Artefatti prodotti:* **Class Diagram UML** iniziale.
- *Documentazione a corredo:* **Dizionari dei Dati** (dizionario delle classi, dizionario delle associazioni e dizionario dei vincoli di business non esprimibili graficamente).

2. Ristrutturazione dello Schema (Revisione Concettuale)

- *Obiettivo:* Adattare il modello concettuale puro ai limiti e ai requisiti del

modello logico di destinazione (es. eliminazione attributi strutturati/multivalore, gestione delle gerarchie, ottimizzazione delle chiavi).

- *Artefatti prodotti*: **Class Diagram Revisionato**.
- *Documentazione a corredo*: Aggiornamento dei dizionari. In particolare, il **Dizionario dei Vincoli** subisce un forte arricchimento per recuperare semanticamente tutte le informazioni perse durante gli appiattimenti strutturali (es. vincoli per le ereditarietà accorpate).

3. Progettazione Logica (Traduzione Relazionale)

- *Obiettivo*: Tradurre gli elementi del diagramma revisionato in strutture dati compatibili con l'algebra relazionale.
- *Artefatti prodotti*: **Schemi Relazionali** definitivi, con esplicita indicazione degli identificatori (*Chiavi Primarie* - *PK*) e dei legami di navigabilità (*Chiavi Esterne* - *FK*).

4. Implementazione e Definizione dei Metadati (Verso il livello Fisico)

- *Obiettivo*: Istruire materialmente il motore del database (DBMS) a creare le strutture logiche appena progettate.
- *Strumento*: Utilizzo del linguaggio **SQL** (in particolare il sotto-insieme DDL, *Data Definition Language*), attraverso il quale gli schemi relazionali e i vincoli vengono codificati nei metadati del sistema.

Normalizzazione Relazionale

Nel capitolo precedente abbiamo affrontato la transizione dal modello concettuale (Class Diagram UML) agli schemi relazionali. Durante la fase di ristrutturazione (si vedano, ad esempio, i paragrafi sulla risoluzione degli attributi strutturati e multivalore), abbiamo già sfiorato e applicato intuitivamente alcuni principi volti a "pulire" le tabelle e separare i concetti.

Tuttavia, l'ingegneria del software non può basarsi unicamente sulle euristiche o sull'intuito del progettista. In questo capitolo tratteremo la normalizzazione (e le relative Forme Normali) in modo rigoroso, dettagliato e formale. Questa teoria rappresenta il "collaudo" definitivo degli schemi relazionali appena prodotti: fornisce gli strumenti esatti per dimostrare che la struttura del database è solida e immune da difetti. Solo superando questo test formale, il discorso sulla progettazione logica può considerarsi completato e pronto per la traduzione in codice SQL.

3.1 Ridondanze e Anomalie

Un database relazionale è progettato male quando un singolo schema (una tabella) raggruppa forzatamente concetti eterogenei che non dovrebbero stare insieme. Questo errore di modellazione genera **ridondanza** (la duplicazione inutile della stessa informazione su più righe) che, a sua volta, innesca tre tipologie di anomalie critiche durante il normale ciclo di vita del software.

Esempio 3.1 (Il disastro della tabella non normalizzata). Supponiamo di aver modellato un'unica tabella mista per gestire i dipendenti e i dipartimenti in cui lavorano, eleggendo la matricola a chiave primaria:

DIPENDENTE(matricola, nome, dipartimento, sede_dipartimento)

Se il dipartimento "IT" si trova a "Milano" e ha 1000 dipendenti, la stringa "Milano" verrà memorizzata fisicamente 1000 volte. Questa ridondanza apre le porte alle tre anomalie relazionali:

1. **Anomalia di Aggiornamento (Update Anomaly):** Se il dipartimento "IT" viene trasferito a "Roma", l'applicativo dovrà eseguire una UPDATE su 1000 righe distinte. Se per un calo di rete o un crash del server l'aggiornamento si interrompe a 500 righe, il database entra in uno stato *inconsistente*: metà dei dipendenti IT risulterà a Roma, l'altra metà a Milano. Il dato perde affidabilità.
2. **Anomalia di Inserimento (Insert Anomaly):** L'azienda apre un nuovo dipartimento "Marketing" a "Napoli", ma non ha ancora assunto nessun dipendente. Poiché la matricola è chiave primaria (quindi NOT NULL per definizione), il database rifiuterà l'inserimento. È strutturalmente impossibile registrare l'esistenza di un dipartimento vuoto.
3. **Anomalia di Cancellazione (Delete Anomaly):** Il dipartimento "HR" ha un solo dipendente. Se questo dipendente si dimette, l'applicativo eseguirà una DELETE sulla sua riga. Oltre a cancellare il dipendente, **distruggeremo involontariamente anche l'informazione che il dipartimento HR esiste** e che si trovava in una determinata sede. Abbiamo perso un dato vitale per l'azienda a causa dell'eliminazione di un dato non correlato.

Per risolvere alla radice queste anomalie, il progettista non deve scrivere codice applicativo più complesso, ma deve modificare la struttura del database. La soluzione consiste nello **spaccare** (decomporre) la tabella difettosa in due tabelle distinte, legate da una chiave esterna:

- DIPENDENTE(matricola, nome, dipartimento*)
- DIPARTIMENTO(nome_dip, sede_dipartimento)

La teoria della normalizzazione (le Forme Normali) fornisce le regole esatte per capire *quando* e *come* effettuare questa scomposizione in modo sicuro.

3.2 Dipendenze Funzionali

Nel paragrafo precedente abbiamo visto come le ridondanze generino bug strutturali (le anomalie). Per evitare di dover scovare queste ridondanze "a occhio" o basandosi solo sull'intuito, la teoria relazionale fornisce uno strumento analitico per identificare le anomalie: la **Dipendenza Funzionale** (Spesso abbreviata in *FD* - *Functional Dependency*).

Una dipendenza funzionale stabilisce che il valore di un determinato campo "decide" in modo univoco il valore di un altro campo. Se conosco il primo, non ho dubbi sul secondo.

Formalmente, data una relazione (tabella) r definita su uno schema $R(X)$, e dati due sottoinsiemi di attributi non vuoti Y e Z (appartenenti a X), diremo che esiste su r una dipendenza funzionale tra Y e Z se:

Per ogni coppia di tuple (righe) t_1 e t_2 presenti in r che hanno **gli stessi valori sugli attributi Y** , risulta obbligatoriamente che t_1 e t_2 hanno **gli stessi valori anche sugli attributi Z** .

In notazione algebrica, questo vincolo di integrità si scrive:

$$Y \rightarrow Z$$

E si legge: " Y determina funzionalmente Z ", oppure " Z dipende funzionalmente da Y ". L'insieme Y prende il nome di **Determinante**.

Esempio 3.2. Immaginiamo una tabella temporanea che traccia i veicoli assicurati e le relative polizze:

COPERTURA(numero_polizza, targa, marca, modello)

Osservando la natura dei dati (ovvero la semantica del mondo reale), possiamo affermare con certezza matematica che esiste la dipendenza funzionale:

$$\text{targa} \rightarrow \text{marca, modello}$$

Il motivo risponde esattamente alla definizione formale: se prendiamo due righe qualsiasi della tabella (t_1 e t_2) che presentano lo stesso valore nel campo **targa** (es. "AB123CD"), quelle due righe dovranno **obbligatoriamente** avere lo stesso valore nei campi **marca** (es. "Fiat") e **modello** (es. "Panda"). Un veicolo non può cambiare marca da una riga all'altra.

L'analisi delle dipendenze funzionali si basa su tre postulati pratici che semplificano il lavoro di progettazione:

1. **Scomposizione (Decomposizione del conseguente):** Se un determinante Y definisce un insieme di attributi Z composto da $A_1, A_2, \dots, A_k$, allora possiamo affermare che la relazione soddisfa $Y \rightarrow Z$ se e solo se soddisfa singolarmente tutte le dipendenze:

$$Y \rightarrow A_1 \quad \text{e} \quad Y \rightarrow A_2 \quad \text{e} \dots \quad Y \rightarrow A_k$$

Esempio: Dire che $\text{targa} \rightarrow \text{marca}$, modello equivale a dire che $\text{targa} \rightarrow \text{marca}$ e $\text{targa} \rightarrow \text{modello}$. Per semplicità, d'ora in avanti considereremo sempre dipendenze del tipo $Y \rightarrow A$ (dove la parte destra è un singolo attributo).

2. **Dipendenze Banali (da ignorare):** Se l'attributo A fa già parte dell'insieme Y , la dipendenza viene definita **banale**. Ad esempio, la dipendenza $\text{targa}, \text{modello} \rightarrow \text{targa}$ è ovvia e non aggiunge alcuna informazione utile al design del database. Nello studio della normalizzazione, si prendono in esame esclusivamente le dipendenze funzionali **non banali**.
3. **La generalizzazione del concetto di Chiave:** Cos'è una Chiave Primaria in ottica matematica? Non è altro che un determinante supremo. Dire che numero_polizza è la chiave della tabella, equivale a dichiarare la dipendenza funzionale:

$$\text{numero_polizza} \rightarrow \text{Tutti gli altri attributi dello schema}$$

La Dipendenza Funzionale, quindi, **generalizza il concetto di chiave**.



L'intuizione dietro le Forme Normali

Questa generalizzazione ci fornisce la chiave di lettura per l'intera teoria della normalizzazione. Se in una tabella troviamo una dipendenza funzionale forte (come $\text{targa} \rightarrow \text{marca}$), ma la **targa non** è la chiave primaria di quella tabella, significa che c'è un difetto strutturale: stiamo costringendo un sotto-tema (il veicolo) dentro un tema più grande (la copertura). È esattamente questa discrepanza a generare le ridondanze e le anomalie che impongono la scomposizione in più tabelle.

3.3 La Forma Normale di Boyce-Codd (BCNF)

Il traguardo ultimo della progettazione logica relazionale è garantire che il database sia totalmente privo di anomalie e ridondanze. Lo strumento che certifica questa garanzia strutturale è la Forma Normale di Boyce-Codd (BCNF).

Definizione della BCNF Uno schema relazionale R è in Forma Normale di Boyce-Codd se, per ogni sua dipendenza funzionale non banale $X \rightarrow Y$, risulta che X è una **superchiave** per lo schema R .

In termini pratici e ingegneristici, questa regola si traduce in un diktat assoluto: *"L'unica cosa che ha il diritto di determinare il valore di un attributo è la chiave primaria (o una chiave candidata) della tabella stessa."* Se un attributo non-chiave si arroga il diritto di determinare altri attributi, il database nasconde un difetto strutturale.

Esempio 3.3 (Violazione della BCNF e Ritorno alle Anomalie). Riprendiamo lo schema difettoso analizzato nel primo paragrafo e applichiamo la lente delle dipendenze funzionali per dimostrarne l'invalidità:

DIPENDENTE(matricola, nome, dipartimento, sede_dipartimento)

In questo schema, l'unica chiave candidata (eletta a chiave primaria) è matricola. Di conseguenza, la matricola è una superchiave. La dipendenza funzionale $\text{matricola} \rightarrow \text{nome}, \text{dipartimento}, \text{sede_dipartimento}$ è quindi del tutto lecita e rispetta la regola della BCNF.

Tuttavia, analizzando la semantica del dominio aziendale, sappiamo che un dipartimento si trova in una specifica sede. Esiste quindi una seconda dipendenza funzionale non banale "nascosta" nello schema:

$\text{dipartimento} \rightarrow \text{sede_dipartimento}$

Sottoponiamo questa dipendenza al test della BCNF: il determinante dipartimento è una superchiave dello schema DIPENDENTE? **Assolutamente no**. Il nome di un dipartimento non identifica in modo univoco una riga, poiché possono coesistere centinaia di impiegati assegnati allo stesso dipartimento.

Poiché esiste almeno una dipendenza funzionale il cui determinante non è una superchiave, l'intero schema **viola la BCNF**. È esattamente questa specifica violazione a generare le ridondanze e le letali anomalie di inserimento, aggiornamento e cancellazione.

3.3.1 Decomposizioni

Quando uno schema fallisce il collaudo della BCNF, la teoria relazionale impone di decomporlo. Il processo consiste nell'isolare la dipendenza funzionale "illegale" e promuoverla a tabella autonoma. In questo nuovo schema, il vecchio determinante difettoso diventa finalmente una chiave primaria legittima.

Applicando la scomposizione al nostro esempio, otteniamo due nuovi schemi:

- DIPARTIMENTO(nome_dip, sede_dipartimento)
 - Dipendenze presenti: $\text{nome_dip} \rightarrow \text{sede_dipartimento}$.
 - Test BCNF: nome_dip è la chiave primaria. L'unica dipendenza rispetta la BCNF.
- DIPENDENTE(matricola, nome, dipartimento*)
 - Dipendenze presenti: $\text{matricola} \rightarrow \text{nome}, \text{dipartimento}$.
 - Test BCNF: matricola è la chiave primaria. L'unica dipendenza rispetta la BCNF.

Entrambi gli schemi generati si trovano ora rigorosamente in Forma Normale di Boyce-Codd. Il database è strutturalmente inattaccabile.

Nell'esempio precedente, la scomposizione delle dipendenze (e quindi dei concetti da esse rappresentati) è stata facilitata dalla struttura delle dipendenze stesse, "naturalmente" separate e indipendenti l'una dall'altra. In molti casi pratici, infatti, la decomposizione in BCNF si ottiene linearmente isolando ogni dipendenza funzionale con un diverso determinante.

Tuttavia, in scenari di dominio più complessi, le dipendenze possono intrecciarsi: può accadere che il tentativo di forzare la scomposizione per raggiungere la BCNF finisca per distruggere (perdere) alcune dipendenze funzionali originarie, rendendo impossibile per il DBMS validarle in modo efficiente.

Per questa ragione, nella pratica ingegneristica, la BCNF rappresenta l'ideale strutturale a cui tendere, ma non un dogma assoluto. Qualora il raggiungimento della BCNF

comportasse la perdita di vincoli semantici o un decadimento inaccettabile delle prestazioni (a causa di un partizionamento verticale eccessivo), il progettista adotta come compromesso architetturale la **Terza Forma Normale (3NF)** (che vedremo a seguire), che tollera lievi ridondanze pur garantendo la preservazione totale delle dipendenze.

3.3.2 Qualità delle Decomposizioni

L'attività di scomposizione di uno schema non deve limitarsi a eliminare le anomalie, ma deve produrre relazioni di "qualità sufficiente". Considereremo una scomposizione accettabile solo se soddisfa due proprietà fondamentali: la **decomposizione senza perdita** e la **conservazione delle dipendenze**.

1. Decomposizione Senza Perdita (Lossless Join) La decomposizione senza perdita garantisce che le informazioni presenti nella relazione originaria siano ricostruibili con precisione, senza la generazione di informazioni spurie, a partire dalle relazioni scomposte. In termini pratici, interrogando le nuove tabelle tramite un'operazione di JOIN, si devono ottenere gli stessi identici risultati che si otterrebbero interrogando la tabella originale.

Per garantire questo risultato, basta applicare una regola: *gli attributi (eventualmente uno solo) in comune utilizzati per dividere le due tabelle devono obbligatoriamente essere una Chiave Primaria (o Superchiave) in almeno una delle due.*

Esempio 3.4 (Decomposizione Senza Perdita vs Con Perdita). Consideriamo uno schema che mappa i sinistri e i periti assegnati:

SINISTRO(id_sinistro, nome_perito, telefono_perito)

Scomposizione Corretta (Senza Perdita): Separiamo l'entità usando un identificatore univoco (creando ad esempio p_iva_perito):

- SINISTRO(id_sinistro, p_iva_perito)
- PERITO(p_iva_perito, nome_perito, telefono_perito)

Ricongiungendo le tabelle tramite p_iva_perito (che è chiave primaria in PERITO), il DBMS ricostruisce esattamente l'abbinamento corretto.

Scomposizione Errata (Con Perdita): Se scomponessimo usando il nome come attributo di legame:

- SINISTRO(id_sinistro, nome_perito)
- PERITO(nome_perito, telefono_perito)

Se esistono due periti diversi che si chiamano "Mario Rossi", un'operazione di JOIN sul nome combinerebbe erroneamente il sinistro con i telefoni di entrambi i periti, generando tuple "spurie" (false) che non esistevano nella relazione originale.

2. Conservazione delle Dipendenze La conservazione delle dipendenze garantisce che le relazioni decomposte mantengano la stessa capacità della relazione originaria di rappresentare i vincoli di integrità del dominio. Questo permette al database di rilevare agevolmente eventuali aggiornamenti illeciti, poiché ad ogni operazione non lecita sullo schema originale corrisponderà una violazione dei vincoli anche sulle tabelle scomposte.

Se una scomposizione "rompe" una dipendenza funzionale separando i suoi attributi in tabelle diverse, il DBMS non sarà più in grado di validare quel vincolo in fase di INSERT o UPDATE senza ricorrere a complessi e lenti controlli incrociati (es. tramite Trigger).

Esempio 3.5 (Conservazione delle Dipendenze). Sia dato lo schema $R(\underline{A}, B, C)$ con le dipendenze: $A \rightarrow B, B \rightarrow C$.

- **Decomposizione che conserva le dipendenze:** Scomporre in $R_1(A, B)$ e $R_2(\underline{B}, C)$ è eccellente. Il DBMS può validare $A \rightarrow B$ guardando solo R_1 , e può validare $B \rightarrow C$ guardando solo R_2 (assicurandosi che per un dato B non vengano inseriti due C diversi).
- **Decomposizione che NON conserva le dipendenze:** Scomporre in $R_1(A, B)$ e $R_3(\underline{A}, C)$. Sebbene questa scomposizione sia "senza perdita" (A è chiave in entrambe), abbiamo distrutto la dipendenza $B \rightarrow C$. Il DBMS permetterà l'inserimento di due tuple in R_1 e R_3 che, combinate, associano allo stesso B due valori di C differenti, violando silenziosamente la regola di business originale.

3.4 La Terza Forma Normale (3NF)

Nella stragrande maggioranza dei casi, l'applicazione della BCNF produce un design perfetto, garantendo sia l'assenza di anomalie (decomposizione senza perdita) sia la conservazione delle dipendenze. Tuttavia, esistono rari e specifici scenari di dominio in cui la BCNF non è applicabile in modo efficace, costringendo il progettista a un compromesso.

Il Limite della BCNF Il limite della BCNF emerge quando uno schema presenta chiavi candidate sovrapposte (che condividono attributi) e dipendenze funzionali incrociate. In questi casi, il tentativo di forzare la scomposizione in BCNF causa la perdita irreparabile di una dipendenza originaria.

Esempio 3.6 (La BCNF che distrugge le dipendenze). Si consideri una tabella che traccia l'assegnazione degli sviluppatori ai vari moduli di un software, e i relativi Tech Lead:

`SVILUPPO(id_sviluppatore, modulo, tech_lead)`

Le regole di business impongono due dipendenze funzionali:

1. Uno sviluppatore assegnato a un modulo risponde a un solo Tech Lead.
(`id_sviluppatore, modulo` $\rightarrow$ `tech_lead`)
2. Ogni Tech Lead è responsabile di un unico modulo software, sebbene gestisca molti sviluppatori.
(`tech_lead` $\rightarrow$ `modulo`)

Poiché un Tech Lead determina il modulo, la coppia $\{\text{id_sviluppatore}, \text{tech_lead}\}$ è una seconda chiave candidata valida. Sottoponiamo lo schema al test della BCNF analizzando la seconda dipendenza (`tech_lead` $\rightarrow$ `modulo`): il determinante `tech_lead` è una superchiave dell'intera tabella? **No**, perché un lead gestisce più sviluppatori, quindi il suo nome si ripeterà su più righe. Lo schema viola la BCNF.

Se applicassimo ciecamente l'algoritmo di normalizzazione, dovremmo scomporre la tabella in:

- $R_1(\text{tech_lead}, \text{modulo})$
- $R_2(\text{id_sviluppatore}, \text{tech_lead})$

Questa scomposizione è senza perdita, ma **non conserva le dipendenze**. Avendo separato `id_sviluppatore` e `modulo` in due tabelle diverse, il database non ha più modo di validare nativamente la prima dipendenza (`id_sviluppatore, modulo` $\rightarrow$ `tech_lead`). Un bug nell'applicativo potrebbe inserire dati discordanti senza che il DBMS se ne accorga.

La Soluzione: Definizione di 3NF Per uscire da questo stallo, la teoria relazionale ammette un rilassamento dei vincoli attraverso la **Terza Forma Normale (3NF)**. Essa tollera situazioni come quella appena descritta (accettando una lieve e controllata ridondanza), ma blinda lo schema contro qualsiasi altra fonte di anomalia, garantendo matematicamente sia la decomposizione senza perdita sia la conservazione totale delle dipendenze.

Uno schema relazionale è in 3NF se, per ogni sua dipendenza funzionale non banale $X \rightarrow Y$, si verifica **almeno una** delle due seguenti condizioni:

1. X contiene una chiave (ovvero, X è una superchiave, identico alla BCNF).
2. Y appartiene ad almeno una chiave candidata dello schema.

Osservazione 3.1 (Metodo di riduzione in 3NF). Applicando la definizione al nostro esempio SVILUPPO, la dipendenza "incriminata" era `tech_lead` $\rightarrow$ `modulo`.

- `tech_lead` è una superchiave? **No** (Fallisce il primo criterio).
- `modulo` appartiene a una chiave candidata? **Sì**, fa parte della chiave primaria `{id_sviluppatore, modulo}`. (Soddisfa il secondo criterio).

Di conseguenza, la tabella SVILUPPO originale è **già in 3NF**. Il progettista sceglie consapevolmente di non scomporla: accetta la ridondanza (il modulo del Tech Lead sarà ripetuto per ogni suo sviluppatore) al fine superiore di preservare intatti tutti i vincoli logici.

3.5 Le Forme Normali 1NF e 2NF

Nel moderno sviluppo software, queste due forme normali vengono spesso soddisfatte in modo automatico grazie all'adozione di best practice architetturali (come l'uso di chiavi surrogate), ma le citiamo comunque per completezza.

Prima Forma Normale (1NF): L'Atomicità Un database relazionale puro non ammette l'esistenza di tuple con strutture dati annidate. La **Prima Forma Normale (1NF)** stabilisce un'unica, inderogabile regola:

Una relazione è in 1NF se e solo se il dominio di ciascun attributo è **atomico** (indivisibile) e ogni cella della tabella contiene un singolo valore scalare.

La 1NF è quindi la traduzione di quanto abbiamo già affrontato nel Capitolo 2 durante la ristrutturazione del Class Diagram. Vieta esplicitamente l'uso di *attributi strutturati* (es. una colonna "Indirizzo" che contiene logicamente via, cap e città) e di *attributi multivalore* (es. un array di "Numeri di Telefono" all'interno di una singola cella). Per rispettare la 1NF, il progettista è costretto ad appiattire gli attributi strutturati o a spostare le liste in tabelle esterne (relazioni 1:N).

Seconda Forma Normale (2NF): Eliminazione delle Dipendenze Parziali La 2NF interviene quando una tabella possiede una Chiave Primaria *composta* (formata da due o più attributi).

Uno schema relazionale è in 2NF se è in 1NF e se ogni attributo non-chiave dipende **interamente** da tutta la chiave primaria, e non solo da una parte di essa (assenza di dipendenze parziali).

Esempio 3.7 (Violazione della 2NF). Si consideri una tabella per la registrazione degli esami, in cui la chiave primaria è la coppia `{matricola, codice_corso}`:

ESAME(matricola, codice_corso, voto, nome_studente)

Questo schema viola la 2NF. Mentre l'attributo `voto` dipende interamente da tutta la chiave composta (serve sia la matricola che il corso per stabilire un voto), l'attributo `nome_studente` dipende funzionalmente **solo da una parte** della chiave: la `matricola`. Il `codice_corso` è ininfluente per determinare il nome. Questa dipendenza parziale genera un'anomalia di aggiornamento (il nome dello studente verrebbe ripetuto per ogni esame che sostiene).

La soluzione consiste nel rimuovere `nome_studente` e spostarlo in una tabella `STUDENTE` a sé stante.

Osservazione 3.2 (Perché la 2NF è quasi "estinta" nel Backend moderno). Nel moderno sviluppo applicativo (es. Spring Boot e Hibernate), i progettisti tendono a evitare le chiavi primarie composte, preferendo dotare ogni entità di una **Chiave Surrogata** a singola colonna (es. un ID auto-incrementante o un UUID). Da un punto di vista puramente matematico, se la chiave primaria di una tabella è formata da un solo attributo, **è impossibile che si verifichi una dipendenza parziale**. Pertanto, qualsiasi tabella dotata di una singola chiave surrogata (e priva di array interni) si trova automaticamente e istantaneamente in Seconda Forma Normale.

Introduzione a SQL e Definizione dei Metadati

4.1 Nozioni introduttive

Conclusa la fase di progettazione logica, il progettista ha tra le mani uno schema relazionale formale e matematicamente inattaccabile. Tuttavia, questo schema risiede ancora su carta. Il passo successivo consiste nell'istanziare fisicamente questo modello all'interno di un **DBMS** (Database Management System, come PostgreSQL, MySQL o Oracle).

Per comunicare con il motore del database si utilizza **SQL (Structured Query Language)**, il linguaggio standard universale per la gestione dei database relazionali.

Lo Standard SQL e la Portabilità

Sebbene SQL sia il linguaggio standard per i database relazionali (codificato dagli enti internazionali ANSI e ISO), nella realtà commerciale ogni motore DBMS (PostgreSQL, MySQL, Oracle, ecc.) implementa un proprio "dialetto", introducendo estensioni e funzioni proprietarie non previste dallo standard.

Per questo motivo, il manuale ufficiale di un buon DBMS inizia solitamente dichiarando esplicitamente il proprio livello di aderenza allo standard internazionale. Dal punto di vista dell'ingegneria del software, scrivere codice il più aderente possibile allo standard puro è una *best practice* fondamentale: assicura un'elevata **portabilità** del sistema, garantendo la possibilità di migrare l'infrastruttura da un DBMS a un altro con sforzi e costi di riscrittura minimi. Nel corso della progettazione si farà riferimento esclusivamente alle direttive dell'SQL standard.

Il concetto di Metadato e il Catalogo di Sistema

Prima di poter inserire i dati reali dell'applicazione (es. Mario Rossi, matricola 12345), il DBMS deve conoscere la "forma" che questi dati avranno, le regole a cui devono sottostare e i legami che li uniscono. Queste informazioni strutturali prendono il nome di **Metadati** (letteralmente: *dati che descrivono altri dati*).

Quando si definiscono gli schemi e i vincoli in SQL, il motore del database non sta ancora salvando informazioni applicative, ma sta popolando il proprio **Catalogo di Sistema** (o *Data Dictionary*). Il catalogo è un set di tabelle interne, invisibili all'utente finale, che il DBMS usa per ricordare quante tabelle esistono, come si chiamano le colonne, quali sono i tipi di dato ammessi e quali vincoli di chiave primaria o esterna devono essere imposti.

L'Anatomia di SQL: Un Linguaggio Dichiarativo

A differenza di linguaggi imperativi come Java, C o Python, in cui il programmatore deve descrivere *passo dopo passo* come eseguire un'operazione, SQL è un linguaggio **dichiarativo**: il progettista si limita a dichiarare *cosa* vuole ottenere o *come* deve essere fatta la struttura; sarà il motore di ottimizzazione interno del DBMS a decidere la strategia algoritmica migliore per allocare la memoria o estrarre i dati.

Per gestire l'intero ciclo di vita del dato, il linguaggio SQL è suddiviso in tre grandi sotto-insiemi funzionali (sub-languages):

1. **DDL (Data Definition Language):** È il linguaggio di definizione dei metadati. Viene utilizzato esclusivamente per manipolare la struttura del database (il contenitore). I suoi comandi principali sono:
 - **CREATE:** per generare nuove tabelle, indici o viste.
 - **ALTER:** per modificare la struttura di una tabella esistente (es. aggiungere una colonna).
 - **DROP:** per distruggere fisicamente una struttura e i suoi dati.
2. **DML (Data Manipulation Language):** È il linguaggio utilizzato per manipolare il contenuto (i dati veri e propri) all'interno delle strutture precedentemente create col DDL. I comandi principali sono:
 - **INSERT:** per inserire nuove tuple.
 - **UPDATE:** per aggiornare i valori di tuple esistenti.
 - **DELETE:** per rimuovere tuple.
3. **DQL (Data Query Language):** Spesso considerato parte del DML, è l'istruzione **SELECT**, il cuore pulsante dei database relazionali, utilizzata per interrogare il motore e filtrare, aggregare o unire (**JOIN**) i dati.

4.2 I Domini di Base (Tipi di Dato) nello Standard SQL

Nella creazione di una tabella tramite DDL, ogni attributo deve essere obbligatoriamente associato a un **Dominio** (tipo di dato). Il dominio definisce lo spazio dei valori ammissibili per quella colonna, lo spazio fisico occupato su disco e le operazioni matematiche o logiche consentite.

Lo Standard SQL classifica i tipi di dato in grandi famiglie. Di seguito vengono presentati i domini fondamentali e maggiormente utilizzati.

Dati di tipo Stringa (Caratteri)

Servono per memorizzare testo, codici alfanumerici o numeri su cui non si devono effettuare operazioni matematiche (es. numeri di telefono, CAP, codici fiscali).

- **CHAR(*n*):** Stringa a lunghezza **fissa** pari a *n* caratteri. Se si inserisce una stringa più corta, il DBMS la "riempie" (padding) con spazi vuoti fino a raggiungere *n*.
 - *Uso ingegneristico:* Estremamente veloce per la ricerca. Si usa **solo** quando il dato ha lunghezza rigidamente costante (es. **CHAR(16)** per il Codice Fiscale, **CHAR(2)** per la sigla della Provincia).
- **VARCHAR(*n*):** Stringa a lunghezza **variabile** con limite massimo di *n* caratteri. Se si inserisce una stringa più corta, occupa su disco solo lo spazio effettivo più un marcatore di fine stringa.
 - *Uso ingegneristico:* Lo standard *de facto* per quasi tutto il testo (**VARCHAR(50)** per il nome, **VARCHAR(255)** per l'email, ad esempio). Fa risparmiare enorme spazio su disco, a costo di una microscopica latenza in più durante l'allocazione della memoria.

Dati di tipo Numerico

SQL distingue rigorosamente tra numeri esatti (dove $1.1 + 1.1$ fa esattamente 2.2) e numeri approssimati (soggetti a virgola mobile e possibili errori di precisione).

- **Numerici Esatti Interi:**

- **INT** o **INTEGER**: Numero intero standard con segno (solitamente a 32 bit, range da circa -2 a +2 miliardi). È il re indiscusso delle chiavi primarie surrogate e dei contatori.
- **SMALLINT**: Numero intero "piccolo" (solitamente a 16 bit). Utile per risparmiare RAM se il range di valori è sicuramente ridotto (es. l'anno di immatricolazione, o valori da 1 a 100).
- **Numerici Decimali Esatti**:
 - **NUMERIC(p, s)** o **DECIMAL(p, s)**: Numeri a virgola fissa. Il parametro *p* (*precision*) indica il numero totale di cifre, mentre *s* (*scale*) indica quante di queste cifre stanno dopo la virgola.
 - *Esempio*: **NUMERIC(5, 2)** può memorizzare valori da -999.99 a 999.99.
 - *Uso ingegneristico*: Obbligatorio per i dati **finanziari e monetari** (stipendi, prezzi, bilanci). Garantisce che non ci siano mai arrotondamenti imprevisti.
- **Numerici Approssimati**:
 - **REAL, DOUBLE PRECISION, FLOAT**: Numeri a virgola mobile (standard IEEE 754).
 - *Uso ingegneristico*: Si usano **solo** per calcoli scientifici, coordinate GPS o misurazioni fisiche dove la scala è immensa e una micro-approssimazione (es. 1.9999999 al posto di 2) è tollerabile. Da evitare assolutamente per la valuta.

Dati di tipo Temporale

- **DATE**: Memorizza solo la data (Anno, Mese, Giorno). Formato: 'YYYY-MM-GG'. Ottimo per date di nascita o date di assunzione.
- **TIME**: Memorizza solo l'orario (Ore, Minuti, Secondi). Formato: 'HH-MI-SS'.
- **TIMESTAMP** o **DATETIME**: Memorizza data e ora esatte in un unico campo. Formato: 'YYYY-MM-GG HH-MI-SS'. Fondamentale per i log di sistema, la data di creazione di un record o per tracciare transazioni.

La Rappresentazione Fisica dei Tipi Temporal

Nonostante in fase di interrogazione e inserimento le date vengano espresse in formato testuale (es. 'YYYY-MM-DD'), a livello di immagazzinamento fisico i tipi temporali **non sono stringhe**.

Il DBMS analizza la stringa immessa e la converte in un numero (spesso un intero a 32 o 64 bit che rappresenta i giorni o i millisecondi trascorsi da una specifica data di riferimento, detta *Epoch*). Questa astrazione matematica è cruciale per tre motivi:

1. **Performance aritmetica**: Permette al DBMS di sommare giorni o calcolare intervalli tramite velocissime operazioni matematiche a livello di CPU, evitando il costoso *parsing* testuale.
2. **Ottimizzazione spaziale**: Un **DATE** occupa tipicamente 4 byte, contro i 10 byte di un **CHAR(10)**.
3. **Validazione semantica**: Il DBMS controlla nativamente l'esistenza reale della data (es. impedendo l'inserimento del 30 Febbraio).

Dati di tipo Booleano

- **BOOLEAN**: Ammette solo i valori di verità **TRUE** o **FALSE** (più il valore **NULL** se il campo non è obbligatorio). Ottimo per i flag (es. **is_attivo**, **is_pagato**).

⚠ Il NULL non è un tipo di dato

È fondamentale ricordare che NULL non appartiene a nessun dominio specifico, ma è uno **stato** universale. NULL in SQL non significa "zero" né "stringa vuota", ma rappresenta "informazione mancante, sconosciuta o inapplicabile"; intuitivamente, si può pensare come una variabile incognita (come in una equazione matematica). Qualsiasi tipo di dato descritto sopra, in assenza di vincoli, può assumere lo stato NULL.

🔍 Il supporto al tipo BOOLEAN e l'approccio legacy

Sebbene lo standard SQL preveda nativamente il dominio **BOOLEAN**, nella pratica ingegneristica è necessario tenere conto delle limitazioni specifiche di alcuni DBMS. Il caso storico più celebre è Oracle Database, che non ha supportato il tipo booleano a livello di tabella fino alla versione 23c (2023).

Quando si opera su sistemi legacy o DBMS privi di questo dominio, la *best practice* architetturale consiste nel "simulare" il booleano utilizzando un tipo di dato minimale, come **CHAR(1)** o **SMALLINT/NUMBER(1)**, e "blindarlo" tramite un vincolo di colonna personalizzato (il vincolo **CHECK**, che vedremo a breve) che limiti esplicitamente l'inserimento a due soli valori convenzionali.

4.3 Operatori Logici e di Confronto

Per imporre regole di business (tramite vincoli **CHECK**) o per filtrare i dati durante le interrogazioni, SQL utilizza espressioni condizionali. Un'espressione condizionale valuta il contenuto di una cella e restituisce un risultato booleano a tre vie: **TRUE**, **FALSE** oppure **UNKNOWN** (se coinvolge un valore NULL).

Di seguito i principali operatori standard previsti dal linguaggio:

Operatori di Confronto Standard

Si applicano a stringhe, numeri e date:

- **=** (Uguale)
- **<>** oppure **!=** (Diverso)
- **<**, **>**, **<=**, **>=** (Minore, Maggiore e relativi uguali)

Operatori Logici (Algebra di Boole)

Permettono di combinare più condizioni elementari:

- **AND**: Restituisce **TRUE** solo se *tutte* le condizioni combinate sono vere.
- **OR**: Restituisce **TRUE** se *almeno una* delle condizioni è vera.
- **NOT**: Inverte il risultato logico della condizione che lo segue.

Operatori Speciali di SQL

Sono costrutti sintattici avanzati che semplificano enormemente la scrittura del codice, ottimizzando le performance del motore di ricerca:

- **BETWEEN x AND y**: Verifica se un valore è compreso in un intervallo continuo (inclusi gli estremi x e y). Ottimo per le date o i range numerici.
 - *Esempio*: voto **BETWEEN 18 AND 30** equivale a voto **>= 18 AND voto <= 30**.

- **IN** (*val1*, *val2*, ...): Verifica se un valore appartiene a una lista di elementi discreti.
 - *Esempio*: `provincia IN ('RM', 'MI', 'NA')` equivale a `provincia = 'RM' OR provincia = 'MI' OR provincia = 'NA'`.
- **LIKE** '*pattern*': Permette il confronto testuale parziale (pattern matching) sulle stringhe. Utilizza due caratteri jolly (wildcard):
 - `%` (Percentuale): Sostituisce una sequenza di zero o più caratteri.
 - `_` (Underscore): Sostituisce esattamente un singolo carattere.
 - *Esempio*: `cognome LIKE 'R%'` (Trova tutti i cognomi che iniziano per R); `nome LIKE '_ario'` (Trova Mario, Dario, ma non Ilario).

⚠ La trappola del NULL: IS NULL vs

Per verificare se una cella è vuota, l'intuito del programmatore suggerirebbe di scrivere `colonna = NULL`. In SQL, questa sintassi è **errata** e restituirà sempre UNKNOWN, facendo fallire la condizione. Poiché NULL rappresenta un valore "incognito", è impossibile affermare matematicamente che un'incognita sia "uguale" a un'altra incognita.

L'unico modo legittimo in SQL per verificare l'assenza o la presenza di un dato è utilizzare gli operatori specifici di stato:

- **IS NULL** (Verifica se il campo è vuoto)
- **IS NOT NULL** (Verifica se il campo contiene un valore)

4.4 Istruzioni DDL

Il Data Definition Language (DDL) permette di definire la struttura fisica del database. I due comandi fondamentali per istanziare l'architettura dei dati sono **CREATE SCHEMA** e **CREATE TABLE**.

✓ Convenzione Stilistica (Case Sensitivity)

Lo standard SQL è *case-insensitive* per quanto riguarda le parole chiave del linguaggio. Tuttavia, è convenzione universale nell'ingegneria del software scrivere le keyword in **MAIUSCOLO** (es. `CREATE TABLE`, `VARCHAR`) e gli identificatori definiti dall'utente in **minuscolo** (es. `studenti`, `matricola`). Questo garantisce la massima leggibilità del codice.

4.4.1 Creazione di uno schema logico

CREATE SCHEMA crea quello che in informatica viene chiamato un *namespace* o, più semplicemente, un contenitore logico (un "recinto"). Serve a raggruppare un insieme di tabelle, viste e vincoli che appartengono alla stessa applicazione, separandoli da altri progetti presenti nello stesso server database. È importante notare che una tabella non può fluttuare nel vuoto: deve sempre risiedere all'interno di uno schema. Se non si specifica uno schema personalizzato con **CREATE SCHEMA**, molti DBMS inseriscono automaticamente le tabelle in uno schema di default chiamato comunemente **public** o **dbo**.

La sintassi di base è:

```
1 CREATE SCHEMA <nome_schema>;
```

4.4.2 Creazione di una tabella

Il comando `CREATE TABLE` istanzia fisicamente uno schema relazionale $S(A_1, \dots, A_k)$ associando a ciascun attributo A_i il proprio dominio $dom(A_i)$. Il nome assegnato alla tabella deve essere **unico** all'interno dello schema (o del database, se non si usano schemi espliciti).

La struttura del comando prevede una coppia di parentesi tonde all'interno delle quali si susseguono istruzioni separate da virgole.

Esempio 4.1 (Creazione tabella senza vincoli espliciti). Di seguito si riporta la traduzione in SQL dello schema relazionale relativo agli studenti. In questa prima fase esplorativa, ci limitiamo a definire i nomi degli attributi e i rispettivi domini (tipi di dato), omettendo temporaneamente i vincoli (come la chiave primaria).

```

1 CREATE TABLE studenti (
2     matricola CHAR(9),
3     cf        CHAR(16),
4     nome      VARCHAR(20),
5     cognome   VARCHAR(20),
6     dataN     DATE
7 );

```

Logicamente, il corpo del comando si divide in due blocchi sequenziali:

1. **Definizione degli Attributi:** In questa prima fase si elencano le colonne. Per ogni colonna si deve specificare il nome, il tipo di dato e, opzionalmente, dei vincoli specifici applicabili alla singola cella (vincoli intra-attributo).
2. **Definizione dei Vincoli di Tabella:** In questa seconda fase (che vedremo subito a seguire), si esplicitano i vincoli complessi che coinvolgono più colonne contemporaneamente (come chiavi primarie composte o chiavi esterne).

La sintassi per la definizione di un singolo attributo (la riga base) è la seguente:

`<nome_attributo> <tipo_attributo> [vincoli_attributo]`

Si noti come la scelta dei domini rispecchi quanto discusso nel paragrafo precedente:

- `matricola` e `cf` utilizzano `CHAR` poiché hanno una lunghezza fissa e rigidamente definita (rispettivamente 9 e 16 caratteri).
- `nome` e `cognome` utilizzano `VARCHAR` per ottimizzare lo spazio su disco, dato che la lunghezza reale del testo inserito è variabile.
- `dataN` (data di nascita) utilizza il tipo nativo `DATE` per garantire validazione e correttezza semantica.

4.4.3 I Vincoli Intra-attributo (o in-line)

I vincoli in-line sono regole di integrità applicate direttamente alla definizione di un singolo attributo, sulla stessa riga in cui viene dichiarato il tipo di dato. Servono a limitare i valori ammissibili nella singola cella, agendo come prima barriera contro la corruzione dei dati.

Lo Standard SQL prevede quattro vincoli principali in-line:

NOT NULL (Obbligatorietà) Indica che il campo non può mai assumere lo stato `NULL`. All'atto dell'inserimento di una nuova riga, l'utente è obbligato a fornire un valore valido per questa colonna.

- *Uso:* Si applica a tutti i dati vitali che definiscono l'entità (es. il cognome di una persona, il titolo di un libro).

UNIQUE (Univocità di colonna) Impone che tutti i valori presenti in quella colonna siano distinti tra loro. Il DBMS rifiuterà qualsiasi tentativo di inserire una riga con un valore già esistente in un'altra riga per quel campo.

- *Uso:* Si applica alle chiavi naturali candidate (es. il Codice Fiscale, l'Email). Nello standard SQL, a meno di specifiche implementazioni dei singoli DBMS, una colonna **UNIQUE** consente l'inserimento di molteplici valori **NULL**, poiché **NULL** rappresenta una incognita e due incognite non sono considerate uguali tra loro.

PRIMARY KEY (Chiave Primaria a livello di colonna) Elege l'attributo a identificatore principale della tabella. Specificare **PRIMARY KEY** su una colonna implica contemporaneamente i vincoli **NOT NULL** e **UNIQUE**. Una tabella può avere **una sola** chiave primaria.

- *Uso:* Si usa quando la chiave primaria è composta da un solo attributo (es. una chiave surrogata come **id** o una chiave naturale singola come **matricola**).

DEFAULT (Valore predefinito) Non è un vincolo di integrità in senso stretto, ma una direttiva di assegnazione. Se durante un inserimento (**INSERT**) l'utente omette il valore per questa colonna, il DBMS vi inserisce automaticamente il valore specificato dopo la keyword **DEFAULT**.

- *Uso:* Ottimo per impostare stati iniziali (es. **is_attivo** **BOOLEAN** **DEFAULT** **TRUE**) o timestamp di creazione (es. **data_inserimento** **DATE** **DEFAULT** **CURRENT_DATE**).

Esempio 4.2 (Evoluzione dello schema studenti con vincoli di colonna). Riprendiamo lo schema della tabella **studenti** e applichiamo i vincoli intra-attributo per blindarne la coerenza semantica ed evitare anomalie:

```

1 CREATE TABLE studenti (
2     matricola CHAR(9) PRIMARY KEY,
3     cf        CHAR(16) UNIQUE NOT NULL,
4     nome      VARCHAR(20) NOT NULL,
5     cognome   VARCHAR(20) NOT NULL,
6     dataN     DATE
7 );

```

Analisi delle scelte effettuate:

- **matricola:** È l'identificatore principale. Diventa **PRIMARY KEY**, quindi sarà univoca e obbligatoria per definizione.
- **cf:** Il codice fiscale deve essere univoco (**UNIQUE**) per evitare duplicati nella realtà, ma deve anche essere obbligatorio (**NOT NULL**) per ogni studente registrato.
- **nome e cognome:** Viene applicato **NOT NULL** perché un'istanza di studente priva di identità anagrafica non avrebbe alcun significato logico.
- **dataN:** Rimane priva di vincoli espliciti; questo significa che il sistema accetterà il valore **NULL** qualora la data di nascita non fosse momentaneamente disponibile al momento della registrazione.

4.4.4 I Vincoli di Tabella (Out-of-line)

Fino a questo momento abbiamo visto i vincoli applicati direttamente sulla stessa riga della definizione dell'attributo (sintassi *in-line*). Tuttavia, lo standard SQL permette (e in molti casi impone) di dichiarare i vincoli in una sezione dedicata alla fine del comando

CREATE TABLE, separata dalla definizione delle colonne. Questa formulazione prende il nome di **Vincolo di Tabella** (o *out-of-line*).

La sintassi generale per dichiarare un vincolo a livello di tabella è:

```
CONSTRAINT <nome_vincolo> <TIPO_VINCOLO> (colonna1, colonna2, ...)
```

Sebbene l'indicazione del nome del vincolo sia opzionale (se omessa, il DBMS genera internamente un nome alfanumerico casuale, es. SYS_C00142), nella pratica si raccomanda di assegnare sempre un nome esplicito e semantico (es. `pk_studenti` o `unq_cf`).

Nominare i vincoli è fondamentale per due operazioni architetturali:

1. **Leggibilità degli errori:** Se un'applicazione viola una regola, il database genererà un'eccezione specificando il nome del vincolo. Leggere "*Violazione del vincolo unq_cf*" permette al programmatore di capire istantaneamente che c'è un problema di codici fiscali duplicati, senza dover fare debug alla cieca.
2. **Manipolazione a posteriori (Enable/Disable):** Quando il vincolo viene dichiarato, il DBMS lo crea e lo attiva automaticamente. Tuttavia, conoscendone il nome, l'amministratore del database può manipolarlo in futuro senza distruggere i dati. Tramite istruzioni specifiche (es. `ALTER TABLE ... DISABLE CONSTRAINT`), un vincolo può essere momentaneamente spento. Questa operazione è comunissima durante l'importazione massiva di milioni di dati (*bulk insert*), dove spegnere i controlli accelera drasticamente la scrittura, per poi riattivarli (*ENABLE*) a importazione finita.

La motivazione tecnica principale che rende indispensabile la sintassi a livello di tabella è la gestione dei vincoli multi-colonna.

Nel nostro esempio precedente, la chiave primaria era formata da un solo attributo (*matricola*). Ma cosa accade se la progettazione logica produce una **chiave primaria composta** (es. una tabella che memorizza gli esami superati da uno studente in un determinato corso)? In SQL è illegale scrivere PRIMARY KEY su due righe separate, perché il DBMS tenterebbe di creare *due* chiavi primarie distinte per la stessa tabella. In questo scenario, **l'uso del vincolo di tabella diventa strutturalmente obbligatorio**, poiché permette di racchiudere più colonne sotto un'unica regola.

Esempio 4.3 (Riscrittura e Chiavi Composte). Mostriamo la sintassi applicata allo schema `studenti` e a una nuova tabella associativa `esami_sostenuti`.

```

1  -- Esempio 1: Riscrittura con vincoli di tabella nominati
2  CREATE TABLE studenti (
3      matricola CHAR(9),
4      cf        CHAR(16) NOT NULL, -- NOT NULL resta solitamente in
      -line
5      nome      VARCHAR(20) NOT NULL,
6      cognome    VARCHAR(20) NOT NULL,
7      dataN      DATE,
8
9      -- Sezione Vincoli di Tabella
10     CONSTRAINT pk_studenti PRIMARY KEY (matricola),
11     CONSTRAINT uq_studenti_cf UNIQUE (cf)
12 );
13
14 -- Esempio 2: Obbligatorietà del vincolo per PK composta
15 CREATE TABLE esame_sostenuto (

```



```
16     studente_matr CHAR(9),
17     codice_corso  CHAR(5),
18     voto          INT NOT NULL,
19
20     -- La PK unisce due attributi e deve essere definita qui
21     CONSTRAINT pk_esami PRIMARY KEY (studente_matr, codice_corso)
22 );
```



Convenzione e Best Practice

Come regola pratica di sviluppo: il vincolo d'obbligatorietà (**NOT NULL**) si definisce **sempre e solo in-line** accanto all'attributo. Al contrario, i vincoli strutturali d'identità (**PRIMARY KEY**, **UNIQUE**) e quelli di navigabilità (**FOREIGN KEY**, che vedremo a breve) vengono quasi sempre traslati a fine tabella e nominati esplicitamente, per favorire l'ordine mentale e la manutenzione del codice.

4.4.5 Classificazione Concettuale dei Vincoli

Nei paragrafi precedenti abbiamo analizzato una classificazione prettamente *sintattica* (vincoli *in-line* vs *out-of-line*), che riguarda esclusivamente la forma e il posizionamento del codice all'interno dell'istruzione **CREATE TABLE**.

Esiste tuttavia una classificazione ben più importante, di natura **concettuale**, che categorizza i vincoli in base al loro "raggio d'azione", ovvero alla porzione di dati che il DBMS deve leggere per poter verificare se il vincolo è rispettato o violato. Secondo questa logica, i vincoli si dividono in quattro categorie a complessità crescente:

1. **Vincoli di Dominio (o di colonna):** Esprimono condizioni sui valori ammissibili per un singolo attributo, valutando la singola cella isolata dal resto. Sono i vincoli più leggeri da verificare per il motore del database.
 - *Osservazione sintattica:* Questo è tipicamente l'unico tipo di vincolo che ha senso logico definire *in-line*.
 - *Esempi già incontrati:* Il vincolo di obbligatorietà (**NOT NULL**) e l'assegnazione di un valore predefinito (**DEFAULT**). Per verificare se un campo è nullo, il DBMS deve guardare solo quella specifica cella.
2. **Vincoli di tupla (o di riga):** Esprimono condizioni logiche che coinvolgono contemporaneamente più attributi appartenenti alla stessa riga (tupla). Per verificare il vincolo, il DBMS deve estrarre l'intera riga, ma non ha bisogno di confrontarla con le altre righe della tabella.
3. **Vincoli Intra-relazionali (o di tabella):** Esprimono condizioni che coinvolgono più record all'interno della stessa tabella. Per validare il nuovo dato, il DBMS è costretto a scansionare o interrogare le altre righe già presenti.
 - *Esempi già incontrati:* **UNIQUE** e **PRIMARY KEY**. Per garantire che un Codice Fiscale sia univoco, non basta guardare la riga appena inserita; il database deve verificare che quel valore non sia già presente in nessun altro record della tabella.
4. **Vincoli Inter-relazionali (o di Schema):** Esprimono condizioni che legano tra loro record appartenenti a due o più tabelle distinte. Sono i vincoli più complessi e costosi computazionalmente, in quanto costringono il DBMS a "navigare" tra relazioni diverse per validare l'integrità strutturale del database.

Esempio 4.4 (Istanziamento della Tabella Esami e Classificazione dei Vincoli). Esempifichiamo la classificazione concettuale dei vincoli modellando la tabella **esami**, la quale memorizza la registrazione definitiva degli esami superati dagli studenti.

Dal punto di vista della *business logic*:

- La chiave primaria deve essere composta dalla coppia (**matr**, **corso**), garantendo che uno studente possa registrare un determinato esame una sola volta.
- Il voto deve essere unicamente compreso tra 18 e 30 (esame superato).
- La lode (valori 'S'/'N', default 'N') può essere assegnata solo se il voto è pari a 30.
- La matricola deve fare riferimento a uno studente reale.

Ecco il codice SQL DDL definitivo:

```

1 CREATE TABLE esame (
2     matr CHAR(9),
3     corso VARCHAR(20) NOT NULL,
4     data DATE NOT NULL,
5     voto INTEGER NOT NULL,
6     lode CHAR(1) DEFAULT 'N',
7
8     -- Sezione Vincoli di Tabella
9     CONSTRAINT pk_esame PRIMARY KEY (matr, corso),
10
11     CONSTRAINT chk_voto_superato CHECK (voto BETWEEN 18 AND 30),
12
13     CONSTRAINT chk_lode_coerenza CHECK (lode IN ('S', 'N') AND (
14         lode = 'N' OR voto = 30)),
15
16     CONSTRAINT fk_esame_studente FOREIGN KEY (matr)
17         REFERENCES studente(matricola)
18 );

```

Analizziamo ogni singolo vincolo applicato alla tabella **esami**, esplicitando la categoria concettuale di appartenenza in base al suo spettro d'azione:

1. Vincoli di Dominio (o di colonna):

- NOT NULL (su **corso**, **data**, **voto**): Sono vincoli di dominio in-line. Impongono che le singole celle non siano vuote. Il DBMS verifica la singola cella isolata.
- DEFAULT 'N' (su **lode**): È una direttiva di dominio che interviene sulla singola cella in assenza di dato.
- **chk_voto_superato** (CHECK (voto BETWEEN 18 AND 30)): È un vincolo di dominio out-of-line. Anche se scritto in fondo, analizza **esclusivamente** i valori dell'attributo **voto**, isolato dal resto della riga.

2. Vincoli di n-pla (o di tupla/riga):

- **chk_lode_coerenza**: Questo vincolo impone che il carattere sia 'S' o 'N' e che, qualora sia 'S', il voto sia tassativamente 30. Dal momento che l'espressione logica mette in relazione **due attributi distinti dello stesso record** (**lode** e **voto**), si tratta di un puro vincolo di n-pla. Il DBMS deve leggere l'intera riga per validarlo.

3. Vincoli Intra-relazionali (o di Tabella):

- **pk_esame** (PRIMARY KEY (**matr**, **corso**)): Definisce l'identificatore. Per garantire che la coppia **matricola**-**corso** sia unica, il DBMS non può limitarsi

a guardare la riga corrente, ma deve scansionare l'intera tabella **esame** per escludere duplicati. Agisce quindi a livello di **più record della stessa tabella**.

4. Vincoli Inter-relazionali (o di Schema):

- **fk_esame_studente (FOREIGN KEY)**: È il vincolo che garantisce l'integrità referenziale. Stabilisce un legame tra la tabella corrente (**esami**) e una tabella esterna (**studente**). Il database, per accettare la riga, è costretto a saltare fuori dalla tabella corrente e verificare la presenza della matricola nello schema dello studente. **Questo è l'unico tipo di vincolo inter-relazionale che si può inserire in una tabella.**

4.4.6 Integrità Referenziale e Chiavi Esterne (FOREIGN KEY)

Il vincolo di **Chiave Esterna (FOREIGN KEY)** è il più importante tra i vincoli **inter-relazionali** (o di schema). Esso stabilisce una relazione di dipendenza tra due tabelle, imponendo che i valori presenti in un set di attributi della tabella corrente (*tabella figlia* o *referenziante*) debbano obbligatoriamente esistere all'interno del set di attributi che costituisce la chiave primaria (o un'altra chiave univoca) di un'altra tabella (*tabella padre* o *referenziata*).

Come abbiamo visto nel paragrafo precedente, la sintassi standard a livello di tabella è la seguente:

```
1 CONSTRAINT <nome_fk> FOREIGN KEY (colonna_locale_1, ...)
2   REFERENCES <tabella_padre> (colonna_padre_1, ...)
```



Ordine Posizionale degli Attributi Referenziati

Nel caso in cui la relazione coinvolga chiavi composte da più attributi (es. una tabella figlia che punta alla nostra tabella **esami**, la cui Primary Key è la coppia **matr, corso**), l'**ordine** in cui le colonne vengono dichiarate all'interno delle parentesi tonde è tassativo.

L'associazione effettuata dal motore SQL **non avviene per nome, ma è strettamente posizionale** (uno-a-uno): il primo attributo elencato in **FOREIGN KEY** viene mappato sul primo attributo elencato in **REFERENCES**, il secondo sul secondo, e così via. Un'inversione accidentale dell'ordine genererà un errore di compilazione (se i domini non corrispondono) o, nel peggiore dei casi, una drammatica corruzione semantica dei dati referenziati.

Il problema delle modifiche sui dati referenziati

Il vero fulcro ingegneristico della chiave esterna emerge quando si tenta di alterare lo stato della tabella padre. Supponiamo che nel nostro database esista uno studente con matricola '12345' e che nella tabella **esami** vi siano tre record associati a tale matricola.

Cosa accade se un operatore tenta di **cancellare** lo studente '12345' dalla tabella **studenti**, o di **modificarne** la matricola? Se il DBMS permettesse questa operazione alla leggera, i tre record della tabella **esami** punterebbero a una matricola inesistente.

Per risolvere questo problema, lo Standard SQL impone al progettista di specificare esplicitamente una **politica di reazione** sia per la cancellazione (**ON DELETE**) sia per l'aggiornamento (**ON UPDATE**).

Il progettista può scegliere tra quattro comportamenti distinti, da dichiarare in coda al vincolo di chiave esterna:

1. **RESTRICT / NO ACTION (Comportamento di Default):** Il DBMS blocca l'operazione alla radice. Se si tenta di cancellare lo studente padre, il database lancia un'eccezione di violazione di vincolo e rifiuta l'operazione, costringendo l'utente a cancellare manualmente prima tutti gli esami correlati.
 - Sintassi: `ON DELETE RESTRICT`
2. **CASCADE (Cancellazione/Modifica a cascata):** L'operazione eseguita sul padre si ripercuote automaticamente sui figli. Se cancello lo studente, il DBMS cancella autonomamente tutte le righe dei suoi esami. Se modifico la matricola del padre, il DBMS aggiorna la matricola in tutti gli esami figli.
 - Sintassi: `ON DELETE CASCADE`
3. **SET NULL (Annullamento del puntatore):** Se il padre viene eliminato o modificato, i record figli sopravvivono, ma il valore della loro chiave esterna viene impostato automaticamente a NULL.
 - Sintassi: `ON DELETE SET NULL`
 - *Vincolo strutturale:* Questa politica è applicabile **solo se** le colonne della chiave esterna nella tabella figlia ammettono il valore NULL (ovvero non sono marcate come NOT NULL o non fanno parte della Primary Key).
4. **SET DEFAULT (Ripristino del valore predefinito):** Simile a SET NULL, ma i campi della chiave esterna nei record figli assumono il valore specificato nella clausola DEFAULT di quella colonna.
 - Sintassi: `ON DELETE SET DEFAULT`

Euristiche di scelta delle politiche

La selezione della politica referenziale dipende esclusivamente dal significato semantico della relazione:

- Si usa **CASCADE** quando la tabella figlia descrive entità deboli che non hanno senso di esistere senza il padre (es. le righe di un ordine d'acquisto: se cancello l'ordine, devo distruggere le sue righe).
- Si usa **RESTRICT** quando l'eliminazione del padre sarebbe un errore operativo grave (es. non devo poter cancellare un corso di laurea se ci sono ancora studenti iscritti ad esso).
- Si usa **SET NULL** quando l'entità figlia ha vita propria ma perde il legame con il padre (es. se una scuderia di Formula 1 fallisce e si cancella la scuderia, i piloti rimangono nel database ma con la colonna `id_scuderia` impostata a NULL).

Dichiarazione Multipla e Comportamenti Misti

È prassi comune assegnare comportamenti logici diversi per la cancellazione e per l'aggiornamento. Sintatticamente, le due clausole si scrivono in sequenza all'interno della dichiarazione del vincolo.

Esempio di combinazione (Best Practice industriale):

```

1 CONSTRAINT fk_esami_studenti FOREIGN KEY (matr)
2   REFERENCES studenti(matricola)
```



4

```
ON DELETE RESTRICT    -- Protegge dalla perdita
                        accidentale dei dati
ON UPDATE CASCADE     -- Garantisce flessibilita'
                        operativa sui codici
```

In questo specifico (e comunissimo) scenario, il DBMS impedirà la cancellazione di uno studente che ha già sostenuto esami, ma permetterà alla segreteria di correggerne o aggiornarne la matricola, propagando istantaneamente il nuovo codice in tutte le righe della tabella **esami**.



Design Pattern: Il Record Fittizio (Dummy Record)

Durante la modellazione fisica, la gestione dei valori NULL nelle chiavi esterne può generare complessità in fase di interrogazione dell'applicativo (necessità di *Outer Join* e gestione delle incognite).

Per ovviare a questo problema, i progettisti adottano spesso il pattern del **Record Fittizio**. La tecnica consiste nell'inserire nella tabella padre una tupla convenzionale (es. con Primary Key 0 o -1 e descrizione "Non Assegnato"). Nella tabella figlia, la colonna dedicata alla chiave esterna viene dichiarata NOT NULL (impedendo l'incognita) e le viene assegnato come DEFAULT l'identificatore del record fittizio.

Vantaggi architettonici:

- **Integrità totale:** Ogni riga figlia punta sempre a un record padre esistente, eliminando il concetto di "puntatore vuoto".
- **Sinergia con le politiche referenziali:** Questo pattern rende implementabile in modo logico la direttiva ON DELETE SET DEFAULT. Se un'entità padre viene eliminata, i record figli ad essa associati non assumono il valore NULL, ma vengono automaticamente riassegnati al record "Sconosciuto" (valore di default), preservando la continuità strutturale dei dati.



In alcuni contesti didattici, con **Vincolo di Integrità Referenziale** si intende che la FK nella tabella figlia debba **sempre** puntare ad una tupla nella tabella padre, eventualmente un dummy record (ad esempio, nel caso in cui la FK sia NULL).

4.4.7 Identificatori Univoci: Chiavi Surrogate e il costrutto IDENTITY

Nella modellazione di un database relazionale, ogni riga di una tabella deve poter essere identificata in modo univoco (Chiave Primaria). In molti scenari reali, i dati "natural" di un'entità non sono adatti a questo scopo: i nomi omonimi esistono, le email possono essere cambiate e persino i codici fiscali possono essere soggetti a ricalcolo.

Come sappiamo, per garantire un'identificazione robusta e immutabile nel tempo, l'ingegneria del software ricorre quasi sempre alle **Chiavi Surrogate**: attributi artificiali, tipicamente un numero intero progressivo (es. 1, 2, 3...), privi di qualsiasi significato logico per l'utente, ma utilizzati dal motore relazionale per collegare le tabelle tra loro in modo sicuro.

Q L'anti-pattern del $\text{MAX}(\text{id}) + 1$

Perché non possiamo semplicemente calcolare il nuovo ID con una query del tipo *"Prendi il numero massimo attuale e aggiungi 1"* ($\text{MAX}(\text{id}) + 1$)? In un'applicazione aziendale, centinaia di utenti potrebbero tentare di registrarsi contemporaneamente. Se due transazioni eseguissero $\text{MAX}(\text{id}) + 1$ nello stesso esatto millisecondo, leggerebbero entrambe lo stesso numero (es. 100), calcolerebbero lo stesso nuovo ID (101) e cercherebbero di inserirlo. Il database andrebbe in crash per violazione della chiave primaria.

Per risolvere questo problema serviva un generatore di numeri **atomico e isolato** dalle transazioni: un oggetto capace di "staccare un biglietto" diverso per ogni richiesta, garantendo che due richieste simultanee non ricevano mai lo stesso numero. Questo oggetto matematico prende il nome di **Sequenza** (SEQUENCE).

La Soluzione dello Standard SQL Moderno Nello standard SQL moderno (pienamente supportato da PostgreSQL 10+, Oracle 12c+ e DB2), non è più necessario creare e gestire manualmente l'oggetto Sequenza. Il linguaggio offre un costrutto dichiarativo estremamente elegante da inserire direttamente nella definizione della tabella: la clausola **IDENTITY**.

Quando si dichiara una colonna come **IDENTITY**, il DBMS crea in automatico, "sotto il cofano", una sequenza blindata e la associa indissolubilmente a quell'attributo.

Sintassi e Comportamento Esistono due varianti sintattiche principali che ne determinano il comportamento durante l'istruzione **INSERT**:

1. GENERATED ALWAYS AS IDENTITY Questa è la forma più restrittiva e sicura. Il motore relazionale prende il controllo totale della colonna.

```
1 CREATE TABLE utente (
2     id_utente INT GENERATED ALWAYS AS IDENTITY PRIMARY KEY,
3     nome VARCHAR(50),
4     email VARCHAR(100)
5 );
```

In fase di popolamento, lo sviluppatore **deve obbligatoriamente ignorare** la colonna `id_utente`:

```
1 -- L'ID verrà generato e inserito autonomamente dal motore
2 INSERT INTO utente (nome, email) VALUES ('Mario Rossi', '
   mario@email.it');
```

Se si tenta di forzare l'inserimento manuale di un ID (es. `VALUES (15, 'Mario', ...)`), il DBMS bloccherà l'operazione restituendo un errore, preservando così l'integrità del contatore numerico.

2. GENERATED BY DEFAULT AS IDENTITY Questa variante agisce come un paracadute. Il motore genererà il numero progressivo in automatico, ma se l'applicazione ha un motivo specifico per forzare l'inserimento di un ID manuale (ad esempio, durante il ripristino di un backup o la migrazione da un vecchio database), il DBMS lo accetterà.

Q I "buchi" di numerazione

Una delle regole più controintuitive per chi si affaccia ai database riguarda i "buchi" (*gaps*) nelle sequenze. Se una transazione "stacca" l'ID numero 105 per inserire un

Q utente, ma l'inserimento fallisce (es. perché l'email violava un vincolo UNIQUE), o se la riga viene successivamente cancellata con una DELETE, **l'ID 105 è perso per sempre**. L'inserimento successivo riceverà il 106.

Questo comportamento non è un difetto, ma una caratteristica fondamentale per garantire alte prestazioni. Se le sequenze dovessero aspettare l'esito di ogni transazione per riutilizzare i numeri scartati, creerebbero colli di bottiglia catastrofici. Le chiavi surrogate devono unicamente garantire l'univocità, non la continuità numerica assoluta.

SQL e Algebra Relazionale

In questo capitolo studieremo in parallelo ulteriori elementi del linguaggio SQL e l'algebra relazionale; non si tratta di un mero esercizio didattico, ma la chiave di volta per comprendere la reale architettura di calcolo di un Database Management System (DBMS).

Esiste una dicotomia fondamentale tra i due strumenti, che si riflette nella natura stessa dei linguaggi:

- **L'Algebra Relazionale è procedurale (o costruttiva):** Definisce rigorosamente la sequenza dei passi (le operazioni elementari come selezioni, proiezioni e join) necessari per manipolare le relazioni e ottenere il risultato.
- **SQL è un linguaggio di alto livello dichiarativo (o descrittivo):** Il programmatore si limita a definire le proprietà dell'insieme dei dati che desidera ottenere (*cosa*), ignorando del tutto la strategia fisica per reperirli (*come*). Un'unica istruzione **SELECT** agisce come un costrutto sintattico che incapsula molteplici operazioni algebriche.

Comprendere l'algebra relazionale significa avere la visibilità strutturale su ciò che il DBMS esegue "sotto il cofano". Quando il sistema riceve una **SELECT**, innesca un processo di traduzione e ottimizzazione totalmente trasparente all'utente:

1. **Parsing e Traduzione:** L'istruzione SQL viene analizzata e tradotta in un albero sintattico di espressioni di algebra relazionale.
2. **Ottimizzazione Algebrica (Query Optimizer):** È il motore matematico del DBMS. L'ottimizzatore sfrutta le proprietà di equivalenza degli operatori relazionali (es. commutatività, associatività e distributività) per trasformare l'espressione iniziale in un'espressione logicamente equivalente ma con un costo computazionale minimo (ad esempio, eseguendo le selezioni per ridurre la cardinalità degli insiemi prima di effettuare un prodotto cartesiano).
3. **Piano di Esecuzione (Execution Plan):** L'espressione algebrica ottimizzata viene mappata in operazioni fisiche di accesso alla memoria (o al disco) e infine eseguita.

Q Limiti di scopo e Amministrazione della Base di Dati

I DBMS relazionali mettono a disposizione dei progettisti istruzioni di basso livello (dette *Hint*) per manipolare e forzare manualmente i piani di esecuzione, esautorando di fatto il Query Optimizer dalle sue scelte automatiche.

Tuttavia, queste operazioni di *Query Tuning* fisico appartengono a un dominio avanzato (tipico dell'amministrazione di database o di corsi di livello magistrale). L'obiettivo di questo documento si limita al livello logico-dichiarativo: ci concentreremo sulla **correttezza semantica** delle interrogazioni scritte, delegando completamente l'efficienza procedurale e la gestione algoritmica al motore del DBMS.

Prima di introdurre le operazioni algebriche, è imperativo distinguere l'aspetto intensionale (la struttura) dall'aspetto estensionale (i dati) di una base di dati:

- **Schema Relazionale (Intensione):** Rappresenta la struttura statica della tabella. Definisce il nome della relazione e l'elenco dei suoi attributi con i rispettivi domini. Esempio: *STUDENTE(matricola, nome, cognome, dataN, città)*.

- **Relazione (Estensione):** Corrisponde all'istanza dello schema in un dato istante di tempo, ovvero alle righe della tabella. Dal momento che il modello si fonda sulla teoria degli insiemi, una relazione è rigorosamente un **insieme di tuple**.

matricola	nome	cognome	dataN	citta	lavoro
10001	Mario	Rossi	1999-05-14	Roma	NULL
10002	Luigi	Verdi	2000-11-22	Milano	Barista
10003	Giulia	Rossi	2001-02-10	Roma	NULL
10004	Anna	Bianchi	1999-08-30	Napoli	Developer
10005	Mario	Neri	2000-01-15	Milano	NULL

Tabella 5.1: Istanza aggiornata della relazione **Studente**, con l'attributo parziale **Lavoro**.

Osservazione 5.1 (Proprietà Insiemistiche della Relazione). Ricordiamo che, trattandosi di un insieme matematico, per definizione:

1. **Non ammette duplicati:** Non possono esistere due tuple identiche in ogni loro attributo.
2. **L'ordine è irrilevante:** Cambiando l'ordine fisico delle righe, l'insieme rimane matematicamente lo stesso.

5.1 Proiezioni

La proiezione è un operatore **unario** (agisce su una singola relazione) che effettua una "decomposizione verticale". In termini pratici, estrae una sottotabella conservando solo gli attributi specificati e scartando gli altri.

Sia $R(X)$ uno schema di relazione, e sia $Y \subseteq X$ un sottoinsieme dei suoi attributi. La proiezione della relazione r sull'insieme di attributi Y è una relazione sullo schema $R(Y)$ che si denota con $\pi_Y(r)$.

- **Schema risultante:** Lo schema della relazione proiettata contiene meno attributi (o al massimo gli stessi) della relazione di partenza. A differenza dell'ordine delle tuple (irrilevante), l'**ordine degli attributi** specificato nell'operatore definisce la struttura della nuova relazione (es. $\pi_{nome,cognome}(r)$ produce uno schema diverso da $\pi_{cognome,nome}(r)$).
- **Riduzione della cardinalità (De-duplicazione):** Poiché il risultato dell'operazione deve essere ancora una relazione (un insieme), il numero di tuple della proiezione può essere inferiore al numero di tuple della relazione originale. Se la rimozione delle colonne genera tuple identiche, l'operatore matematico "collassa" i duplicati in un unico elemento.
- **Composizione:** Essendo chiusa rispetto all'algebra, è possibile applicare proiezioni in cascata. Risulta banale dimostrare che se $Z \subseteq Y \subseteq X$, allora $\pi_Z(\pi_Y(r)) = \pi_Z(r)$.

Esempio 5.1. Applichiamo l'operatore di proiezione all'istanza della relazione **Studente** precedentemente definita in tabella 5.1:

$\pi_{nome,cognome}(\text{studente})$	$\pi_{cognome,nome}(\text{studente})$	$\pi_{cognome}(\text{studente})$
nome cognome	cognome nome	cognome
Mario Rossi	Rossi Mario	Rossi
Luigi Verdi	Verdi Luigi	Verdi
Giulia Rossi	Rossi Giulia	Bianchi
Anna Bianchi	Bianchi Anna	Neri
Mario Neri	Neri Mario	

Tabella 5.2: Confronto delle proiezioni: l'ultima operazione evidenzia la riduzione della cardinalità (da 5 a 4 tuple) dovuta alla de-duplicazione insiemistica.

5.1.1 La Proiezione in SQL: Il comando SELECT

Il linguaggio SQL implementa l'operatore di proiezione tramite la clausola **SELECT**. Il risultato di un'interrogazione non viene salvato fisicamente sul disco (a meno di comandi DDL espliciti), ma costituisce una **tabella virtuale** (o *Result Set*) che risiede temporaneamente nella memoria di lavoro.

Sintassi generale della proiezione SQL:

```

1 SELECT [ALL | DISTINCT] <elenco_attributi>
2 FROM <nome_tabella>;

```

Esiste una divergenza critica tra l'operatore matematico π e la clausola **SELECT**:

- **SELECT ALL (Comportamento di default):** SQL, per ottimizzare le prestazioni computazionali, tratta le tabelle come *multinsiemi* (bag). Se calcoliamo **SELECT cognome FROM studente**, e nel database esistono dieci studenti di nome "Rossi", la tabella virtuale risultante conterrà dieci righe identiche. Questo viola la definizione di insieme.
- **SELECT DISTINCT:** Per forzare il DBMS a comportarsi in modo rigoroso secondo le regole dell'algebra relazionale, è necessario utilizzare la keyword **DISTINCT**. Questa impone al motore di ordinare i risultati ed eliminare i duplicati, restituendo un insieme matematico puro (a fronte di un maggiore costo di calcolo).

5.2 Selezioni

La selezione è, come la proiezione, un operatore **unario** che estrae un sottoinsieme delle tuple che soddisfano una determinata condizione logica, lasciando inalterata la struttura della tabella (a differenza delle proiezioni).

Sia $R(X)$ uno schema di relazione e sia p un predicato (condizione logica) definito sugli attributi di X . La selezione della relazione r rispetto al predicato p è una relazione sullo stesso schema $R(X)$ che si denota con $\sigma_p(r)$.

- **Schema risultante:** Lo schema della relazione selezionata è identico allo schema della relazione di partenza. Il numero, il nome e l'ordine degli attributi non subiscono variazioni.

- **Riduzione della cardinalità:** Il risultato è un sottoinsieme insiemistico della relazione originale. Il numero di tuple prodotte è minore o uguale al numero di tuple di partenza.
- **Composizione:** L'applicazione di selezioni in cascata equivale a una singola selezione il cui predicato è la congiunzione logica (AND) dei singoli predicati. Risulta banale dimostrare che $\sigma_{p_1}(\sigma_{p_2}(r)) = \sigma_{p_1 \wedge p_2}(r)$.

Esempio 5.2. Applichiamo l'operatore di selezione all'istanza della relazione **studente** (Tabella 5.1) per estrarre le sole tuple relative agli studenti residenti a Roma:

$$\sigma_{citta='Roma'}(\text{studente})$$

matricola	nome	cognome	dataN	citta	lavoro
10001	Mario	Rossi	1999-05-14	Roma	NULL
10003	Giulia	Rossi	2001-02-10	Roma	NULL

Tabella 5.3: Risultato della selezione: lo schema resta inalterato (6 attributi), ma la cardinalità si riduce alle sole tuple che rendono vero il predicato.

5.2.1 La Selezione in SQL: La clausola WHERE

In SQL, l'operazione logica di selezione si implementa aggiungendo la clausola **WHERE** al blocco di interrogazione. Il predicato dell'algebra relazionale viene letteralmente tradotto in un'espressione condizionale.

Sintassi generale della selezione SQL:

```

1 SELECT *
2 FROM <nome_tabella>
3 WHERE <condizione>;

```

Nelle interrogazioni SQL, l'asterisco (*) agisce come un carattere jolly (wildcard) universale che indica **"tutti gli attributi"**.

Dal punto di vista dell'algebra relazionale, la clausola **SELECT** implementa l'operatore di proiezione (π). Tuttavia, per limiti sintattici storici del linguaggio SQL, un'interrogazione deve *obbligatoriamente* iniziare con il comando **SELECT**, anche qualora il progettista desideri eseguire unicamente un'estrazione orizzontale (selezione σ), mantenendo intatta la struttura della tabella.

L'asterisco fornisce la scappatoia formale a questo vincolo sintattico: comunica al motore del DBMS di **non effettuare alcuna operazione di decomposizione verticale**, restituendo la tupla nella sua interezza.

- **Proiezione esplicita:** **SELECT nome, cognome** equivale a $\pi_{nome,cognome}(r)$
- **Nessuna proiezione:** **SELECT *** preserva lo schema $R(X)$ originale inalterato.

Pertanto, l'espressione algebrica composta esclusivamente da una selezione

$\sigma_{citta='Roma'}(\text{studente})$ si traduce in SQL in:

```

1 SELECT * FROM studente
2 WHERE citta = 'Roma';

```

⚠ L'anti-pattern del SELECT *

Sebbene l'uso di `SELECT *` sia estremamente comodo durante l'ispezione manuale dei dati, il suo impiego all'interno del codice sorgente di un'applicazione software (Backend) è universalmente considerato un **anti-pattern** per due ragioni architetturali:

1. **Spreco computazionale (I/O e Rete):** Trasferire colonne non necessarie all'applicativo consuma inutilmente larghezza di banda e memoria RAM.
2. **Fragilità strutturale (Breaking Changes):** Se in futuro lo schema del database dovesse evolvere (`ALTER TABLE`) con l'aggiunta di nuove colonne pesanti (es. dati binari) o sensibili, la vecchia query inizierebbe silenziosamente a scaricarle, potendo causare saturazione della memoria o vulnerabilità di sicurezza.

La prassi ingegneristica standard impone di **elencare sempre esplicitamente** gli attributi desiderati dopo la clausola `SELECT`, persino nel caso in cui coincidano con l'intero schema relazionale in quel preciso momento.

5.2.2 Attributi Parziali e Logica Ternaria

Un aspetto teorico e pratico cruciale riguarda la valutazione dei predicati in presenza di attributi parziali (non obbligatori). Si assume per definizione che il valore speciale **NULL** sia sempre incluso nel dominio di qualsiasi attributo, a meno che la sua presenza non sia esplicitamente vietata da un vincolo strutturale (es. `NOT NULL` o `PRIMARY KEY`).

Poiché `NULL` rappresenta l'incognita ("valore sconosciuto" o "inapplicabile"), **qualsiasi tipo di predicato o operazione** (non solo i normali operatori di confronto relazionale come `=`, `>`, `<`, `<>`, ma anche operatori avanzati come `LIKE`, `IN` o `BETWEEN`) fallisce nel restituire un risultato certo. Valutare matematicamente un'incognita all'interno di una condizione non restituisce necessariamente `TRUE` o `FALSE`, ma introduce un terzo stato logico: **UNKNOWN**.

Nelle espressioni booleane complesse, l'algebra relazionale adotta dunque una logica a tre valori (ternaria). Il principio per risolvere le tabelle di verità senza memorizzarle è puramente intuitivo: **il risultato globale è TRUE o FALSE solo se l'incognita non è più determinante per l'esito; altrimenti resta UNKNOWN**.

- `TRUE AND UNKNOWN = UNKNOWN` (L'esito dipende dall'incognita).
- `FALSE AND UNKNOWN = FALSE` (Un solo `FALSE` in un `AND` è sufficiente per falsificare tutto, l'incognita è irrilevante).
- `TRUE OR UNKNOWN = TRUE` (Un `TRUE` in un `OR` è sufficiente per verificare tutto).
- `FALSE OR UNKNOWN = UNKNOWN` (L'esito dipende dall'incognita).

⚠ Criterio di Ammissione della Tupla

Affinché l'operatore di selezione σ includa una tupla nel risultato finale, il predicato complessivo deve valutarsi **strettamente a TRUE**. Qualsiasi tupla che restituisca `FALSE` o `UNKNOWN` viene inesorabilmente scartata.

Operatori per l'identificazione delle incognite Per poter filtrare deliberatamente le tuple in base all'assenza di dati, SQL introduce due operatori dedicati che restituiscono sempre `TRUE` o `FALSE`, bypassando lo stato `UNKNOWN`:

- **IS NULL**: Valuta a TRUE se la cella è priva di valore.
- **IS NOT NULL**: Valuta a TRUE se la cella contiene un valore definito e valido.

Esempio 5.3 (Filtrare tramite operatori dedicati: studenti non lavoratori). Per selezionare gli studenti che non hanno un impiego registrato, è necessario intercettare il valore NULL usando l'operatore specifico, non l'uguaglianza.

$$\sigma_{\text{lavoro IS NULL}}(\text{studente})$$

```
1 SELECT * FROM studente WHERE lavoro IS NULL;
```

matricola	nome	cognome	dataN	citta	lavoro
10001	Mario	Rossi	1999-05-14	Roma	NULL
10003	Giulia	Rossi	2001-02-10	Roma	NULL
10005	Mario	Neri	2000-01-15	Milano	NULL

Esempio 5.4 (Insidia degli operatori standard con il valore NULL). Si voglia estrarre l'elenco degli studenti il cui lavoro è **diverso** da 'Developer'.

$$\sigma_{\text{lavoro} \neq \text{'Developer'}}(\text{studente})$$

```
1 SELECT * FROM studente WHERE lavoro <> 'Developer';
```

È un errore logico comune aspettarsi che gli studenti disoccupati (con lavoro a NULL) vengano inclusi. In realtà, valutare $\text{NULL} \neq \text{'Developer'}$ restituisce UNKNOWN. Di conseguenza, la selezione ammette solo chi ha un lavoro esplicitato diverso da quello cercato, scartando i NULL.

matricola	nome	cognome	dataN	citta	lavoro
10002	Luigi	Verdi	2000-11-22	Milano	Barista

5.3 Composizione di Proiezioni e Selezioni

Nell'algebra relazionale, le interrogazioni complesse si costruiscono componendo gli operatori di base. Poiché il risultato di un'operazione relazionale è sempre a sua volta una relazione (proprietà di chiusura algebrica), gli operatori possono essere annidati e applicati in cascata.

L'espressione più comune consiste nell'estrazione di specifiche colonne da un sottoinsieme filtrato di tuple. Matematicamente, questo si modella applicando prima una selezione e successivamente una proiezione sul risultato:

$$\pi_Y(\sigma_p(r))$$

dove r è la relazione di partenza, p è il predicato logico di selezione e Y è l'insieme degli attributi da proiettare.

A differenza dell'algebra classica o della composizione tra operatori identici (es. due selezioni consecutive commutano sempre), **la proiezione e la selezione non commutano in generale**:

$$\pi_Y(\sigma_p(r)) \neq \sigma_p(\pi_Y(r))$$

Questa disuguaglianza diventa un vero e proprio errore di valutazione se la condizione logica p coinvolge attributi della relazione originale che *non* sono inclusi nell'insieme proiettato Y . Se la proiezione venisse eseguita per prima, scarterebbe prematuramente le colonne necessarie per valutare la condizione della selezione successiva.

Esempio 5.5 (Controesempio sulla Non-Commutatività). Consideriamo l'istanza della relazione **studente** (Tabella 5.1) e supponiamo di voler estrarre solo i **nomi** degli studenti residenti a **Roma**.

1. Composizione Corretta: $\pi_{nome}(\sigma_{citta='Roma'}(studente))$

- L'operatore più interno (σ) filtra le tuple tenendo solo quelle che soddisfano **Citta** = 'Roma'.
- L'operatore esterno (π) riceve la relazione filtrata e scarta tutte le colonne eccetto il **nome**.
- **Risultato:** Una relazione formalmente corretta contenente le tuple "Mario" e "Giulia".

2. Composizione Errata (Commutazione): $\sigma_{citta='Roma'}(\pi_{nome}(studente))$

- L'operatore più interno (π) scarta subito tutte le colonne eccetto **Nome**. Lo schema risultante diviene $R_{tmp}(nome)$.
- L'operatore esterno (σ) tenta di valutare la condizione **citta** = 'Roma' sulle tuple di R_{tmp} , ma l'attributo **citta** **non esiste più** nello schema corrente.
- **Risultato:** L'espressione è **matematicamente indefinita (mal formata)**. Poiché l'attributo **citta** non appartiene allo schema di R_{tmp} , il predicato non può essere logicamente valutato (non restituisce "Falso" generando un insieme vuoto, ma costituisce una violazione del dominio di definizione). In SQL, questa interrogazione non restituisce un *result set* vuoto, ma genera un errore semantico a tempo di compilazione.

5.3.1 La Composizione in SQL e l'Ordine Logico di Esecuzione

Il linguaggio SQL traduce la composizione $\pi_Y(\sigma_p(r))$ avvalendosi simultaneamente delle clausole **SELECT** (per la proiezione) e **WHERE** (per la selezione):

```
1 SELECT nome
2 FROM studente
3 WHERE citta = 'Roma';
```

Per evitare a livello architetturale i paradossi della non-commutatività appena dimostrati, lo standard SQL impone al Query Optimizer un rigoroso **ordine logico di valutazione**, che differisce dall'ordine di scrittura del codice sorgente (dall'alto verso il basso).

Il DBMS analizza l'interrogazione applicando le operazioni esattamente nella sequenza insiemistica sicura:

1. **FROM:** Identifica la relazione (o le relazioni) di partenza.
2. **WHERE:** Applica la condizione di selezione (σ), filtrando orizzontalmente le righe valide.
3. **SELECT:** Applica la proiezione (π) sull'insieme filtrato, scartando verticalmente le colonne escluse.

Di conseguenza, nella clausola **WHERE** è sempre possibile e legittimo fare riferimento ad attributi della tabella che non compaiono nella **SELECT**, poiché la valutazione del filtro avviene *prima* del taglio delle colonne.

5.4 Qualificazione degli Attributi

Finora abbiamo utilizzato una sintassi semplificata per indicare le colonne (es. **SELECT nome**). Tuttavia, la teoria dei database relazionali impone che il riferimento a un attributo debba essere assoluto e privo di ambiguità.

La notazione estesa, o **qualificazione**, si esprime tramite la sintassi (vedremo gli alias nel paragrafo a seguire):

nome_tabella.nome_attributo oppure alias_tabella.nome_attributo

Osservazione 5.2 (Risoluzione delle Ambiguità). Quando un'interrogazione agisce su una singola tabella, il DBMS risolve implicitamente l'appartenenza dell'attributo. Tuttavia, quando si lavora su prodotti cartesiani o *Join* tra più tabelle (es. **studente** e **professore**), è matematicamente frequente che due tabelle possiedano attributi omonimi (es. **nome** e **cognome**).

In questi scenari, omettere la qualificazione genera un errore semantico bloccante (*Ambiguous column name*). Pertanto, è considerata una **best practice ingegneristica e di stile** adottare sempre la sintassi estesa **tabella.attributo** (spesso in sinergia con un alias di tabella) non appena l'interrogazione coinvolge più di una relazione.

5.5 Ridenominazione

La ridenominazione è un operatore **unario** (agisce su una singola relazione) atipico: a differenza di proiezione e selezione, non filtra né estrae dati, ma agisce esclusivamente sull'intensione (lo schema), lasciando inalterata l'estensione (le tuple). In termini pratici, modifica il nome della tabella o dei suoi attributi.

Sia $R(X)$ uno schema di relazione. L'operatore di ridenominazione si denota tipicamente con la lettera greca rho (ρ). Può assumere tre forme:

- **Ridenominazione di attributi:** $\rho_{nuovo \leftarrow vecchio}(r)$ altera il nome di specifiche colonne.
- **Ridenominazione di relazione:** $\rho_s(r)$ assegna il nuovo nome s all'intera relazione.
- **Ridenominazione completa (relazione e attributi):** $\rho_{s(a_1, a_2, \dots, a_n)}(r)$ assegna il nuovo nome s all'intera relazione e, simultaneamente, rinomina **tutti** i suoi attributi in $a_1, a_2, \dots, a_n$. Affinché l'espressione sia matematicamente ben formata, il numero dei nuovi attributi forniti tra parentesi deve corrispondere esattamente al grado (numero di colonne) della relazione di partenza. L'assegnazione avviene per rigoroso ordine posizionale (da sinistra verso destra).

Lo schema della relazione prodotta possiede lo stesso grado (numero di attributi) e gli stessi identici domini della relazione di partenza, ma identificatori testuali differenti; anche il numero e il contenuto delle tuple rimane del tutto invariato.

Sebbene ad una prima occhiata non sembri un operatore di grande utilità, vedremo presto che è fondamentale per la risoluzione di ambiguità.

5.5.1 La Ridenominazione in SQL: L'operatore AS

Il linguaggio SQL implementa l'operatore algebrico ρ attraverso la parola chiave **AS** (spesso definita *Alias*).

Coerentemente con la natura duale dell'operatore matematico (che agisce su attributi o su relazioni), anche in SQL l'alias può essere impiegato in due contesti architetturali distinti:

Ridenominazione degli Attributi (nella clausola SELECT) Applicato all'elenco delle colonne, l'operatore modifica l'intestazione della tabella virtuale (Result Set) prodotta in output. Risulta indispensabile quando si estraggono dati derivati da espressioni matematiche o funzioni, che altrimenti verrebbero restituiti dal motore del database privi di un nome di colonna definito (o con nomi generati automaticamente come ?column?).

Sintassi per attributi:

```
1 SELECT matricola AS id_studente, cognome
2 FROM studente;
```

Ridenominazione delle Relazioni (nella clausola FROM) Applicato alle tabelle di origine, assegna un nome temporaneo all'intera relazione per la durata dell'interrogazione. Questo meccanismo, unito alla notazione puntata, è l'unico modo per risolvere le ambiguità quando si interrogano simultaneamente più tabelle che presentano attributi omonimi, o quando si incrocia una tabella con sé stessa.

Sintassi per relazioni (con notazione puntata):

```
1 SELECT s.nome, s.cognome
2 FROM studente AS s
3 WHERE s.città = 'Roma';
```

5.6 Operazioni Insiemistiche

Essendo il modello relazionale fondato sulla teoria degli insiemi, è possibile combinare due relazioni utilizzando i classici operatori insiemistici: **Unione** ($\cup$), **Intersezione** ($\cap$) e **Differenza** ($-$).

Tuttavia, diversamente dalla matematica pura dove è possibile unire insiemi eterogenei (es. un insieme di mele con un insieme di numeri), nell'algebra relazionale le tuple elaborate devono avere una struttura coerente.

⚠ Condizione di Compatibilità degli Schemi

Le operazioni insiemistiche tra due relazioni r_1 e r_2 sono definite **se e solo se** i loro schemi sono compatibili. Due schemi sono compatibili quando:

1. Hanno lo stesso numero di attributi (stesso grado).
2. Gli attributi corrispondenti (letti da sinistra verso destra) appartengono allo stesso dominio semantico.

Se questa condizione è rispettata, lo schema della relazione risultante coinciderà tipicamente con lo schema del primo operando (r_1).

Date due relazioni r_1 e r_2 compatibili:

- **Unione** ($r_1 \cup r_2$): Costruisce una relazione contenente tutte le tuple che appartengono a r_1 , a r_2 , o a entrambe. Essendo un insieme, le tuple presenti in entrambe le relazioni compaiono una sola volta nel risultato.
- **Intersezione** ($r_1 \cap r_2$): Costruisce una relazione contenente esclusivamente le tuple che appartengono contemporaneamente sia a r_1 che a r_2 .

- **Differenza** ($r_1 - r_2$): Costruisce una relazione contenente le tuple che appartengono a r_1 ma **non** appartengono a r_2 .

Per comprendere l'importanza di questi operatori, consideriamo il seguente schema relazionale che modella gli incontri di un campionato di calcio:

$$PARTITE(data, stadio, squadra1, squadra2, goals1, goals2)$$

Nota: Convenzionalmente, **squadra1** gioca in casa e **goals1** rappresenta le sue reti.

Esempio 5.6 (L'Unione: squadre vincenti allo stadio Maradona). Vogliamo ricavare l'elenco di tutte le squadre che hanno vinto almeno una partita allo stadio 'Maradona'.

Il problema logico risiede nel fatto che una squadra può aver vinto giocando in casa (**squadra1**) oppure in trasferta (**squadra2**). Non potendo estrarre due colonne diverse contemporaneamente in una singola proiezione lineare, l'unione risulta essenziale:

1. Ricaviamo chi ha vinto in casa:

$$v_{casa} = \pi_{squadra1}(\sigma_{stadio='Maradona' \wedge goals1 > goals2}(partite))$$

2. Ricaviamo chi ha vinto in trasferta:

$$v_{trasferta} = \pi_{squadra2}(\sigma_{stadio='Maradona' \wedge goals2 > goals1}(partite))$$

3. Uniamo i risultati (gli schemi sono compatibili, essendo entrambi domini di squadre):

$$risultato = v_{casa} \cup v_{trasferta}$$

Esempio 5.7 (La Differenza e la logica del "Mai"). Vogliamo ricavare l'elenco delle squadre che **non hanno mai vinto** allo stadio 'Maradona'.

Questa interrogazione non può essere risolta con una semplice selezione negata. Questo limite logico rende l'operatore di differenza insostituibile ogni volta che un requisito contiene parole come "mai" o "nessuno". L'approccio corretto è sempre per sottrazione: **Tutti - Chi ha l'attributo negato**.

1. Troviamo **tutte** le squadre del campionato:

$$tutte = \pi_{squadra1}(partite) \cup \pi_{squadra2}(partite)$$

2. Troviamo le squadre che **hanno vinto almeno una volta** al Maradona (espressione *risultato* dell'esempio precedente, chiamiamola *vincitriciM*). 3. Applichiamo la differenza:

$$maiVintoM = tutte - vincitriciM$$

Esempio 5.8 (Trappola Logica: Squadre che non hanno mai vinto in casa). Un errore classicamente commesso in fase di modellazione è tentare di risolvere i problemi di negazione universale ("mai") utilizzando i normali operatori di confronto.

Soluzione Sbagliata: Si potrebbe essere tentati di scrivere:

$$errata = \pi_{squadra1}(\sigma_{goals1 \leq goals2}(partite))$$

Perché è sbagliata? Questa espressione restituisce le squadre che hanno perso o pareggiato *almeno una partita* in casa. Tuttavia, se una squadra ha perso una partita in casa (entrando così nel risultato) ma ne ha vinta un'altra, il requisito "non ha mai vinto" viene violato.

Soluzione Corretta (tramite Differenza): 1. Troviamo tutte le squadre che hanno giocato in casa: $t_{casa} = \pi_{squadra1}(partite)$ 2. Troviamo le squadre che hanno vinto almeno una volta in casa: $v_{casa} = \pi_{squadra1}(\sigma_{goals1 > goals2}(partite))$ 3. Sottraiamo:

$$risultato = t_{casa} - v_{casa}$$

5.6.1 Le Operazioni Insiemistiche in SQL

Il linguaggio SQL implementa le operazioni insiemistiche dell'algebra relazionale attraverso i costrutti **UNION** (Unione), **INTERSECT** (Intersezione) ed **EXCEPT** (Differenza). Alcuni dialetti commerciali storici (come Oracle) utilizzano la keyword **MINUS** in luogo di **EXCEPT**.

Affinché l'interrogazione possa essere compilata ed eseguita dal DBMS senza errori semantici, i due blocchi **SELECT** coinvolti devono rigorosamente rispettare la condizione di compatibilità degli schemi vista in precedenza: stesso numero di colonne estratte e tipi di dato compatibili, valutati in rigoroso ordine posizionale.

Sintassi generale delle operazioni insiemistiche:

```

1 SELECT <elenco_attributi_A> FROM <tabella_A>
2 [ UNION | INTERSECT | EXCEPT ]
3 SELECT <elenco_attributi_B> FROM <tabella_B>;

```

Nei paragrafi precedenti abbiamo dimostrato come il comando **SELECT** base non applichi la de-duplicazione, comportandosi di default come un multinsieme (**ALL**). Con gli operatori insiemistici, lo standard SQL capovolge questa logica:

- **Comportamento di default (Puro Insieme):** L'uso di **UNION**, **INTERSECT** o **EXCEPT** forza implicitamente il motore del database a comportarsi secondo la matematica pura. Il DBMS ordinerà i risultati intermedi ed **eliminerà tutti i duplicati** (**DISTINCT** implicito).
- **Comportamento Multinsieme (ALL):** Qualora il programmatore desideri mantenere i duplicati (comportamento non relazionale ma spesso necessario a fini statistici o per risparmiare il gravoso costo computazionale dell'ordinamento), è necessario esplicitare la parola chiave **ALL** subito dopo l'operatore (es. **UNION ALL**, **EXCEPT ALL**).

Esempio 5.9 (Traduzione SQL: Squadre vincenti allo stadio Maradona). L'espressione algebrica $v_{casa} \cup v_{trasferta}$ vista nel primo esempio si traduce strutturando due query indipendenti collegate dall'operatore **UNION**:

```

1 (SELECT squadra1 AS squadra_vincente
2  FROM partite
3  WHERE stadio = 'Maradona' AND goals1 > goals2)
4 UNION
5 (SELECT squadra2
6  FROM partite
7  WHERE stadio = 'Maradona' AND goals2 > goals1;)

```

Nota Architetture: Lo schema della tabella virtuale risultante eredita sempre i nomi delle colonne dal **primo** blocco **SELECT** (valutato top-down). Pertanto, un'eventuale ridenominazione tramite alias (**AS squadra_vincente**) ha effetto solo se applicata nella query superiore; eventuali alias nel blocco inferiore verrebbero ignorati dal motore SQL.

Quando si concatenano più blocchi **SELECT** tramite operazioni insiemistiche, lo standard SQL impone una gerarchia matematica rigida: l'operatore **INTERSECT** ha la priorità massima, mentre **UNION** ed **EXCEPT** vengono valutati successivamente, rigorosamente da sinistra verso destra. Di conseguenza, un'espressione come $A \cup B \cap C$ viene implicitamente calcolata dal DBMS come $A \cup (B \cap C)$.

Per ragioni di leggibilità e manutenibilità, la prassi impone di utilizzare sempre le **parentesi tonde** per esplicitare i blocchi logici, sovrascrivendo l'ordine di default ove

necessario (es. $(A \cup B) \cap C$). Nell'esempio di sopra le parentesi erano ridondanti, ma sono state comunque inserite per maggiore leggibilità.

5.7 Il Prodotto Cartesiano e le Giunzioni (Join)

Fino a questo momento abbiamo interrogato le relazioni in modo isolato (operazioni unarie) o abbiamo unito tabelle con struttura identica (operazioni insiemistiche). Tuttavia, la vera potenza del modello relazionale consiste nella capacità di combinare informazioni di natura diversa incrociando relazioni eterogenee. L'operatore primitivo che permette questa combinazione orizzontale è il prodotto cartesiano.

Il Prodotto Cartesiano Siano $r_1 \in R_1(X_1)$ e $r_2 \in R_2(X_2)$ due istanze di relazione. Il prodotto cartesiano $r_1 \times r_2$ è un operatore binario che concatena ogni singola tupla della prima relazione con ogni singola tupla della seconda.

- **Schema risultante (Grado):** Lo schema della nuova relazione è costituito dall'unione degli insiemi di attributi ($X_1 \cup X_2$). Se la prima relazione ha N_1 attributi e la seconda ne ha N_2 , il grado risultante è $N_1 + N_2$.
- **Impatto sulla cardinalità:** Il numero totale di tuple generate è il prodotto matematico delle cardinalità di partenza: $n_1 \cdot n_2$.

Osservazione 5.3 (L'esplosione combinatoria e l'assenza di semantica). Dal punto di vista ingegneristico, il prodotto cartesiano genera una rapida esplosione dimensionale. Se incrociamo una tabella di 1.000 studenti con una di 100 corsi, generiamo istantaneamente 100.000 tuple. Ancora più grave è il problema semantico: il prodotto cartesiano puro concatena tuple in modo incondizionato, associando (ad esempio) ogni studente a **tutti** gli esami esistenti, compresi quelli che non ha mai sostenuto. Genera dunque un'enorme quantità di informazioni prive di significato logico. Per questo motivo, l'operatore libero ($\times$) non viene quasi mai utilizzato direttamente, ma funge da base matematica per definire l'operatore di giunzione.

L'Operatore di Giunzione (Theta-Join) Per estrarre solo le tuple "coerenti" dal prodotto cartesiano, è necessario applicare una selezione che filtri le righe basandosi su un legame logico tra le due tabelle. Poiché questa combinazione è ubiquitaria nei database, l'algebra relazionale la fonde in un unico operatore derivato: la **Giunzione** (o Join), denotata dal simbolo a farfalla ($\bowtie$).

Per definizione, una giunzione condizionale (o Theta-Join) rispetto a un predicato p equivale a eseguire una selezione su un prodotto cartesiano:

$$r_1 \bowtie_p r_2 \equiv \sigma_p(r_1 \times r_2)$$

La condizione di giunzione è **quasi sempre un'uguaglianza** (Equi-Join) che lega la Primary Key (PK) di una tabella alla corrispondente Foreign Key (FK) dell'altra.

Esempio 5.10 (Equi-Join: Associazione corretta tra Studenti ed Esami). Consideriamo gli schemi $STUDENTE(matricola, nome, cognome)$ ed $ESAME(matr, corso, voto)$, dove $matr$ in $ESAME$ è chiave esterna verso $STUDENTE$. Per visualizzare l'elenco corretto degli esami sostenuti da ciascuno studente, applichiamo un Equi-Join sulla condizione di legame:

$$studente \bowtie_{matricola=matr} esame$$

Il risultato restituirà solo le tuple in cui il codice dello studente coincide, eliminando le migliaia di combinazioni errate generate dal prodotto cartesiano sottostante.

La Giunzione Naturale (Natural Join) La giunzione naturale è una variante del Join in cui **la condizione viene omessa** dal pedice dell'operatore. Il sistema applica automaticamente una condizione implicita e altera lo schema risultante in modo peculiare.

- **Condizione implicita:** Le due relazioni vengono giunte uguagliando i valori di **tutti gli attributi con lo stesso nome** (e stesso dominio) presenti in entrambi gli schemi. In termini insiemistici, la giunzione naturale opera un'intersezione sui valori delle colonne in comune.
- **Fusione delle colonne:** A differenza del Theta-Join (che duplica le colonne messe a confronto), la giunzione naturale **fonde gli attributi in comune** in un'unica colonna, evitando ridondanze nello schema risultante.
- **Assenza di attributi in comune:** Se le due tabelle non hanno alcun attributo omonimo, l'operatore non potendo applicare alcuna condizione "degenera" in un prodotto cartesiano incondizionato ($r_1 \bowtie r_2 \equiv r_1 \times r_2$).

Esempio 5.11 (Giunzione Naturale tramite Ridenominazione). Nell'esempio precedente, non possiamo eseguire un Natural Join diretto tra *studente* ed *esame* poiché l'attributo identificativo ha nomi diversi (*matricola* e *matr*). Possiamo però forzare la natura dell'operatore applicando prima una ridenominazione sull'istanza *esame*:

$$studente \bowtie \rho_{esame_mod(matricola, corso, voto)}(esame)$$

Essendo comparso un attributo omonimo (*matricola*), l'operatore eseguirà l'uguaglianza implicita e restituirà uno schema compatto: $R(matricola, nome, cognome, corso, voto)$.

Self-Join e l'uso strategico della Ridenominazione L'operazione di ridenominazione completa (ρ) si rivela indispensabile quando occorre confrontare tuple appartenenti alla **stessa** relazione. In algebra relazionale non è possibile eseguire $r \bowtie r$ senza creare un'insanabile ambiguità sugli attributi. È necessario generare una copia "virtuale" della relazione con un'intestazione diversa.

Esempio 5.12 (Cambiamenti di città). Siano dati gli schemi (dove PK è il primo attributo o la prima coppia):

- $PERSONA(cf, nome, cognome, dataN, dataM)$
- $RESIDENZA(cf, dataInizio, dataFine, città, via)$

Interrogazione: Estrarre Codice Fiscale, Nome e Cognome delle persone che hanno cambiato città di residenza nel corso del tempo.

Per risolvere l'interrogazione, dobbiamo cercare due tuple di residenza che abbiano lo stesso *cf* ma attributo *città* differente.

1. **Creazione della copia:** Rinominiamo la tabella residenza per avere due fattori indipendenti:

$$r_{old} = \rho_{r_old(cf', dI', dF', città', via')}(residenza)$$

2. **Self-Join e Filtraggio:** Uniamo l'istanza originale con la copia imponendo che la persona sia la stessa ($cf = cf'$), ed estraiamo con una selezione chi ha cambiato città:

$$cambi = \pi_{cf}(\sigma_{città \neq città'}(residenza \bowtie_{cf=cf'} r_{old}))$$

3. **Recupero anagrafica (Natural Join):** L'espressione *cambi* contiene solo il *cf*. Per ottenere Nome e Cognome, eseguiamo una giunzione naturale con l'anagrafica:

$$risultato = \pi_{cf, nome, cognome}(cambi \bowtie persona)$$

5.7.1 I Join in SQL

Lo standard SQL moderno (ANSI-92) implementa la giunzione separandola nettamente dalle normali condizioni di filtraggio (clausola `WHERE`), dedicando costrutti specifici direttamente nella clausola `FROM`.

Sintassi generale (Equi-Join):

```
1 SELECT <elenco_attributi>
2 FROM <tabella_1>
3 [INNER] JOIN <tabella_2> ON <condizione_di_giunzione>;
```

Q La parola chiave INNER

Scrivere `JOIN` oppure `INNER JOIN` ordina al DBMS di eseguire **esattamente la stessa operazione** (il Theta-Join o Equi-Join dell'algebra relazionale).

Il termine `INNER` (interno) ribadisce soltanto la natura restrittiva dell'operatore: le tuple che non soddisfano la condizione di giunzione vengono scartate. Poiché questo è il comportamento di default storico di SQL, la parola può essere omessa, ma viene spesso esplicitata nelle basi di codice di produzione per distinguere nettamente l'operazione dalle giunzioni esterne (`OUTER JOIN`).

La traduzione del nostro esempio originario ($studente \bowtie_{matricola=matr} esame$) si realizza applicando la notazione puntata per esplicitare la condizione tra Primary Key e Foreign Key:

```
1 SELECT s.matricola, s.nome, s.cognome, e.corso, e.voto
2 FROM studente AS s
3 JOIN esame AS e ON s.matricola = e.matr;
```

✓ Evitare la Giunzione Naturale in SQL

Sebbene la giunzione naturale ($\bowtie$) sia un operatore teorico fondamentale per semplificare le espressioni in algebra relazionale, la sua traduzione diretta in SQL (tramite la parola chiave `NATURAL JOIN`) è universalmente considerata un **anti-pattern ingegneristico** e non viene quasi mai utilizzata nel codice di produzione.

La regola d'oro nello sviluppo del software impone che **l'esplicito sia sempre meglio dell'implicito**. Pertanto, si utilizza pressoché esclusivamente la giunzione con condizione esplicita (`JOIN ... ON ...`) per tre ragioni architetturali critiche:

1. **Prevenzione delle evoluzioni occulte (Schema Evolution):** Se in futuro un amministratore aggiungesse a entrambe le tabelle una colonna di servizio con lo stesso nome (es. `data_modifica` o `stato`), un `NATURAL JOIN` la ingloberebbe silenziosamente nella condizione di uguaglianza. La query inizierebbe a fallire o a omettere risultati senza mai generare un errore di compilazione.
2. **Prevenzione delle collisioni di naming:** Spesso chiavi primarie di entità diverse condividono lo stesso identificatore generico (es. `id`). La giunzione naturale le uguaglierebbe in automatico, estraendo dati matematicamente associati ma semanticamente spazzatura.



3. Leggibilità del codice (Manutenibilità): Esplicitare la clausola **ON** permette a chiunque legga la query di comprendere istantaneamente il legame strutturale tra le tabelle, senza essere costretto a interrogare la struttura fisica del database per capire quali colonne omonime il motore SQL abbia deciso di fondere.

Per questi motivi, la sintassi e l'uso del **NATURAL JOIN** vengono qui deliberatamente omessi a favore dell'approccio rigoroso basato sulle condizioni esplicite. Un paragrafo al riguardo è comunque presente in appendice per completezza.

5.7.2 Il Prodotto Cartesiano in SQL: **CROSS JOIN**

Sebbene il prodotto cartesiano libero ($\times$) sia un'operazione quasi assente nella logica applicativa quotidiana a causa dell'esplosione combinatoria e della carenza di semantica, lo standard SQL fornisce un costrutto esplicito per realizzarlo: **CROSS JOIN**.

A differenza degli altri tipi di giunzione, il **CROSS JOIN** non ammette la clausola **ON**, poiché per definizione matematica non vi è alcuna condizione logica da valutare: ogni singola tupla della prima tabella viene incondizionatamente concatenata con ogni tupla della seconda.

Sintassi esplicita standard (ANSI-92):

```
1 SELECT s.matricola, s.nome, e.corso
2 FROM studente AS s
3 CROSS JOIN esame AS e;
```

La sintassi implicita (Legacy) Prima che lo standard ANSI-92 introducesse le keyword esplicite per i **JOIN**, il prodotto cartesiano veniva generato semplicemente elencando le tabelle all'interno della clausola **FROM**, separandole con una virgola.

Sintassi implicita (ANSI-89):

```
1 SELECT s.matricola, s.nome, e.corso
2 FROM studente AS s, esame AS e;
```



La genesi dei bug silenti

La sintassi implicita con la virgola è ancora oggi pienamente supportata da tutti i DBMS commerciali per ragioni di retrocompatibilità. Tuttavia, rappresenta il vettore principale per i disastri prestazionali.

Se un progettista intende eseguire una normale giunzione interna usando la sintassi legacy, ma dimentica accidentalmente di inserire la condizione di uguaglianza nella clausola **WHERE**, il motore del database non genererà alcun errore di sintassi. Ripiegherà invece in modo totalmente silente sull'esecuzione di un prodotto cartesiano puro, saturando la memoria del server. L'impiego del costrutto esplicito **CROSS JOIN**, al contrario, rende inequivocabile l'intento matematico dello sviluppatore, rendendo il codice sicuro e auto-documentato.

5.8 Giunzioni Esterne (Outer Join)

L'operatore di giunzione interna (Inner Join) è intrinsecamente "distruttivo": se una tupla della prima relazione non trova alcuna corrispondenza nella seconda relazione rispetto alla condizione imposta, viene inesorabilmente scartata dal risultato finale.

In molti scenari informativi, tuttavia, è essenziale preservare traccia delle entità "orfane" (es. gli studenti che non hanno ancora sostenuto alcun esame, o i clienti che non hanno effettuato ordini). Per rispondere a questa esigenza, l'algebra relazionale introduce le **Giunzioni Esterne** (Outer Join).

Le giunzioni esterne estendono il risultato di una normale giunzione recuperando le tuple scartate e "riempiendo" (padding) gli attributi mancanti della relazione opposta con il marcatore di incognita NULL.

Esistono tre varianti di questo operatore (anche per esse vale il divieto architetturale di omettere la condizione a favore della variante "naturale"):

- **Left Outer Join** ($r_1 \bowtie\!\!\!\lrcorner r_2$): Preserva **tutte** le tuple della relazione di sinistra (r_1). Se una tupla di r_1 non ha corrispondenze in r_2 , viene inclusa nel risultato assegnando NULL a tutti gli attributi provenienti da r_2 .
- **Right Outer Join** ($r_1 \bowtie\!\!\!\rceil r_2$): Preserva **tutte** le tuple della relazione di destra (r_2), inserendo NULL negli attributi di r_1 per le tuple orfane.
- **Full Outer Join** ($r_1 \bowtie\!\!\!\bowtie r_2$): È l'unione dei due comportamenti. Preserva integralmente entrambe le relazioni, inserendo NULL a destra o a sinistra ove manchi la corrispondenza.

Esempio 5.13 (Left Join: Studenti con e senza esami). Siano dati gli schemi:

$STUDENTE(matricola, nome), ESAME(matr, corso, voto),$

con le seguenti piccole istanze:

- *studente*: {(101, 'Mario'), (102, 'Luigi')}
- *esame*: {(101, 'Basi di Dati', 30)}

Eseguito una giunzione esterna sinistra:

$studente \bowtie\!\!\!\lrcorner_{matricola=matr} esame$

La tupla di Mario trova corrispondenza e si fonde regolarmente. La tupla di Luigi non trova corrispondenza in *esame*, ma viene "salvata" dall'operatore *Left*, generando la riga: (102, 'Luigi', NULL, NULL, NULL).

5.8.1 Le Giunzioni Esterne in SQL

Il linguaggio SQL implementa questi operatori estendendo la sintassi della clausola JOIN. Esattamente come per INNER, la parola chiave OUTER è considerata ridondante e facoltativa dallo standard (es. scrivere LEFT JOIN equivale a tutti gli effetti a scrivere LEFT OUTER JOIN).

Sintassi generale (Left Join):

```

1 SELECT s.matricola, s.nome, e.corso, e.voto
2 FROM studente AS s
3 LEFT JOIN esame AS e ON s.matricola = e.matr;
```

✓ Realtà di Produzione: Il tramonto del RIGHT JOIN

Sebbene il `RIGHT JOIN` esista e sia formalmente corretto, nel codice di produzione moderno è universalmente evitato. Per convenzione di leggibilità *top-down* (dall'alto verso il basso), gli ingegneri preferiscono posizionare la tabella principale nella clausola `FROM` ed eseguire `LEFT JOIN` a cascata, piuttosto che invertire mentalmente l'ordine di lettura usando giunzioni destre. Un `RIGHT JOIN` può (e deve) sempre essere convertito in un `LEFT JOIN` semplicemente scambiando l'ordine delle tabelle nel codice.

Osservazione 5.4 (La natura della `FULL JOIN`). A causa della scarsa utilità pratica di estrarre orfani bidirezionali in modelli dati strutturati, la `FULL JOIN` è così rara che molti DBMS commerciali (come MySQL o SQLite) non la implementano nativamente. Per ottenerla, si ricorre all'algebra insiemistica: l'unione (`UNION`) tra una giunzione sinistra (`A LEFT JOIN B`) e la sua inversa (`B LEFT JOIN A`).

5.8.2 Il pattern "Anti-Join": Isolare le assenze

L'utilità strategica più importante delle giunzioni esterne non risiede solo nel "mostrare tutto", ma nel fornire uno strumento per **isolare matematicamente le anomalie o le mancanze**.

Per rispondere all'interrogazione "*Trovare gli studenti che **non** hanno sostenuto alcun esame*", l'approccio più robusto consiste nel combinare un Outer Join con la gestione della logica ternaria dei valori `NULL` studiata nei capitoli precedenti.

1. Si esegue un `LEFT JOIN` per estrarre tutti gli studenti, portando a bordo i `NULL` per chi non ha esami. 2. Si applica una selezione (`WHERE`) filtrando deliberatamente **solo** le righe in cui la Primary Key della tabella agganciata risulta `NULL`.

Questo costrutto ingegneristico è noto come **Anti-Join**. È enormemente più efficiente rispetto all'uso di operazioni insiemistiche (`EXCEPT`) perché evita al motore del database di dover eseguire onerosi ordinamenti per la de-duplicazione.

```

1  -- Estrae solo gli studenti SENZA esami
2  SELECT s.matricola, s.nome
3  FROM studente AS s
4  LEFT JOIN esame AS e ON s.matricola = e.matr
5  WHERE e.matr IS NULL;
```

5.9 Raggruppamento e Funzioni di Aggregazione

L'algebra relazionale di base (selezione, proiezione, join) opera sulle singole tuple in modo isolato. Tuttavia, per rispondere a interrogazioni di natura statistica o riassuntiva (es. "Qual è la media dei voti?", "Quanti esami ha superato ogni studente?"), è necessario estendere l'algebra introducendo l'operatore di **Raggruppamento e Aggregazione**, denotato tipicamente con la lettera calligrafica $\mathcal{F}$ (oppure γ).

Il processo logico si divide in due fasi:

1. **Partizionamento:** L'istanza della relazione viene suddivisa in sottoinsiemi (partizioni) in base a un *criterio di raggruppamento*. Due tuple appartengono alla stessa partizione se e solo se presentano esattamente la stessa combinazione di valori sugli attributi scelti.

2. **Conteggio/Calcolo:** Su ogni partizione generata vengono applicate una o più funzioni matematiche che "collassano" l'insieme di tuple in un singolo valore scalare. Le cinque funzioni di aggregazione standard sono:

- **COUNT:** Calcola il numero di tuple (o di valori non nulli) nella partizione.
- **SUM:** Calcola la somma dei valori di un attributo numerico.
- **AVG:** Calcola la media aritmetica dei valori di un attributo numerico.
- **MIN / MAX:** Identifica rispettivamente il valore minimo e massimo.

Definizione e Sintassi in Algebra Relazionale Sia r un'istanza di relazione. La sintassi formale dell'operatore è:

$$A_1, \dots, A_n \mathcal{F}_{f_1(B_1), \dots, f_m(B_m)}(r)$$

Dove:

- L'elenco a sinistra a pedice $(A_1, \dots, A_n)$ rappresenta gli **attributi di raggruppamento**.
- L'elenco a destra a pedice $(f_1(B_1), \dots, f_m(B_m))$ rappresenta le **funzioni di aggregazione** da applicare.

Struttura risultante: La relazione prodotta in output avrà esattamente $n + m$ colonne (gli attributi di raggruppamento seguiti dai risultati delle funzioni) e un numero di righe esattamente pari al numero di partizioni individuate.

Esempio 5.14 (Raggruppamento su singolo attributo: Numero di esami per studente). Consideriamo l'istanza r basata sullo schema

$$R(\text{matricola}, \text{nome}, \text{cognome}, \text{corso}, \text{voto}, \text{lode})$$

prodotta in precedenza da un Left Outer Join tra studenti ed esami. La tupla di "Luigi" presenta valori NULL negli attributi degli esami.

matricola	nome	cognome	corso	voto	lode
101	Mario	Rossi	Basi di Dati	30	false
101	Mario	Rossi	Reti	28	false
102	Luigi	Verdi	NULL	NULL	NULL

Vogliamo calcolare il numero di esami sostenuti da ciascuno studente:

$$\text{matricola} \mathcal{F}_{\text{COUNT}(\text{corso})}(r)$$

matricola	COUNT(corso)
101	2
102	0

Tabella 5.4: La funzione COUNT applicata all'attributo "corso" ignora matematicamente i valori NULL, restituendo correttamente 0 per lo studente senza esami.

Esempio 5.15 (Raggruppamento vuoto: Il voto più alto in assoluto). Se la lista degli attributi di raggruppamento a sinistra è **vuota**, l'intera relazione viene considerata come un'unica partizione.

$$\mathcal{F}_{\text{MAX}(\text{voto})}(r)$$

MAX(voto)
30

Tabella 5.5: Attenzione teorica: sebbene il risultato sembri un semplice numero (30), per la chiusura dell'algebra relazionale esso è a tutti gli effetti un **insieme** (una relazione composta da 1 colonna e 1 riga), e come tale non può essere confrontato direttamente con uno scalare in un predicato logico senza opportune conversioni.

Esempio 5.16 (Raggruppamento multiplo e Ridenominazione). Sia dato lo schema $ESAME(matricola, corso, voto, cfu, anno, lode)$. Vogliamo calcolare, **per ogni studente e per ogni anno accademico**, il numero di esami superati e il totale dei crediti acquisiti, rinominando le colonne calcolate per futuri utilizzi algebrici.

$$\rho_{stat}(matricola, anno, n_esami, tot_cfu)(matricola, anno \mathcal{F}_{COUNT(corso), SUM(cfu)}(esame))$$

matricola	anno	n_esami	tot_cfu
101	2023	2	15
101	2024	1	9
102	2023	1	6

Tabella 5.6: L'elenco di raggruppamento $(matricola, anno)$ genera partizioni basate sulla coppia. Il partizionamento spezza il percorso accademico dello studente 101 su due righe distinte, sommandone i crediti separatamente per anno.

5.9.1 Le Funzioni di Aggregazione in SQL: GROUP BY

In SQL, il concetto algebrico di partizionamento viene implementato rigorosamente attraverso la clausola **GROUP BY**. Le funzioni di aggregazione (COUNT, SUM, AVG, MIN, MAX) vengono dichiarate direttamente nel blocco **SELECT**.

Sintassi generale (traduzione del terzo esempio):

```

1 SELECT matricola, anno, COUNT(corso) AS n_esami, SUM(cfu) AS
   tot_cfu
2 FROM esame
3 GROUP BY matricola, anno;
```



La Regola del GROUP BY

Per prassi logica gli attributi in **SELECT** sono quasi sempre gli stessi che compaiono



in **GROUP BY**. Questo potrebbe non essere sempre vero in alcuni casi particolari, ma in ogni caso bisogna sempre rispettare una precisa regola sintattica in SQL: **ogni attributo nella SELECT non incluso in una funzione di aggregazione deve essere presente nel GROUP BY**

Il motivo di questa regola sarà chiaro quando esamineremo l'ordine di esecuzione delle istruzioni in SQL.

5.9.2 Filtrare i Gruppi: La clausola HAVING

Come abbiamo visto, la clausola **WHERE** agisce sulle singole righe di una relazione, scartando le tuple che non soddisfano un determinato predicato logico. Tuttavia, la sintassi SQL impedisce categoricamente l'uso di funzioni di aggregazione all'interno del **WHERE**, poiché la selezione avviene a livello di singola riga, quando i calcoli statistici non sono ancora stati eseguiti.

Per risolvere l'esigenza di **filtrare interi gruppi** (anziché singole tuple), lo standard ha introdotto la clausola **HAVING**. Questa direttiva viene valutata dal motore del database *dopo* l'avvenuta creazione delle partizioni tramite **GROUP BY**.

La condizione imposta dall'**HAVING** può basarsi esclusivamente su due tipi di espressioni:

1. **Valori di conteggio o aggregazione:** Il risultato di funzioni come **COUNT()**, **SUM()**, **AVG()** applicate alle tuple della singola partizione.
2. **Valori di raggruppamento:** Gli attributi esplicitamente dichiarati nella clausola **GROUP BY** (sebbene filtrare su questi attributi risulti computazionalmente molto più efficiente se effettuato a monte, all'interno del **WHERE**).

Esempio 5.17 (**HAVING** in combinazione con **WHERE**). Supponiamo di voler estrarre la matricola degli studenti che hanno sostenuto esami nell'anno solare 2023, ma **solo** se la media dei voti da loro ottenuta in quell'anno è strettamente superiore a 25.

```

1 SELECT matr, AVG(voto) AS media_voti
2 FROM esame
3 WHERE anno = 2023
4 GROUP BY matr
5 HAVING AVG(voto) > 25;
```

Osservazione 5.5 (Visibilità e Scope delle Funzioni di Aggregazione). Esiste un'importante distinzione architetturale tra il *calcolare* una statistica di gruppo e il *visualizzarla* nel risultato finale. Lo standard SQL non obbliga in alcun modo a inserire una funzione di aggregazione (come **COUNT** o **AVG**) all'interno della clausola **SELECT**.

Se l'obiettivo è utilizzare il calcolo statistico esclusivamente per prendere decisioni logiche o per l'ordinamento, la funzione può risiedere unicamente:

- Nell'**HAVING**: per filtrare intere partizioni (es. scartare i corsi con **COUNT(*) < 5**).
- Nell'**ORDER BY** (che vedremo a breve): per ordinare il *Result Set* finale (es. elencare i dipartimenti in base al **SUM(budget)** decrescente).

In questi casi, il calcolo avviene "dietro le quinte" nella memoria del server e il valore numerico non verrà estratto o trasmesso all'applicativo chiamante.

Esempio 5.18 (Aggregazione "silenziosa" in **HAVING** e **ORDER BY**). Vogliamo ottenere unicamente i nomi dei corsi la cui media voti è superiore a 25. Vogliamo inoltre che la lista sia ordinata in base al numero complessivo di esami registrati per quel corso (dal più

frequentato al meno frequentato). Né la media matematica né il conteggio totale verranno estratti o visualizzati.

```
1 SELECT corso
2 FROM esame
3 GROUP BY corso
4 HAVING AVG(voto) > 25
5 ORDER BY COUNT(*) DESC;
```

5.10 L'Ordine Logico di Esecuzione in SQL

Una delle maggiori insidie del linguaggio SQL risiede nella discrepanza tra l'ordine in cui il codice viene scritto (imposto dalla grammatica del linguaggio) e l'ordine in cui il motore del database (DBMS) esegue effettivamente le direttive.

Nonostante un'interrogazione si apra sempre con la clausola **SELECT**, essa è di fatto l'ultima operazione logica ad essere elaborata. L'ordine reale di valutazione di una query aggregata è il seguente:

1. **FROM (e relativi JOIN)**: Il motore individua le tabelle sorgente e calcola le giunzioni o i prodotti cartesiani necessari per generare la "tabella di lavoro" di base.
2. **WHERE**: Le singole righe della tabella di lavoro vengono filtrate secondo i predicati logici. Le tuple che non soddisfano la condizione vengono fisicamente scartate.
3. **GROUP BY**: Le righe superstiti vengono partizionate in gruppi sulla base degli attributi specificati.
4. **HAVING**: Il filtro viene applicato ai gruppi appena creati. Le intere partizioni che non soddisfano la condizione vengono eliminate.
5. **SELECT**: Solo in questa fase finale vengono eseguite le proiezioni (estrazione delle colonne richieste), valutate le funzioni di aggregazione e applicate eventuali ridenominazioni (**AS**).

Il limite di visibilità tra Tuple e Gruppi

L'ordine di esecuzione spiega perché non si possono mescolare i campi d'azione dei filtri:

- Non posso inserire una condizione statistica (es. **COUNT**) nella clausola **WHERE**, perché in quella fase, cronologicamente, i gruppi non esistono ancora.
- Al termine del passaggio 3 (**GROUP BY**), le singole righe originali collassano scomparendo definitivamente. Pertanto, è impossibile inserire nell'**HAVING** una condizione relativa a uno specifico attributo di una singola tupla originale, a meno che tale attributo non faccia parte della chiave di raggruppamento.

Osservazione 5.6 (L'invisibilità degli Alias nell'**HAVING**). Poiché la clausola **HAVING** viene valutata al passaggio 4, mentre la ridenominazione delle colonne (tramite l'alias **AS**) avviene solo nel blocco **SELECT** al passaggio 5, molti DBMS standard (es. Oracle, SQL Server, PostgreSQL in modalità strict) genereranno un errore se si tenta di utilizzare un alias definito nella **SELECT** all'interno della clausola **HAVING**. Il motore relazionale, banalmente, "non conosce ancora" quel nuovo nome.



Conseguenze Architettureali: La Regola del GROUP BY

Questo rigido ordine cronologico fornisce la spiegazione alla regola sintattica fondamentale introdotta nel paragrafo sulle aggregazioni: *"Ogni attributo nella SELECT non incluso in una funzione di aggregazione deve essere presente nel GROUP BY"*.

Esempio 5.19 (La logica dietro l'errore di sintassi). Consideriamo l'interrogazione errata:

```

1  SELECT matricola, corso, COUNT(voto)
2  FROM esame
3  GROUP BY matricola;
```

Seguendo l'ordine di esecuzione, al passaggio 3 il motore comprime tutti gli esami di una *matricola* in un'unica partizione. Al passaggio 5, la **SELECT** chiede di stampare il *corso*. Tuttavia, poiché in una partizione ci sono esami di corsi multipli ormai fusi insieme, il motore si trova di fronte a un'ambiguità irrisolvibile: non sa quale stringa di corso estrarre e blocca l'esecuzione.

5.11 Le Viste (Views): Tabelle Virtuali

Una vista (*View*) è un'interrogazione SQL a cui viene associato un nome univoco all'interno del dizionario del database. Le viste vengono impiegate per scopi di astrazione: permettono di incapsulare logiche complesse (come giunzioni o raggruppamenti) per favorirne la leggibilità e il riutilizzo continuo, evitando la duplicazione del codice all'interno degli applicativi.

Osservazione 5.7 (La natura virtuale delle Viste). A differenza delle tabelle fisiche (le *Base Tables*), il risultato di una vista standard **non viene memorizzato in modo permanente** sul disco. Il DBMS si limita a salvare la definizione logica (il codice SQL) della vista stessa. Ogni volta che un utente la interroga, il motore ricalcola i dati al volo basandosi sulle tabelle sottostanti. Si tratta, a tutti gli effetti, di una **tabella virtuale**.

Una volta creata, una vista può essere utilizzata in operazioni di lettura (tramite **SELECT**) esattamente come se fosse una normale tabella residente nel database. Esistono anche casistiche in cui è possibile utilizzare le viste in scrittura (**INSERT**, **UPDATE**, **DELETE**) per modificare direttamente i dati delle tabelle fisiche originarie, ma l'aggiornabilità di una vista è soggetta a vincoli strutturali molto stringenti che verranno analizzati in seguito.

Sintassi e Creazione La sintassi standard per istanziare una tabella virtuale prevede la dichiarazione del nome e, opzionalmente, la definizione di una lista di parametri per ridenominare le colonne in uscita:

```

1  CREATE VIEW nome_vista (attr_1, attr_2, ..., attr_n) AS
2  SELECT espressione_1, espressione_2, ..., espressione_n
3  FROM ...
```

Vincolo di firma: Se si sceglie di esplicitare la lista dei parametri tra parentesi tonde, è obbligatorio che il numero di questi parametri corrisponda esattamente al numero di attributi generati dalla clausola **SELECT** sottostante. Inoltre, deve esserci una perfetta corrispondenza ordinata per quanto riguarda il dominio (il tipo di dato) di ciascuna colonna.

Esempio 5.20 (Creazione di una vista statistica). Vogliamo creare una vista riutilizzabile che fornisca dinamicamente, per ogni studente, il numero totale di esami sostenuti e la media dei voti. Sfruttiamo i parametri della vista per rinominare in modo pulito le funzioni di aggregazione.

```
1 CREATE VIEW statistiche_studenti (matricola, numero_esami,
   media_voti) AS
2 SELECT matr, COUNT(corso), AVG(voto)
3 FROM esame
4 GROUP BY matr;
```

Da questo momento in poi, è possibile interrogare queste statistiche pre-calcolate in modo estremamente compatto, trattando la vista come una tabella fisica e ignorando la complessità del raggruppamento originario:

```
1 SELECT matricola, media_voti
2 FROM statistiche_studenti
3 WHERE media_voti > 27;
```

5.12 Interrogazioni Nidificate (Subqueries)

Nel linguaggio SQL, un'interrogazione (SELECT) può essere innestata all'interno di un'altra in diversi punti della struttura sintattica.

5.12.1 Subqueries in WHERE (filtri dinamici)

L'operatore [NOT] EXISTS e le Subquery Correlate La clausola EXISTS valuta esclusivamente se l'insieme restituito dalla subquery è vuoto o meno. Restituisce TRUE se la sotto-interrogazione produce almeno una riga, FALSE in caso contrario.

Esempio 5.21 (Studenti senza esami tramite NOT EXISTS). Per isolare gli studenti privi di esami registrati:

```
1 SELECT s.matricola, s.nome
2 FROM studente AS s
3 WHERE NOT EXISTS (
4     SELECT *
5     FROM esame AS e
6     WHERE e.matr = s.matricola
7 );
```

Osservazione 5.8 (Scope dei Nomi e Interrogazioni Correlate). Nell'esempio precedente, la query interna utilizza nella clausola WHERE l'alias `s.matricola`, che appartiene allo *scope* (visibilità) della query esterna. In SQL, una subquery può "vedere" gli attributi definiti nei blocchi superiori, ma non viceversa.

Quando una sotto-interrogazione riferenzia uno o più attributi della query esterna, viene definita **correlata**. Una subquery correlata è totalmente dipendente dal contesto esterno e **non** può essere eseguita in modo isolato. Di conseguenza, *il motore del database è costretto a rieseguire la query interna per ogni singola riga analizzata dalla query esterna.*

L'operatore [NOT] IN e le Subquery Scorrelate L'operatore IN viene impiegato per verificare se un determinato attributo (o gruppo di attributi) è presente all'interno dell'elenco di valori generato dalla subquery.

Esempio 5.22 (Uso di IN con Subquery Scorrelata).

```

1 SELECT matricola, nome
2 FROM studente
3 WHERE matricola IN (
4     SELECT matr
5     FROM esame
6     WHERE corso = 'Basi di Dati'
7 );

```

⚠ Analisi Prestazionale: Correlata vs Scorrelata

Nell'esempio soprastante, la subquery non fa alcun riferimento agli attributi della query esterna. Viene pertanto definita **scorrelata** (o indipendente).

Il vantaggio principale di una subquery scorrelata è che può essere eseguita **una sola volta** in modo del tutto autonomo. Il DBMS calcola il risultato, lo immagazzina in memoria e lo utilizza per valutare tutte le righe della tabella principale. Tuttavia, non bisogna commettere l'errore di credere che le interrogazioni scorrelate siano sempre prestazionalmente superiori a quelle correlate. Se la tabella virtuale generata dalla subquery scorrelata dovesse risultare enorme (es. milioni di record), il costo computazionale di dover scansionare ripetutamente quell'immensa lista per ogni riga esterna diventerebbe insostenibile. La natura scorrelata garantisce un vantaggio significativo **se e solo se** la tabella risultante è di dimensioni sufficientemente ridotte da poter essere interrogata rapidamente.

Sintassi Generale e Confronto Multi-Attributo L'uso degli operatori come IN o gli operatori di confronto standard non è limitato alla singola colonna. È possibile valutare intere n -uple di attributi contemporaneamente, purché venga rispettato un vincolo strutturale: **il numero e il dominio degli attributi a sinistra dell'operatore deve corrispondere ordinatamente a quelli degli attributi presenti nella SELECT innestata**. In caso di più attributi, l'uso delle parentesi è obbligatorio.

Sintassi generale:

```

1 SELECT <elenco_attributi_esterni>
2 FROM <tabella_esterna>
3 WHERE (attr_1, attr_2, ..., attr_n) [OPERATORE] (
4     SELECT espressione_1, espressione_2, ..., espressione_n
5     FROM <tabella_interna>
6     WHERE <condizione>
7 );

```

Esempio 5.23 (Confronto simultaneo su tuple multiple). Supponiamo di voler estrarre gli studenti che hanno conseguito esattamente lo stesso voto, nello stesso corso, di un determinato studente target (es. la matricola 101).

```

1 SELECT matr, corso, voto
2 FROM esame

```

```

3 WHERE matr <> 101 AND (corso, voto) IN (
4     SELECT corso, voto
5     FROM esame
6     WHERE matr = 101
7 );

```

Il DBMS valuterà in blocco la tupla (*corso*, *voto*) della query esterna verificandone la presenza tra le tuple (*corso*, *voto*) restituite dalla query interna.

Confronti Scalari Quando la logica dell'interrogazione garantisce che la subquery restituirà **esattamente un unico valore** (una sola riga e una sola colonna), essa prende il nome di subquery scalare. In questo scenario, la query interna può essere utilizzata sul lato destro dei tradizionali operatori di confronto relazionale (=, <>, >, <, >=, <=).

Questo costrutto è tipicamente impiegato quando la subquery utilizza una funzione di aggregazione globale (senza clausola GROUP BY), la quale per definizione matematica collassa l'intera relazione in uno scalare.

Esempio 5.24 (Subquery scalare: Voti sopra la media). Vogliamo estrarre le matricole degli studenti che hanno superato l'esame di "Basi di Dati" con un voto strettamente superiore alla media generale di quello specifico corso.

```

1 SELECT matr, voto
2 FROM esame
3 WHERE corso = 'Basi di Dati'
4     AND voto > (
5         SELECT AVG(voto)
6         FROM esame
7         WHERE corso = 'Basi di Dati'
8     );

```

Se, a tempo di esecuzione, una subquery concepita come scalare dovesse inaspettatamente restituire più di una riga, il DBMS solleverebbe un errore critico bloccando l'esecuzione.

Quantificatori: ANY, SOME e ALL Per generalizzare il concetto dei confronti qualora la subquery restituisca **una colonna con molteplici righe**, lo standard SQL estende i normali operatori relazionali combinandoli con i quantificatori logici ANY (il cui sinonimo perfettamente equivalente è SOME) e ALL.

- **> ANY (o < ANY, = ANY, ecc.):** Il predicato risulta TRUE se la condizione è soddisfatta per **almeno uno** dei valori restituiti dalla subquery. (Nota: = ANY è logicamente e sintatticamente identico all'operatore IN).
- **> ALL (o < ALL, <> ALL, ecc.):** Il predicato risulta TRUE se la condizione è soddisfatta per **tutti** i valori restituiti dalla subquery. (Nota: <> ALL è logicamente e sintatticamente identico all'operatore NOT IN).

Esempio 5.25 (Uso di ANY e ALL). Supponiamo di voler trovare gli studenti che hanno preso un voto in "Basi di Dati" superiore a *qualsiasi* voto mai registrato nel corso di "Reti".

Se vogliamo assicurarci che il voto sia maggiore del più alto voto in assoluto preso in "Reti" (deve batterli **tutti**):

```

1 SELECT matr
2 FROM esame

```



```
3 WHERE corso = 'Basi di Dati'
4 AND voto > ALL (
5     SELECT voto
6     FROM esame
7     WHERE corso = 'Reti'
8 );
```

Se invece ci basta che il voto sia superiore ad **almeno uno** dei voti presi in "Reti" (ovvero, basta che sia maggiore del voto minimo registrato in quel corso):

```
1 SELECT matr
2 FROM esame
3 WHERE corso = 'Basi di Dati'
4 AND voto > ANY (
5     SELECT voto
6     FROM esame
7     WHERE corso = 'Reti'
8 );
```

✓ La realtà ingegneristica: MAX e MIN vs ANY e ALL

Sebbene ANY e ALL appartengano allo standard SQL fin dalle origini per pura aderenza alla teoria degli insiemi, nella pratica industriale moderna sono operatori considerati obsoleti e raramente utilizzati dagli sviluppatori.

Il motivo è duplice: sono controintuitivi da leggere per un umano e, spesso, meno ottimizzati dai DBMS. Scrivere `voto > ALL (...)` è funzionalmente identico a scrivere `voto > (SELECT MAX(voto) ...)`, ma la seconda versione (con la subquery scalare e la funzione MAX) risulta semanticamente molto più esplicita e permette al motore di estrarre il valore massimo in modo estremamente più rapido ed efficiente rispetto a un controllo logico riga per riga richiesto da ALL.

5.12.2 Subqueries in FROM (Tabelle Derivate o Inline Views)

Oltre ad essere utilizzate come filtri dinamici nella clausola **WHERE**, le interrogazioni nidificate possono essere posizionate direttamente all'interno della clausola **FROM**. In questo contesto, il risultato della subquery viene trattato dal motore relazionale come se fosse una vera e propria tabella fisica temporanea, sulla quale è possibile effettuare proiezioni, selezioni e giunzioni (**JOIN**).

Questa struttura prende il nome tecnico di **Tabella Derivata** (*Derived Table*) o **Inline View** (vista in linea). Il concetto logico è identico a quello di una vista permanente (**VIEW**), con la differenza fondamentale che una tabella derivata è effimera: esiste esclusivamente nella memoria del server per la durata di esecuzione della singola interrogazione principale, senza inquinare il dizionario del database.

Il Vincolo dell'Alias Obbligatorio A differenza delle normali tabelle fisiche in cui la ridenominazione è opzionale, lo standard SQL impone una regola sintattica ferrea per le tabelle derivate: **ogni subquery posizionata nel FROM deve obbligatoriamente essere seguita da un alias** (es. `AS nome_alias`). Senza questo identificativo, la query esterna non avrebbe alcun modo per referenziare le colonne prodotte dalla subquery, e il DBMS bloccherebbe la compilazione sollevando un errore di sintassi.

Esempio 5.26 (Pre-aggregazione tramite Tabella Derivata). Supponiamo di voler estrarre l'anagrafica degli studenti affiancando a ciascuno il proprio voto massimo, ma prendendo in considerazione esclusivamente coloro che hanno conseguito almeno un 30.

Invece di raggruppare l'intera unione di studenti ed esami, possiamo creare una tabella temporanea già raggruppata e pre-filtrata, per poi congiungerla con i dati anagrafici tramite un INNER JOIN:

```

1 SELECT s.matricola, s.nome, eccellenze.voto_max
2 FROM studente AS s
3 JOIN (
4     SELECT matr, MAX(voto) AS voto_max
5     FROM esame
6     GROUP BY matr
7     HAVING MAX(voto) = 30
8 ) AS eccellenze ON s.matricola = eccellenze.matr;
```

In questo scenario, l'ordine di esecuzione prevede che il DBMS calcoli prima la subquery isolata, costruendo in memoria la tabella virtuale `eccellenze`. Solo successivamente elaborerà il JOIN tra la tabella fisica `studente` e la neonata tabella derivata.



Vantaggio Architetture: Sostituire le Correlate

Nell'ingegneria dei dati reale, l'uso delle subquery nel `FROM` rappresenta la soluzione d'elezione per ottimizzare le prestazioni. Molte interrogazioni che verrebbero intuitivamente scritte utilizzando lente *subquery correlate* nel `WHERE` (le quali costringono il motore a riavviare il calcolo riga per riga), possono essere "srotolate" calcolando preventivamente il dato in blocco all'interno di una tabella derivata nel `FROM`, unendolo poi con un normale JOIN.

Questo approccio sposta il carico di lavoro dai lenti cicli iterativi ai velocissimi algoritmi di giunzione massiva del DBMS, abbattendo drasticamente la complessità computazionale e i tempi di esecuzione.

5.12.3 Subqueries in SELECT (Colonne Calcolate)

L'ultimo posizionamento strutturale consentito dallo standard SQL per un'interrogazione nidificata è all'interno della clausola `SELECT`. In questo scenario, la subquery non filtra le tuple né genera tabelle di base, ma agisce a tutti gli effetti come un'espressione che calcola dinamicamente il valore di una singola colonna per ogni riga restituita dall'interrogazione esterna.

Il Vincolo Scalare Assoluto La regola matematica per le subquery nella clausola `SELECT` è la più rigida in assoluto: **l'interrogazione interna deve essere strettamente scalare**. Deve cioè restituire, in ogni circostanza possibile, **esattamente una sola riga e una sola colonna**. Se a tempo di esecuzione la subquery producesse due colonne o due righe, il DBMS non saprebbe come incastrare quell'insieme bidimensionale all'interno di una singola cella della tabella risultante e solleverebbe un'eccezione bloccante (errore di cardinalità).

Esempio 5.27 (Affiancare aggregazioni globali a dati di riga). Un classico caso d'uso ingegneristico è la necessità di confrontare un dato specifico con una statistica globale,

evitando di alterare la granularità dell'interrogazione esterna con un complesso **GROUP BY**. Vogliamo estrarre il nome degli studenti e affiancare a ciascuno la media generale di tutto l'ateneo, per poterne successivamente calcolare lo scostamento:

```

1 SELECT
2     matricola ,
3     nome ,
4     (SELECT AVG(voto) FROM esame) AS media_ateneo
5 FROM studente;
```

Poiché la subquery è scorrelata, il motore calcolerà la media globale una sola volta (es. 26.5) e replicherà quel valore scalare identico in ogni riga prodotta dalla tabella **studente**.

⚠ Il collasso prestazionale: Subquery Correlate nella SELECT

È possibile, e sintatticamente lecito, inserire subquery **correlate** all'interno della **SELECT**. Ad esempio, per stampare per ogni studente il numero dei suoi esami sostenuti:

```

1 SELECT
2     s.matricola ,
3     s.nome ,
4     (SELECT COUNT(*) FROM esame AS e WHERE e.matr = s.
5         matricola) AS num_esami
6 FROM studente AS s;
```

Sebbene il codice risulti compatto e molto leggibile, dal punto di vista ingegneristico questa pratica è considerata un **anti-pattern gravissimo** se applicata a tabelle di grandi dimensioni. Poiché la subquery è correlata, il motore del database è fisicamente costretto a lanciare l'interrogazione di conteggio *ex novo* per ogni singola riga della tabella principale. Su una tabella di un milione di studenti, il DBMS eseguirà un milione e una interrogazioni separate.

Nel codice di produzione, questa logica deve quasi sempre essere riscritta utilizzando un **LEFT OUTER JOIN** abbinato a un **GROUP BY**, oppure sfruttando una Tabella Derivata nel **FROM**, delegando così il calcolo ai velocissimi algoritmi insiemistici del motore relazionale anziché a un ciclo iterativo riga per riga.

5.13 Viste vs Interrogazioni Nidificate

Dal punto di vista puramente logico e algebrico, una Vista e una sotto-interrogazione (in particolare una *Inline View* nella clausola **FROM**) svolgono la medesima funzione: incapsulano una logica di calcolo per generare una "tabella virtuale". Poiché le viste offrono un grado di modularità, astrazione e leggibilità nettamente superiore, è lecito domandarsi per quale motivo lo standard SQL e l'industria del software continuino a fare un uso massiccio delle subquery.

La risposta risiede in tre limiti strutturali e ingegneristici che rendono le interrogazioni nidificate uno strumento insostituibile nel codice di produzione.

L'Inquinamento dello Schema (Schema Bloat) Una vista non è una semplice stringa di codice temporanea, ma un **oggetto di database permanente** che viene registrato formalmente nel *Data Dictionary* del sistema. Nello sviluppo di applicazioni aziendali,

un'interrogazione complessa può richiedere molteplici filtri o raggruppamenti intermedi usa-e-getta. Se gli sviluppatori dovessero creare una vista permanente per ogni singolo calcolo intermedio, il database accumulerebbe in breve tempo decine di migliaia di viste inutili, rendendo la manutenzione del sistema un incubo. Le subquery, al contrario, sono **effimere**: nascono, elaborano i dati e vengono distrutte dalla memoria non appena la query principale termina l'esecuzione, mantenendo l'architettura del database pulita.

Il Limite: L'impossibilità della Correlazione Questo rappresenta l'ostacolo teorico invalicabile per la sostituzione totale. Come analizzato nei capitoli precedenti, una subquery **correlata** riceve e utilizza un parametro dinamico passato riga per riga dalla query esterna. Una vista, al contrario, è per definizione un oggetto **statico** e precompilato, privo di argomenti in ingresso. È quindi impossibile trasformare una subquery correlata (es. `WHERE e.matr = s.matricola`) in una vista autonoma, poiché al di fuori di quell'esatto ciclo di esecuzione la vista ignorerebbe totalmente il significato dell'alias `s.matricola`.

Permessi di Sicurezza e Amministrazione (DML vs DDL) Nei sistemi informativi reali, l'accesso ai database è governato dal principio del privilegio minimo.

- Scrivere ed eseguire una subquery è un'operazione di interrogazione dati (**DML** - *Data Manipulation Language*). Richiede unicamente il permesso di lettura (**SELECT**) sulle tabelle di base coinvolte.
- Creare una vista è un'operazione di definizione strutturale (**DDL** - *Data Definition Language*). Richiede permessi di amministrazione avanzati (es. **CREATE VIEW**) che gli sviluppatori standard o gli analisti dati non possiedono mai negli ambienti di produzione per ragioni di sicurezza.

Senza la possibilità di formulare subquery al volo, gli utenti non privilegiati sarebbero di fatto impossibilitati a scrivere interrogazioni complesse.

✓ Best Practice: Views vs Subqueries

La separazione dei ruoli nell'ingegneria del software è netta:

- Si implementa una **Vista** quando una specifica logica di calcolo (es. "calcolo del fatturato trimestrale per dipartimento") deve essere interrogata quotidianamente da molteplici utenti, query o applicazioni diverse, massimizzandone il riutilizzo.
- Si utilizza una **Subquery** quando si necessita di un "tavolo di lavoro temporaneo" per isolare e risolvere un problema specifico all'interno di un'unica e isolata interrogazione.

5.14 Ordinamento dei Risultati: La clausola ORDER BY

Nel modello relazionale puro, le tabelle sono insiemi matematici non ordinati. Il DBMS restituisce le righe nell'ordine in cui le legge fisicamente dal disco o dalla memoria, che è del tutto imprevedibile. Per imporre un ordinamento logico e deterministico al *Result Set* finale, lo standard SQL mette a disposizione la clausola **ORDER BY**.

Dal punto di vista della sintassi, **ORDER BY** deve essere **rigorosamente l'ultima clausola** dell'intera interrogazione.

Direzione di Ordinamento e Criteri Multipli La clausola accetta uno o più attributi, a ciascuno dei quali può essere associato un modificatore di direzione:

- **ASC** (Ascendente): Ordina dal minore al maggiore (A-Z, 0-9). È il comportamento di default assoluto e può essere omesso.
- **DESC** (Discendente): Ordina dal maggiore al minore (Z-A, 9-0).

Una delle caratteristiche architetturali più potenti dell'**ORDER BY** è la possibilità di applicare l'ordinamento su **molteplici attributi** in cascata, ciascuno con il proprio criterio indipendente. Il motore elabora la lista da sinistra verso destra: il primo attributo definisce l'ordinamento primario; solo in caso di "pareggio" (valori identici nel primo attributo), il motore utilizza il secondo attributo per risolvere il conflitto, e così via.

Esempio 5.28 (Ordinamento combinato: ASC e DESC). Vogliamo visualizzare un elenco di esami superati, raggruppandoli visivamente per corso (in ordine alfabetico). A parità di corso, vogliamo vedere i voti dal più alto al più basso.

```
1 SELECT corso, matr, voto
2 FROM esame
3 WHERE voto >= 18
4 ORDER BY corso ASC, voto DESC;
```

Il DBMS creerà blocchi alfabetici per i corsi (es. prima "Analisi", poi "Basi di Dati"). All'interno del blocco "Basi di Dati", le righe saranno ordinate internamente partendo dal 30 a scendere.

Ordine di esecuzione e gestione dei NULL

A differenza di quanto si possa intuire, la clausola **ORDER BY** è l'unica istruzione del linguaggio SQL ad essere eseguita **DOPO** la **SELECT**.

La sequenza di esecuzione definitiva del motore è: **FROM** → **WHERE** → **GROUP BY** → **HAVING** → **SELECT** → **ORDER BY**.

Questa è una necessità matematica: operando per ultima, la clausola di ordinamento ha piena visibilità su tutto ciò che è stato calcolato o ridenominato dalla **SELECT**, permettendo di ordinare i risultati in base a colonne generate al volo o ai loro alias.

Un altro dettaglio strutturale vitale nello sviluppo reale riguarda la gestione dei valori **NULL**. Come si posiziona un "dato mancante" in una lista ordinata? Non essendo quantificabile, il DBMS lo tratta come un'entità a sé stante. A seconda del motore utilizzato (Oracle, PostgreSQL, SQL Server), durante un ordinamento tutti i valori **NULL** verranno spinti in blocco all'inizio o alla fine del *Result Set*. Se non si gestisce esplicitamente questa casistica, si rischia di avere interfacce applicative in cui le prime cento righe visualizzate da un utente risultano completamente vuote.

5.15 Espressioni e Funzioni Predefinite nella SELECT

Finora abbiamo utilizzato la clausola **SELECT** per estrarre il valore puro degli attributi fisici presenti nelle tabelle. Tuttavia, lo standard SQL permette di sostituire o affiancare i nomi delle colonne con **espressioni calcolate** al volo. Un'espressione può essere una costante, un'operazione matematica o il risultato di una funzione predefinita.

5.15.1 Uso di Espressioni Costanti

Inserire una costante nella **SELECT** impone al DBMS di proiettare quel valore letterale identico per ogni singola riga del *Result Set*. Questa tecnica trova vastissima applicazione ingegneristica per "taggare" i risultati, specialmente quando si uniscono dati da tabelle diverse tramite **UNION**, o per generare flag di controllo.

Esempio 5.29 (Etichettatura tramite costanti).

```
1 SELECT matricola, nome, 'Studente Attivo' AS stato_iscrizione,
   2024 AS anno_riferimento
2 FROM studente;
```

Il DBMS creerà due nuove colonne virtuali: `stato_iscrizione` conterrà la stringa "Studente Attivo" e `anno_riferimento` conterrà il numero 2024 per tutte le tuple estratte.

5.15.2 Panoramica delle Funzioni Predefinite Standard

Lo standard SQL fornisce una libreria di funzioni scalari (*Built-in Functions*) per manipolare i dati riga per riga. A differenza delle funzioni di aggregazione (**SUM**, **MAX**) che operano su interi gruppi, le funzioni scalari prendono in input i valori di una singola tupla e restituiscono un singolo valore trasformato.

1. Funzioni Matematiche Consentono di eseguire calcoli algebrici direttamente sul server database, evitando di sovraccaricare la memoria del codice applicativo (Java, Python, ecc.).

- **ROUND(valore, decimali)**: Arrotonda un numero alla precisione specificata.
- **ABS(valore)**: Calcola il valore assoluto.
- Operatori aritmetici standard: +, -, *, /

2. Funzioni per le Stringhe Permettono di manipolare campi testuali (es. formattazione, estrazione di sottostringhe).

- **UPPER(stringa)** e **LOWER(stringa)**: Convertono rispettivamente in maiuscolo e minuscolo. Utili per effettuare confronti *case-insensitive* nella clausola **WHERE**.
- **SUBSTRING(stringa FROM inizio FOR lunghezza)**: Estrae una porzione di testo.
- **Concatenazione**: Lo standard ANSI utilizza l'operatore `||` per unire due stringhe (es. `nome || ' ' || cognome`).

3. Funzioni per le Date (Temporal) La gestione del tempo è fondamentale nei database relazionali. Lo standard ANSI SQL definisce funzioni specifiche per l'estrazione e la manipolazione temporale.

- **CURRENT_DATE** e **CURRENT_TIMESTAMP**: Restituiscono rispettivamente la data odierna e la data comprensiva di orario al momento dell'esecuzione della query. (Non richiedono parentesi).
- **EXTRACT(elemento FROM data)**: Estrae una componente specifica (es. **YEAR**, **MONTH**, **DAY**) da un attributo temporale.

Esempio 5.30 (Manipolazione di Date e Stringhe). Vogliamo estrarre un report formattato con i cognomi in maiuscolo e l'anno esatto in cui è stato sostenuto un esame.

```

1 SELECT
2     UPPER(s.cognome) AS cognome_formattato,
3     EXTRACT(YEAR FROM e.data_esame) AS anno_conseguimento,
4     ROUND(e.voto, 0) AS voto_intero
5 FROM studente AS s
6 JOIN esame AS e ON s.matricola = e.matr;

```

⚠ La verità sul campo: La frammentazione dei dialetti SQL

Dal punto di vista ingegneristico, è vitale sapere che le funzioni scalari, e in particolar modo **le funzioni sulle date e la concatenazione delle stringhe**, rappresentano l'area in cui i vari produttori di DBMS si discostano maggiormente dallo standard ANSI.

Se `EXTRACT` è parte dello standard, nel mondo reale:

- **SQL Server (Microsoft)** utilizza `DATEPART()` o `YEAR()` e concatena le stringhe col segno `+`.
- **Oracle** utilizza `TO_CHAR()` per estrarre parti di date.
- **MySQL** utilizza `DATE_FORMAT()` o funzioni dirette come `MONTH()` e concatena con la funzione `CONCAT()`.

In contesti didattici si usa la sintassi standard ANSI (`EXTRACT` e `||`), ma in ambiente lavorativo le interrogazioni contenenti funzioni sulle date raramente sono "portabili" da un database all'altro senza un'attenta riscrittura del codice.

5.16 Manipolazione dei Dati (DML): INSERT, UPDATE, DELETE

Il *Data Manipulation Language* (DML) comprende le istruzioni necessarie per popolare, modificare e svuotare le tabelle del database. Qualsiasi operazione DML è rigorosamente soggetta al controllo dei vincoli di integrità (chiavi primarie, esterne, **UNIQUE**, **NOT NULL**, **CHECK**): se l'operazione viola anche un solo vincolo su una singola riga, l'intero blocco di esecuzione viene rigettato dal motore relazionale.

5.16.1 L'istruzione INSERT

L'inserimento di nuove tuple all'interno di una relazione può avvenire tramite due costrutti sintattici ben distinti, a seconda che i dati provengano da un input esterno o siano calcolati dinamicamente dal database stesso.

1. Inserimento di valori espliciti (INSERT ... VALUES) Questa forma viene impiegata quando i dati da inserire sono noti a priori (es. parametri passati da un'applicazione backend).

```

1 INSERT INTO nome_tabella (attr_A, attr_B, attr_C)
2 VALUES (valore_A, valore_B, valore_C);

```

La lista degli attributi è tecnicamente opzionale, ma ometterla è considerata una pessima pratica, poiché costringe a fornire i valori nell'esatto ordine fisico di creazione della tabella, esponendo il codice a rotture silenti se lo schema dovesse cambiare in futuro.

Osservazione 5.9 (Gestione dei campi opzionali e dei valori NULL). Esplicitare la lista delle colonne permette non solo di ignorare l'ordine fisico originale, ma anche di gestire elegantemente gli attributi non obbligatori. Se una tabella possiede 20 attributi di cui solo 3 sono obbligatori (NOT NULL), è sufficiente elencare e valorizzare solo quei 3. Il DBMS assegnerà automaticamente il valore NULL (o il valore di DEFAULT, se definito) a tutti gli attributi omessi dalla lista. *Vincolo strutturale*: Il numero e il dominio dei valori presenti in VALUES deve sempre corrispondere esattamente al numero e al dominio delle colonne elencate nella direttiva INSERT.

2. Inserimento massivo calcolato (INSERT ... SELECT) Questa variante sostituisce la clausola VALUES con un'intera interrogazione nidificata. È lo strumento d'elezione per le migrazioni di dati o per il popolamento di tabelle di storicizzazione.

Esempio 5.31 (Archiviazione massiva).

```
1 INSERT INTO archivio_esami (matricola, corso, voto_definitivo)
2 SELECT matr, corso, voto
3 FROM esame
4 WHERE anno < 2020;
```

Piuttosto che estrarre i dati in Java, ciclarli in un ciclo `for` e lanciare migliaia di INSERT singole, questa sintassi delega il travaso massivo direttamente al motore relazionale, abbattendo drasticamente i tempi di rete e di esecuzione.

5.16.2 L'istruzione UPDATE

L'istruzione UPDATE altera i valori degli attributi di una o più tuple già esistenti. La clausola SET accetta assegnazioni multiple separate da virgola, e i nuovi valori possono derivare da costanti, espressioni matematiche o *subquery* scalari.

```
1 UPDATE nome_tabella
2 SET
3     attr_1 = espressione_1,
4     attr_2 = (SELECT espressione FROM ... WHERE ...)
5 WHERE condizione;
```

Esempio 5.32 (Aggiornamento basato su espressioni e filtri). Aumentiamo di 2 punti il voto a tutti gli studenti che hanno sostenuto l'esame di "Architettura" con un professore specifico, senza superare il tetto massimo del 30:

```
1 UPDATE esame
2 SET voto = LEAST(voto + 2, 30)
3 WHERE corso = 'Architettura' AND data_esame < '2024-01-01';
```

5.16.3 L'istruzione DELETE

L'istruzione DELETE rimuove fisicamente intere righe dalla tabella. Poiché i database relazionali non permettono di eliminare il valore di una singola cella (per quello si usa UPDATE SET colonna = NULL), la sintassi non richiede l'elenco degli attributi.

```
1 DELETE FROM nome_tabella
2 WHERE condizione;
```


**La Regola del WHERE nel codice di produzione**

Sia in UPDATE che in DELETE, la clausola WHERE è sintatticamente opzionale ma operativamente vitale. Omettere il WHERE in un UPDATE sovrascriverà quel dato in **tutte** le righe della tabella. Ometterlo in una DELETE svuoterà l'intera tabella.

SQL Avanzato in PostgreSQL

Nei capitoli precedenti abbiamo affrontato la sintassi e gli operatori dell'SQL standard (ANSI SQL), concentrandoci sui costrutti universali supportati in modo trasversale da qualsiasi motore relazionale. Questa aderenza rigorosa allo standard è fondamentale per garantire la portabilità di base del database.

Tuttavia, nello sviluppo di applicazioni *Enterprise* moderne (come i backend Java con Spring Boot), limitarsi allo standard puro rappresenta spesso un collo di bottiglia prestazionale. I DBMS moderni mettono a disposizione estensioni e funzioni analitiche avanzate che permettono di spostare carichi di calcolo pesanti (come reportistica, partizionamento statistico e logiche condizionali) dal server applicativo direttamente all'interno del motore del database, abbattendo drasticamente i tempi di rete e l'uso della memoria RAM dell'applicativo.

In questo capitolo abbandoneremo il rigore dell'ANSI SQL puro per esplorare le funzionalità avanzate offerte dal dialetto di **PostgreSQL**. Studieremo i costrutti architetturali indispensabili per l'ingegneria del software moderna: le espressioni di tabella comuni (CTE), la logica condizionale e, soprattutto, le funzioni analitiche (Window Functions).

Per gli esempi di questo capitolo utilizzeremo quasi sempre un dominio assicurativo (polizze auto) riferendoci ad una immaginaria compagnia *SafeDrive InsurTech*.

6.1 Controllo del Result Set: Limit, Offset e Nulls

Prima di esplorare i costrutti avanzati di aggregazione e manipolazione, è fondamentale padroneggiare gli strumenti che governano la dimensione e la forma esatta dei dati restituiti al backend. Estrarre intere tabelle per poi filtrarle o ordinarle in memoria (ad esempio tramite le `Collection` di Java) è l'anti-pattern prestazionale per eccellenza. Il database relazionale deve restituire esclusivamente la porzione di dati che l'interfaccia utente è fisicamente in grado di visualizzare.

6.1.1 Taglio e Paginazione: LIMIT e OFFSET

Nelle applicazioni aziendali, le liste di dati vengono sempre impaginate (es. 10, 50 o 100 risultati per pagina). In PostgreSQL, il controllo granulare sulla quantità di righe estratte è delegato alle clausole `LIMIT` e `OFFSET`, che si posizionano alla fine assoluta dell'istruzione SQL.

- **LIMIT *n***: Istruisce il motore a interrompere l'estrazione non appena ha accumulato esattamente *n* righe.
- **OFFSET *m***: Istruisce il motore a scartare le prime *m* righe prima di iniziare a restituire i risultati.

Esempio 6.1 (Paginazione dei Sinistri (Pagina 2)). Il frontend di *SafeDrive* richiede la "Pagina 2" di una lista cronologica di sinistri, visualizzando 10 risultati per pagina. Questo significa che il database deve saltare i primi 10 record (quelli della Pagina 1) e restituire i successivi 10.

```
1 SELECT codice_sinistro, data_apertura, danno_stimato
2 FROM sinistro
3 ORDER BY data_apertura DESC
```

```
4 | LIMIT 10 OFFSET 10;
```

Osservazione 6.1 (L'Obbligo dell'ORDER BY). L'utilizzo della clausola LIMIT senza un'esplícita clausola ORDER BY è un grave errore di progettazione. Senza un ordinamento forzato, i database relazionali restituiscono le righe nell'ordine fisico casuale in cui le trovano allocate sul disco in quella specifica frazione di secondo (che può variare dopo ogni aggiornamento). Per garantire risultati predicibili e una paginazione che non mostri record duplicati o mancanti tra una pagina e l'altra, LIMIT e ORDER BY devono sempre essere accoppiati.

6.1.2 Gestione delle Incognite nell'Ordinamento: NULLS FIRST / LAST

Un aspetto critico e spesso trascurato dello sviluppo SQL riguarda il posizionamento dei valori NULL (le incognite) durante un ordinamento.

In PostgreSQL, per convenzione matematica interna, un valore NULL è considerato "più grande" di qualsiasi valore noto. Questo genera un effetto collaterale pericoloso per le interfacce utente: se si richiede un ordinamento decrescente (DESC) per mostrare i numeri più alti in cima, tutte le righe con valore NULL appariranno per prime, relegando i dati utili in basso.

Per assumere il controllo totale su questo comportamento, si aggiungono i modificatori NULLS FIRST o NULLS LAST direttamente alla dichiarazione della colonna nella clausola ORDER BY.

Esempio 6.2 (Classifica dei Danni con Dati Mancanti). La direzione richiede la lista dei sinistri ordinata per danno stimato, dal più costoso al meno costoso. Alcuni sinistri appena aperti non sono ancora stati periziati, pertanto il loro danno_stimato è fisicamente NULL.

Utilizzando un semplice ORDER BY danno_stimato DESC, i sinistri senza stima finirebbero in cima al report. Correggiamo la logica forzando i valori nulli alla fine del *Result Set*:

```
1 | SELECT codice_sinistro, danno_stimato
2 | FROM sinistro
3 | ORDER BY danno_stimato DESC NULLS LAST;
```

In questo modo, i sinistri con danni milionari si troveranno correttamente al vertice, mentre tutti i record in attesa di perizia scivoleranno ordinatamente in fondo all'elenco, indipendentemente dalla direzione decrescente dell'ordinamento.

6.2 DML Avanzato in PostgreSQL: RETURNING e UPSERT

Lo standard SQL puro presenta alcune limitazioni architetturali quando il database deve interfacciarsi con un'applicazione backend moderna (es. in Java o Python). PostgreSQL estende il DML standard con due costrutti essenziali per garantire la sicurezza concorrentiale e minimizzare le chiamate di rete.

1. La clausola RETURNING (Recupero immediato) Nelle operazioni standard di INSERT, UPDATE o DELETE, il database restituisce all'applicazione chiamante solo il numero di righe modificate. Se durante un INSERT la Chiave Primaria viene generata automaticamente dal database (es. un campo SERIAL), il backend non ha modo di

conoscerla. Richiedere l'ID con una `SELECT MAX(id)` successiva è un gravissimo anti-pattern soggetto a *race conditions* (condizioni di gara) se altri utenti stanno inserendo dati simultaneamente.

PostgreSQL permette di appendere la clausola **RETURNING** alla fine di qualsiasi istruzione DML per farle restituire i dati appena elaborati, comportandosi a tutti gli effetti come una `SELECT` contestuale.

Esempio 6.3 (Recupero della Chiave Primaria).

```
1 INSERT INTO sinistro (id_cliente, data_apertura, danno_stimato)
2 VALUES (42, CURRENT_DATE, 1500.00)
3 RETURNING codice_sinistro, data_apertura;
```

In una singola operazione di rete (Round-Trip), il backend inserisce i dati e riceve immediatamente indietro il `codice_sinistro` appena generato, pronto per essere utilizzato nella logica di business. I framework ORM moderni come Hibernate utilizzano massicciamente questo costrutto "sotto il cofano".

2. L'Upsert (INSERT ... ON CONFLICT) Uno degli scenari applicativi più comuni è la logica di *Upsert* ("Update or Insert"): se un record non esiste nel database, inseriscilo; se esiste già, aggiornalo. Delegare questa logica al codice Java implica l'apertura di una transazione e l'esecuzione di interrogazioni multiple (una `SELECT` di controllo seguita da un `INSERT` o `UPDATE`), consumando risorse e rischiando conflitti di concorrenza.

PostgreSQL gestisce l'operazione in modo nativo e atomico attraverso la clausola **ON CONFLICT**, che intercetta la violazione di un vincolo di univocità (es. Primary Key o constraint `UNIQUE`) e devia l'inserimento verso un aggiornamento.

Esempio 6.4 (Aggiornamento del Budget Dipartimentale). Se il dipartimento 'IT' non è ancora stato registrato a sistema, lo creiamo con un budget iniziale di 50.000. Se invece esiste già (violazione della chiave primaria sul nome del dipartimento), incrementiamo il suo budget attuale di 50.000.

```
1 INSERT INTO dipartimento (nome, budget_totale)
2 VALUES ('IT', 50000)
3 ON CONFLICT (nome)
4 DO UPDATE SET budget_totale = dipartimento.budget_totale + 50000;
```

La pseudo-tabella `EXCLUDED` può essere utilizzata all'interno del blocco `DO UPDATE` per fare riferimento ai valori proposti dalla clausola `VALUES` originale, rendendo il costrutto dinamicamente adattabile.

6.3 Gestione Dati Semi-Strutturati: Il tipo JSONB

Una delle sfide più ardue nello sviluppo backend è il salvataggio di dati con struttura mutevole o imprevedibile (es. risposte di API di terze parti, configurazioni utente dinamiche, o log di sensori hardware). L'approccio relazionale classico costringerebbe alla creazione di decine di tabelle e complessi anti-pattern come l'EAV (*Entity-Attribute-Value*).

PostgreSQL aggira il problema introducendo il tipo di dato nativo **JSONB** (JSON Binary). A differenza di un semplice campo di testo, il `JSONB` memorizza il documento in un formato binario ottimizzato, permettendo l'indicizzazione completa delle chiavi interne e l'interrogazione chirurgica del JSON direttamente da SQL.

Esempio 6.5 (Analisi Telemetria Veicolare). Nel sistema *SafeDrive*, le scatole nere delle auto inviano ogni minuto un payload JSON con struttura variabile. La tabella `telemetria` contiene un attributo `dati_sensore` di tipo JSONB. Il backend ha bisogno di estrarre e mediare solo la velocità di un veicolo specifico, filtrando la data.

Invece di scaricare l'intero documento in Java e farne il *parsing* con librerie come Jackson o Gson, sfruttiamo gli operatori di estrazione JSON di PostgreSQL (->), che navigano il documento e restituiscono il valore come testo:

```
1 SELECT
2     id_veicolo,
3     -- Estrae il valore 'velocita' e lo casta a intero per la
4         matematica
5     AVG((dati_sensore ->> 'velocita')::INTEGER) AS media_velocita
6 FROM telemetria
7 WHERE id_veicolo = 88
8     AND timestamp_ricezione >= CURRENT_DATE;
```

Questa operazione viene eseguita a velocità native relazionali. L'uso di JSONB permette agli sviluppatori di godere della flessibilità strutturale tipica dei database NoSQL (come MongoDB), senza rinunciare all'integrità referenziale, alle transazioni ACID e all'algebra relazionale.

6.4 Common Table Expressions (CTE): Modularità e Ricorsione

Quando la logica di business richiede estrazioni complesse, l'approccio standard basato sulle interrogazioni nidificate (subquery) genera rapidamente un codice illeggibile. Le query annidate costringono il motore e lo sviluppatore a leggere il codice "dall'interno verso l'esterno", rendendo il debugging un incubo operativo.

PostgreSQL e lo standard SQL moderno offrono una soluzione architetturale elegante a questo problema: le **Common Table Expressions (CTE)**, introdotte tramite la clausola `WITH`.

6.4.1 La clausola `WITH` (Sintassi e semantica)

Una CTE agisce come una vista temporanea o una "variabile di tabella" la cui visibilità (scope) è limitata esclusivamente al tempo di esecuzione dell'interrogazione che la definisce.

In termini di ingegneria del software, usare una CTE equivale ad applicare il pattern di refactoring dell'*Extract Method* in Java: si prende un blocco logico complesso, lo si isola assegnandogli un nome semantico e lo si richiama successivamente nel flusso principale.

La sintassi di base prevede la dichiarazione della clausola `WITH` all'inizio assoluto dell'istruzione, seguita dal nome della CTE e dalla query che la popola:

```
1 WITH nome_cte AS (
2     SELECT espressione_1, espressione_2
3     FROM tabella_origine
4     WHERE condizione
5 )
6 SELECT * FROM nome_cte
7 JOIN altra_tabella ON ...;
```

Dal punto di vista dell'algebra relazionale e del piano di esecuzione interno (Query Optimizer), una CTE semplice e una subquery posizionata nella clausola `FROM` (Inline View) sono spesso perfettamente equivalenti: entrambe generano tabelle virtuali in memoria.

La differenza abissale risiede nella **leggibilità top-down** e nella **manutenibilità** del codice sorgente.

Esempio 6.6 (Il vantaggio della Leggibilità). Assumendo un dominio di tipo assicurativo (polizze auto), supponiamo di voler estrarre il nome dei clienti e la targa dei loro veicoli coinvolti in un sinistro 'Under Review', incrociando l'anagrafica con una pre-aggregazione delle polizze attive. Il percorso logico di navigazione è: **Cliente** → **Veicolo** → **Polizza** → **Sinistro**.

Approccio Legacy (Subquery nel FROM): Il codice è un monolite in cui la logica di estrazione delle polizze attive è "sepolta" al centro dell'istruzione principale, spezzando la naturale lettura delle JOIN.

```
1 SELECT c.nome, c.cognome, v.targa
2 FROM cliente c
3 JOIN veicolo v ON c.id_cliente = v.id_cliente
4 JOIN (
5     SELECT id_veicolo, numero_polizza
6     FROM polizza
7     WHERE data_scadenza > CURRENT_DATE
8 ) p_attive ON v.id_veicolo = p_attive.id_veicolo
9 JOIN sinistro s ON p_attive.numero_polizza = s.numero_polizza
10 WHERE s.status = 'Under Review';
```

Approccio Moderno (CTE): La logica viene srotolata in modo lineare. Prima si definisce il "tavolo di lavoro" (le polizze attive), e poi lo si interroga nel blocco principale, mantenendo la catena delle JOIN pulita e leggibile.

```
1 WITH PolizzeAttive AS (
2     SELECT id_veicolo, numero_polizza
3     FROM polizza
4     WHERE data_scadenza > CURRENT_DATE
5 )
6 SELECT c.nome, c.cognome, v.targa
7 FROM cliente c
8 JOIN veicolo v ON c.id_cliente = v.id_cliente
9 JOIN PolizzeAttive pa ON v.id_veicolo = pa.id_veicolo
10 JOIN sinistro s ON pa.numero_polizza = s.numero_polizza
11 WHERE s.status = 'Under Review';
```

Oltre all'innegabile vantaggio sintattico, le CTE offrono capacità strutturali che le subquery non possono eguagliare, come la concatenazione multipla e la ricorsione, come illustriamo a seguire.

6.4.2 CTE Multiple e Concatenate (Data Pipelining)

Il vero vantaggio architetturale delle Common Table Expressions rispetto alle subquery tradizionali emerge quando l'estrazione richiede elaborazioni intermedie multiple.

Lo standard SQL permette di dichiarare una sequenza di CTE all'interno della medesima clausola **WITH**, separandole semplicemente con una virgola. La potenza di questo costrutto risiede nella **visibilità sequenziale**: una CTE dichiarata per seconda può interrogare direttamente la CTE dichiarata per prima, comportandosi a tutti gli effetti come una "Data Pipeline" (una catena di montaggio dei dati).

Se provassimo a replicare questo comportamento con le subquery classiche (Inline Views), saremmo costretti a nidificare le query l'una dentro l'altra (creando il famigerato anti-pattern della "piramide del codice"), rendendo il debugging di fatto impossibile.

Esempio 6.7 (Data Pipeline: Analisi dei Sinistri Gravi). Nel dominio *SafeDrive InsurTech*, le regole di business stabiliscono che un sinistro può richiedere molteplici perizie (es. danni al veicolo, danni fisici, danni a terzi). Il management richiede l'anagrafica dei clienti coinvolti in "Sinistri Gravi", definendo come "grave" un sinistro in cui la somma totale delle perizie supera i 10.000 euro.

Risolviamo il problema spezzandolo in passaggi logici sequenziali tramite CTE concatenate: 1. Calcoliamo la somma delle perizie per ogni singolo sinistro. 2. Filtriamo solo i sinistri la cui somma supera la soglia. 3. Incrociamo i risultati isolati con la complessa gerarchia anagrafica.

```

1 WITH DanniPerSinistro AS (
2   -- Step 1: Aggregazione (collassa le N perizie di un sinistro in
3     SELECT id_sinistro, SUM(valore_stimato) AS totale_danni
4     FROM perizia
5     GROUP BY id_sinistro
6   ),
7   SinistriGravi AS (
8     -- Step 2: Filtraggio (Interroga DIRETTAMENTE la CTE precedente)
9     SELECT id_sinistro, totale_danni
10    FROM DanniPerSinistro
11    WHERE totale_danni > 10000
12  )
13  -- Step 3: Query Principale (Incrocio anagrafico)
14  SELECT c.nome, c.cognome, s.codice_sinistro, sg.totale_danni
15  FROM cliente c
16  JOIN veicolo v ON c.id_cliente = v.id_cliente
17  JOIN polizza p ON v.id_veicolo = p.id_veicolo
18  JOIN sinistro s ON p.numero_polizza = s.numero_polizza
19  JOIN SinistriGravi sg ON s.id_sinistro = sg.id_sinistro;

```

🔍 Ottimizzazione del Motore PostgreSQL

Dal punto di vista delle prestazioni fisiche, l'uso di CTE multiple non degrada l'efficienza. A partire dalla versione 12, l'ottimizzatore (Query Planner) di PostgreSQL esegue di default l'*Inlining* delle CTE: analizza l'intera catena di WITH e la "srotola" ricomponendola nella strategia di esecuzione a più basso costo, esattamente come farebbe per una vista. Il programmatore ottiene quindi il massimo della leggibilità senza pagare alcuna penalità in termini di tempi di esecuzione.

6.4.3 Cenni alle CTE Ricorsive (WITH RECURSIVE)

Nel mondo reale, i dati aziendali sono spesso organizzati in strutture gerarchiche o ad albero: l'organigramma dei dipendenti, la distinta base di un veicolo, o un sistema di categorie annidate. Estrarre un intero albero logico utilizzando l'SQL standard (o peggio, demandando il ciclo al codice Java tramite decine di query consecutive) risulta letale per le prestazioni a causa del famigerato problema del "N+1 query".

PostgreSQL risolve elegantemente l'esplorazione dei grafi introducendo il modificatore **RECURSIVE**. Una **CTE Ricorsiva** è una tabella temporanea che ha la capacità di richiamare (interrogare) se stessa, comportandosi come un ciclo **while** nativo all'interno del motore del database.

Anatomia della Ricorsione in SQL Una query **WITH RECURSIVE** è rigorosamente composta da tre elementi fondamentali, uniti dall'operatore **UNION ALL**:

1. **Il Termine Non Ricorsivo (Caso Base):** È la query di partenza. Seleziona il nodo (o i nodi) radice dell'albero da cui iniziare l'esplorazione.
2. **L'Operatore di Unione:** **UNION** o **UNION ALL** viene utilizzato per "incollare" i risultati di ogni iterazione a quelli precedenti.
3. **Il Termine Ricorsivo (Passo Induttivo):** È la query che richiama il nome della CTE stessa. Viene eseguita ripetutamente finché non restituisce un insieme vuoto (la condizione di uscita implicita).

Esempio 6.8 (Esplorazione Gerarchica: Rete di Vendita SafeDrive). In *SafeDrive Insur-Tech* la rete di vendita è strutturata in modo gerarchico: ogni agente assicurativo risponde a un capo area, che a sua volta è un agente. Lo schema prevede quindi una *Self-Join* (una chiave esterna che punta alla chiave primaria della stessa tabella):

AGENTE(id_agente, nome, id_responsabile*)

Il management richiede l'estrazione dell'intera catena di comando partendo dal Direttore Generale (**id_agente** = 1) fino all'ultimo collaboratore sul territorio, calcolando dinamicamente il livello di profondità gerarchica.

```

1 WITH RECURSIVE Organigramma AS (
2     -- 1. CASO BASE: Inizializziamo l'albero partendo dal
3     Direttore
4     SELECT id_agente, nome, id_responsabile, 1 AS livello
5     FROM agente
6     WHERE id_agente = 1
7
8     UNION ALL
9
10    -- 2. PASSO RICORSIVO: Troviamo i sottoposti della riga
11    precedente
12    SELECT a.id_agente, a.nome, a.id_responsabile, org.livello +
13    1
14    FROM agente a
15    JOIN Organigramma org ON a.id_responsabile = org.id_agente
16 )
17 -- 3. QUERY FINALE: Estraiamo l'albero completo ordinato per
18 grado
19 SELECT * FROM Organigramma ORDER BY livello, nome;
```

Il funzionamento interno (Query Execution): Inizialmente, la tabella temporanea **Organigramma** viene popolata unicamente con la riga del Direttore. Al passaggio successivo, la **JOIN** del termine ricorsivo prende quella riga e cerca tutti gli agenti il cui **id_responsabile** corrisponde all'ID del Direttore. Questi nuovi agenti (Livello 2) vengono accodati nella CTE tramite l'unione. Alla terza iterazione, la **JOIN** cercherà i sottoposti degli agenti di Livello 2. Questo processo a catena continua in modo del tutto automatico

finché la query non raggiunge la base della piramide (agenti senza sottoposti). Quando un'iterazione non trova più alcun record, la ricorsione si arresta in modo sicuro.

✓ Il Vantaggio Architetture per il Backend

L'utilizzo di `WITH RECURSIVE` rappresenta la *best practice* assoluta per la gestione delle gerarchie. Permette al backend (es. un servizio Spring Boot) di recuperare un intero albero logico con **una singola chiamata di rete**. Il codice Java si limiterà a ricevere un `ResultSet` piatto (la lista degli agenti) che potrà facilmente parsare in memoria per ricostruire l'albero di oggetti, risparmiando al database decine o centinaia di esecuzioni separate.

✓ UNION ALL vs UNION nella Ricorsione

Negli snippet e nella manualistica sulle CTE ricorsive applicate a gerarchie (come un organigramma o una distinta base), l'operatore di giunzione utilizzato è quasi sempre `UNION ALL` anziché il semplice `UNION`. Non si tratta di una scelta stilistica, ma di una regola aurea per l'ottimizzazione estrema delle prestazioni.

- **Assenza naturale di duplicati:** In un albero gerarchico puro (relazione 1:N), ogni nodo figlio (es. il sottoposto) risponde a un solo nodo padre (es. il manager). Di conseguenza, esplorando l'albero dall'alto verso il basso, è matematicamente impossibile che la query estragga la stessa riga più di una volta.
- **Il costo computazionale del DISTINCT implicito:** L'operatore `UNION` non si limita a incollare due insiemi, ma applica sempre un'operazione di `DISTINCT` per scartare le doppie occorrenze. Se venisse usato in una ricorsione, costringerebbe il motore di PostgreSQL a memorizzare, ordinare e confrontare l'intero set di dati accumulato **a ogni singola iterazione**. Usando `UNION ALL`, il motore si limita ad accodare brutalmente i nuovi record in RAM, garantendo un'esecuzione fulminea.

Quando usare UNION? L'uso di `UNION` classico è giustificato e necessario in una CTE ricorsiva *esclusivamente* quando si esplorano **grafi ciclici** (es. reti stradali con percorsi alternativi per la stessa destinazione, o reti sociali). In quel caso, l'eliminazione del duplicato in tempo reale diventa vitale per impedire alla ricorsione di viaggiare in tondo ed entrare in un loop infinito.

6.5 Logica Condizionale e Gestione delle Incognite

Nel classico flusso di sviluppo backend, si tende spesso a utilizzare il database come un mero archivio di lettura (scaricando i dati grezzi) per poi delegare la logica di business e la formattazione al codice Java tramite infiniti cicli `for` e costrutti `if-else`. Questo approccio, noto come anti-pattern *Data Fetching*, sovraccarica inutilmente la memoria RAM del server applicativo e le reti di comunicazione.

L'SQL moderno fornisce potenti costrutti condizionali che permettono di alterare, raggruppare o etichettare l'output riga per riga direttamente all'interno del motore relazionale. Il dato arriva al backend già "digerito" e pronto per essere consumato.

6.5.1 Il costrutto CASE WHEN (Semplice vs Ricercato)

L'espressione CASE è l'equivalente diretto dell'istruzione `switch-case` o degli `if-else` nidificati in Java. Essendo un'espressione logica e non un comando strutturale, può essere inserita ovunque sia ammesso un valore: nella clausola `SELECT` per generare nuove colonne "al volo", nella `WHERE` per applicare filtri dinamici, o nella `ORDER BY` per stabilire ordinamenti personalizzati.

PostgreSQL supporta due varianti sintattiche del costrutto:

1. CASE Semplice (Match di Uguaglianza) Si utilizza quando occorre confrontare una singola colonna contro una lista di valori esatti. È architetturalmente identico allo `switch` di Java.

Esempio 6.9 (Normalizzazione degli Status Aziendali). Nel database *SafeDrive*, lo stato di una polizza è archiviato a livello fisico come un singolo carattere (es. 'A' per Attiva, 'S' per Sospesa, 'E' per Scaduta/Expired). Il frontend web, tuttavia, richiede di visualizzare le etichette descrittive complete. Invece di mappare il dato scaricato in Java tramite un `Enum`, generiamo l'etichetta direttamente in SQL:

```
1 SELECT numero_polizza ,
2       -- il case crea una seconda colonna calcolata
3       CASE status_polizza
4         WHEN 'A' THEN 'Polizza Attiva'
5         WHEN 'S' THEN 'Polizza Sospesa'
6         WHEN 'E' THEN 'Polizza Scaduta'
7         ELSE 'Stato Sconosciuto'
8       -- la colonna viene rinominata
9       END AS descrizione_stato
10 FROM polizza;
```

2. CASE Ricercato (Predicati Complessi) Si utilizza quando le condizioni non sono semplici uguaglianze, ma richiedono operatori logici (es. `>`, `<`, `AND`, `OR`) o il coinvolgimento di più colonne contemporaneamente. È l'equivalente di una catena di `if - else if`.

Esempio 6.10 (Segmentazione del Rischio per il Pricing). Il motore di calcolo dei premi di *SafeDrive* deve categorizzare i veicoli in fasce di rischio basandosi contemporaneamente sull'età del conducente e sulla potenza del veicolo. Valutare queste condizioni sul database riduce drasticamente i dati in transito.

```
1 SELECT c.nome, v.targa,
2       CASE
3         WHEN c.eta < 25 AND v.cavalli > 150 THEN 'Rischio
4           Altissimo'
5         WHEN c.eta < 25 THEN 'Rischio Alto'
6         WHEN c.eta >= 25
7           AND v.alimentazione = 'Elettrica' THEN 'Green
8           Driver'
9         ELSE 'Rischio Standard'
10        END AS fascia_rischio
11 FROM cliente c
12 JOIN veicolo v ON c.id_cliente = v.id_cliente;
```

Osservazione 6.2 (Valutazione Cortocircuitata (Short-Circuit)). Esattamente come la JVM in Java, il motore di PostgreSQL applica al **CASE** una logica a *cortocircuito* (Short-Circuit Evaluation). Le condizioni vengono valutate rigorosamente dall'alto verso il basso. Non appena una clausola **WHEN** risulta vera (**TRUE**), il database restituisce il risultato corrispondente e **interrompe immediatamente** la valutazione delle condizioni successive per quella specifica riga, risparmiando cicli di CPU.

Oltre la clausola SELECT Essendo un'espressione logica che si risolve sempre in un valore scalare, il **CASE** può essere iniettato in quasi ogni punto di un'istruzione SQL, fornendo un controllo granulare sul comportamento del database.

I due scenari applicativi più potenti riguardano l'ordinamento customizzato e gli aggiornamenti massivi condizionali.

Esempio 6.11 (1. Ordinamento Customizzato nella clausola ORDER BY). Le logiche di business spesso richiedono ordinamenti che non sono né alfabetici né numerici. Nel dominio *SafeDrive*, supponiamo che il frontend debba mostrare una lista di sinistri, ma la direzione esige che i sinistri 'Under Review' (In revisione) appaiano sempre in cima alla lista, seguiti da quelli 'Open' (Aperti) e infine quelli 'Closed' (Chiusi). L'ordinamento alfabetico standard fallirebbe (metterebbe prima 'Closed'). Usiamo il **CASE** per assegnare un "peso" numerico invisibile a ogni stato:

```

1 SELECT codice_sinistro, data_apertura, status
2 FROM sinistro
3 ORDER BY
4     CASE status
5         WHEN 'Under Review' THEN 1
6         WHEN 'Open' THEN 2
7         WHEN 'Closed' THEN 3
8         ELSE 4
9     END,
10    data_apertura DESC; -- Ordinamento secondario

```

Esempio 6.12 (2. Aggiornamenti Massivi Condizionali (UPDATE)). Questa è una delle tecniche più performanti in assoluto. Supponiamo di dover applicare un rincaro annuale ai premi delle polizze: +10% per i veicoli a benzina e +5% per i diesel. L'approccio "Junior" (Data Fetching) prevederebbe di scaricare tutte le polizze in Java, iterarle con un **for**, fare gli **if** e lanciare migliaia di query **UPDATE** singole (intasando la rete). L'approccio "Senior" sposta la logica in un'unica transazione atomica direttamente sul DBMS:

```

1 UPDATE polizza
2 SET premio_annuale =
3     CASE
4         WHEN tipo_alimentazione = 'Benzina' THEN premio_annuale *
5             1.10
6         WHEN tipo_alimentazione = 'Diesel' THEN premio_annuale *
7             1.05
8         ELSE premio_annuale -- Nessuna variazione per gli altri
9     END
10 WHERE data_scadenza > CURRENT_DATE;

```

In una singola frazione di secondo, il database aggiorna milioni di righe applicando la logica di business corretta riga per riga, senza che un solo byte transiti verso il server Java.

6.5.2 Pivoting dei Dati (Trasposizione)

I database relazionali sono ottimizzati per memorizzare i dati in righe (formato *Long*). Tuttavia, i requisiti di business per la reportistica e le dashboard richiedono quasi sempre che i dati siano organizzati in colonne (formato *Wide*). Il processo di rotazione dei dati dalle righe alle colonne prende il nome di **Pivoting**.

Nel backend, l'anti-pattern più diffuso tra i developer consiste nell'estrarre i dati grezzi e scrivere complessi algoritmi in Java (spesso abusando delle **Stream** API e di **Map** annidate) per trasformarli in formato *Wide*. L'ingegneria del software moderna impone invece di delegare questa trasformazione al DBMS, combinando le funzioni di aggregazione (come **SUM** o **COUNT**) con il costrutto **CASE WHEN**.

Esempio 6.13 (Generazione di una Dashboard Sinistri). Il frontend di *SafeDrive* richiede una tabella riassuntiva che mostri, per ogni area geografica, il numero totale di sinistri suddivisi per stato.

Se usassimo un raggruppamento classico (**GROUP BY area, status**), otterremmo fino a tre righe per ogni area. Utilizzando la tecnica del pivoting, costringiamo il database a restituire **esattamente una riga per area**, con i dati già incolonnati e pronti per essere serializzati in JSON.

```

1 SELECT
2     area_geografica ,
3     COUNT(id_sinistro) AS totale_sinistri ,
4     SUM(CASE WHEN status = 'Open' THEN 1 ELSE 0 END) AS aperti ,
5     SUM(CASE WHEN status = 'Closed' THEN 1 ELSE 0 END) AS chiusi ,
6     SUM(CASE WHEN status = 'Under Review' THEN 1 ELSE 0 END)
7         AS in_revisione
8 FROM sinistro
9 GROUP BY area_geografica
10 ORDER BY area_geografica;
```

L'algoritmo interno: Per ogni *area_geografica*, il motore collassa le righe. La funzione **SUM** valuta il **CASE** come un "filtro interruttore": se lo stato della riga corrente corrisponde alla colonna in calcolo, il **CASE** restituisce 1 (che viene sommato), altrimenti restituisce 0 (neutro per l'addizione).

✓ Il dialetto PostgreSQL: La clausola **FILTER**

Per eseguire il pivoting, PostgreSQL implementa un'estensione dello standard SQL moderno tramite la clausola **FILTER**. Questa sintassi è semanticamente identica al **SUM(CASE...)** dal punto di vista delle prestazioni, ma risulta architetturealmente più pulita e leggibile:

```

1 SELECT
2     area_geografica ,
3     COUNT(id_sinistro) FILTER (WHERE status = 'Open') AS
4         aperti ,
5     COUNT(id_sinistro) FILTER (WHERE status = 'Closed') AS
6         chiusi
7 FROM sinistro
8 GROUP BY area_geografica;
```

✓ Questa è la *best practice* assoluta quando si opera su stack tecnologici limitati all'ecosistema PostgreSQL.

6.5.3 Funzioni di coalescenza: gestione dei valori NULL

L'algebra relazionale gestisce i dati mancanti tramite il marcatore speciale NULL (incognita). La criticità principale del NULL è la sua propagazione matematica: qualsiasi operazione aritmetica o concatenazione di stringhe che coinvolge un NULL restituisce inesorabilmente NULL.

Nello sviluppo backend, delegare la gestione dei NULL al codice Java è una pratica rischiosa: mappare un NULL proveniente dal database su un tipo primitivo (come `int` o `double`) provoca un'immediata `NullPointerException`, causando il crash dell'applicazione. Lo standard SQL fornisce due funzioni di coalescenza per disinnescare questo problema direttamente nel motore dati.

1. La funzione COALESCE (Fallback garantito) La funzione `COALESCE(valore1, valore2, ..., valoreN)` accetta una lista di argomenti e restituisce **il primo valore non nullo** che incontra scorrendo da sinistra verso destra. È l'equivalente elegante di una serie di `if (valore != null)` in Java.

Esempio 6.14 (Prevenzione della Propagazione del NULL nell'Aritmetica). Nel sistema *SafeDrive*, il calcolo del premio finale di una polizza prevede la somma del `premio_base` e di un eventuale `malus_aggiuntivo`. Se un cliente non ha mai fatto incidenti, la colonna `malus_aggiuntivo` sarà fisicamente NULL. Se eseguiamo la semplice somma `premio_base + malus_aggiuntivo`, tutti i clienti virtuosi otterrebbero un premio totale pari a NULL.

Disinnesciamo il problema fornendo uno "Zero matematico" di fallback:

```
1 SELECT
2     numero_polizza ,
3     premio_base ,
4     malus_aggiuntivo ,
5     (premio_base + COALESCE(malus_aggiuntivo, 0))
6                             AS premio_totale_da_pagare
7 FROM polizza;
```

In questo modo, il backend riceverà sempre un numero decimale valido, garantendo la stabilità del calcolo.

2. La funzione NULLIF (Anti-Divisione per Zero) La funzione `NULLIF(parametro1, parametro2)` ha un comportamento apparentemente controintuitivo: confronta i due parametri e, se sono uguali, restituisce NULL; se sono diversi, restituisce il `parametro1`. Il suo caso d'uso ingegneristico principale è uno solo, ma vitale: **prevenire l'errore di divisione per zero** (`DivisionByZeroException`).

Esempio 6.15 (Prevenzione della Divisione per Zero). La direzione richiede di calcolare il "Costo Medio per Sinistro" di ogni perito. La formula è

$$\text{somma_danni} / \text{numero_sinistri_periziati}$$

Se un perito è appena stato assunto e ha periziato 0 sinistri, l'operazione $X / 0$ manderà in crash l'esecuzione della query.

Utilizziamo `NULLIF` per trasformare lo zero in `NULL`. Poiché l'SQL ammette la divisione per `NULL` (restituendo semplicemente `NULL` senza andare in crash), salviamo l'esecuzione:

```
1 SELECT
2     p.nome_perito,
3     SUM(s.danno_stimato) AS totale_danni,
4     COUNT(s.id_sinistro) AS numero_sinistri,
5     -- NULLIF trasforma lo 0 del COUNT in NULL.
6     -- Il calcolo diventa "X / NULL", che restituisce un NULL
       sicuro.
7     SUM(s.danno_stimato) / NULLIF(COUNT(s.id_sinistro), 0) AS
       media_danno
8 FROM perito p
9 LEFT JOIN sinistro s ON p.id_perito = s.id_perito
10 GROUP BY p.nome_perito;
```

Il backend riceverà un valore nullo per i nuovi periti, che potrà facilmente gestire nell'interfaccia utente (mostrando, ad esempio, "N.D.").

6.6 Funzioni Analitiche (Window Functions)

L'introduzione delle *Window Functions* (Funzioni Finestra o Analitiche) rappresenta la più grande evoluzione dello standard SQL moderno. Queste funzioni permettono di eseguire calcoli complessi su un set di righe correlate alla riga corrente, risolvendo problemi architetturali che storicamente richiedevano centinaia di righe di codice imperativo lato backend (Java, Python) o l'uso di complesse *Stored Procedure*.

L'Anatomia di una Window Function: Aggregazione vs Finestra L'ostacolo principale nell'apprendimento delle funzioni analitiche è comprenderne la differenza rispetto alle classiche funzioni di aggregazione (`GROUP BY`).

- **Aggregazione Classica (`GROUP BY`):** Ha un comportamento "distruttivo". Collassa (fonde) righe multiple in un'unica riga di riepilogo. Le informazioni di dettaglio della singola riga originale vengono perse.
- **Funzione Finestra (`OVER`):** Ha un comportamento "conservativo". Esegue un calcolo aggregato su un gruppo di righe (la *finestra*), ma **preserva intatta ogni singola riga originale**, aggiungendo semplicemente il risultato del calcolo come una nuova colonna.

La clausola chiave che trasforma una normale funzione (`SUM`, `COUNT`, `AVG`) in una funzione analitica è la parola chiave `OVER()`, che definisce esattamente come costruire la "finestra" di righe su cui operare.

Esempio 6.16 (La differenza architetturale in SafeDrive). Il management richiede un report che mostri **ogni singolo sinistro** e, accanto ad esso, il **totale dei danni dell'area geografica** in cui è avvenuto, per permettere al frontend di calcolare l'incidenza percentuale del singolo evento sul totale dell'area.

Se usassimo un `GROUP BY area_geografica`, otterremmo il totale dell'area, ma perderemmo i dettagli del singolo sinistro (es. la targa, la data). Saremmo costretti a fare una query di aggregazione, salvarla in una CTE e poi metterla in `JOIN` con la tabella originale dei sinistri.

Usando una Window Function, il calcolo avviene *inline*:

```

1 SELECT
2     id_sinistro,
3     area_geografica,
4     danno_stimato,
5     -- Inizia la Window Function
6     SUM(danno_stimato) OVER (PARTITION BY area_geografica)
7                             AS totale_danni_area
8 FROM sinistro;

```

Il funzionamento interno: Il motore di PostgreSQL legge la prima riga (es. un sinistro a 'Nord'). Vede la clausola `OVER (PARTITION BY area_geografica)`, quindi apre temporaneamente una "finestra" logica che contiene tutti i sinistri dell'area 'Nord', ne fa la somma, la scrive nella colonna `totale_danni_area` e chiude la finestra. Passa alla seconda riga e ripete l'operazione, senza mai cancellare le righe originali.

6.6.1 L'Ordinamento interno alla Finestra (`ORDER BY`)

Se la clausola `PARTITION BY` definisce i confini "spaziali" della finestra (quali righe raggruppare), la clausola `ORDER BY` al suo interno ne definisce i confini temporali o sequenziali (in che ordine processarle).

L'aggiunta dell'`ORDER BY` all'interno della clausola `OVER()` altera radicalmente il comportamento delle classiche funzioni di aggregazione come `SUM()` o `COUNT()`. Invece di calcolare il totale statico dell'intera partizione riga per riga, il motore calcola un **totale progressivo** (*Running Total*), sommando il valore della riga corrente a quello di tutte le righe precedenti all'interno della partizione.

Esempio 6.17 (Il Totale Progressivo: Analisi Storica dei Sinistri). Nel dominio *Safe-Drive*, il dipartimento antifrode sta indagando su due clienti sospetti e vuole monitorare l'evoluzione temporale dei danni dichiarati da ciascuno. L'obiettivo è mostrare l'elenco cronologico dei sinistri e, di fianco, l'accumulo progressivo dei danni richiesti fino a quella specifica data, **azzerando il contatore** quando si passa da un cliente all'altro.

```

1 SELECT
2     id_cliente,
3     data_apertura,
4     codice_sinistro,
5     danno_stimato,
6     -- Window Function con partizionamento e ordinamento interno
7     SUM(danno_stimato) OVER (
8         PARTITION BY id_cliente
9         ORDER BY data_apertura ASC
10    ) AS totale_danni_progressivo
11 FROM sinistro
12 WHERE id_cliente IN (42, 88)
13 -- Ordinamento fisico per il Client (Backend Java)
14 ORDER BY id_cliente ASC, data_apertura ASC;

```

Il funzionamento del Query Planner:

1. Il `PARTITION BY` divide i dati in due blocchi separati: uno per il cliente 42 e uno per il cliente 88.

2. L'ORDER BY mette in fila i sinistri dal più vecchio al più recente *all'interno* di ogni blocco.
3. Alla prima riga del cliente 42 (es. danno 1.000€), il progressivo stampato è 1.000€.
4. Alla seconda riga (es. danno 500€), la funzione calcola la somma, restituendo 1.500€.
5. Quando il motore esaurisce i sinistri del cliente 42 e passa alla prima riga del cliente 88, **la finestra logica si chiude e si riapre**: il totale progressivo si azzerava e inizia a sommare da capo i danni del nuovo cliente.

Questo meccanismo risolve in modo elegante un problema che in Java richiederebbe mappe annidate e l'inizializzazione di variabili contatore esterne a un ciclo, azzerando il rischio di errori di calcolo nel backend.

Osservazione 6.3 (Omissione del Partizionamento). In SQL, l'uso dell'ORDER BY in una Window Function non rende obbligatorio il PARTITION BY. Se nel nostro esempio avessimo filtrato in partenza un singolo cliente (`WHERE id_cliente = 42`), l'intero set di risultati in memoria sarebbe già appartenuto unicamente a lui. In quel caso specifico, omettere il partizionamento e scrivere semplicemente `OVER (ORDER BY data_apertura ASC)` sarebbe stato non solo valido, ma sufficiente per ottenere il totale progressivo corretto, trattando l'intera tabella come un'unica grande partizione.

⚠ L'Anti-pattern dell'Ordinamento Implicito

È un errore comune presumere che l'ordinamento definito all'interno di una Window Function determini l'ordine finale di output della query. L'ORDER BY dentro la clausola OVER regola esclusivamente il comportamento della funzione. Se l'applicazione client necessita di dati ordinati per la visualizzazione o l'impaginazione, è tassativo dichiarare un ORDER BY globale alla fine dell'istruzione SQL, come abbiamo fatto sopra.

⚠ Attenzione al Comportamento di Default

È fondamentale ricordare una regola silente dello standard SQL: quando si omette l'ORDER BY in una Window Function, la finestra comprende implicitamente *tutte* le righe della partizione contemporaneamente. Quando invece si inserisce l'ORDER BY, la finestra viene automaticamente limitata dalla regola: "dalla prima riga della partizione fino alla riga corrente". Questo concetto avanzato, che permette di definire in modo millimetrico l'ampiezza della finestra di calcolo, prende il nome di **Framing** e verrà affrontato nel paragrafo successivo.

6.6.2 Il Framing (ROWS BETWEEN / RANGE BETWEEN)

Mentre il PARTITION BY definisce i confini assoluti del gruppo di calcolo, il **Framing** permette di restringere ulteriormente la finestra logica, creando una sotto-finestra mobile (*sliding window*) che si sposta dinamicamente seguendo la riga corrente.

È lo strumento architetturale indispensabile per calcolare analitiche cumulative su finestre fisse, come le **medie mobili** o il controllo dei trend temporali ristretti a un numero specifico di eventi consecutivi.

Il Framing Fisico(RANGE BETWEEN) La sintassi fondamentale si inserisce dopo l' ORDER BY interno alla clausola OVER ed utilizza la seguente struttura:

ROWS BETWEEN <inizio> AND <fine>

I modificatori più utilizzati per stabilire i punti di inizio e fine sono:

- **CURRENT ROW**: La riga su cui il motore si trova in questo esatto momento.
- **N PRECEDING**: Le N righe fisiche precedenti alla riga corrente.
- **N FOLLOWING**: Le N righe fisiche successive alla riga corrente.
- **UNBOUNDED PRECEDING**: Dalla primissima riga della partizione.
- **UNBOUNDED FOLLOWING**: Fino all'ultimissima riga della partizione.

Esempio 6.18 (Media Mobile Antifrode: Finestra di Controllo Mobile). Nel sistema *SafeDrive*, gli analisti antifrode notano che una truffa comune prevede la denuncia di sinistri con danni via via crescenti in un breve lasso di tempo. Per intercettare questa anomalia, il backend deve calcolare la **media mobile del danno stimato sugli ultimi 3 sinistri consecutivi** di ogni veicolo (quello corrente e i due immediatamente precedenti).

Se provassimo a implementare questo algoritmo in Java, dovremmo gestire buffer circolari e code in memoria per ogni veicolo. In SQL si risolve in una riga:

```

1 SELECT
2     id_veicolo,
3     data_apertura,
4     codice_sinistro,
5     danno_stimato,
6     -- Calcolo della media mobile su finestra fissa di 3 elementi
7     AVG(danno_stimato) OVER (
8         PARTITION BY id_veicolo
9         ORDER BY data_apertura ASC
10        ROWS BETWEEN 2 PRECEDING AND CURRENT ROW
11    ) AS media_mobile_3_sinistri
12 FROM sinistro
13 ORDER BY id_veicolo, data_apertura;
```

Il funzionamento del Query Planner:

1. Il motore isola i sinistri dello stesso veicolo e li ordina per data.
2. Alla riga 1 del veicolo, non ci sono righe precedenti. La finestra contiene solo la riga corrente. La media mobile coincide con il danno singolo.
3. Alla riga 2, c'è una sola riga precedente. Il motore calcola la media tra la riga 1 e la riga 2.
4. Dalla riga 3 in poi, l'algoritmo entra a pieno regime: la sotto-finestra si blocca sulla dimensione fissa di 3 righe (la corrente e le 2 precedenti), scartando i sinistri ancora più vecchi. Il backend ottiene un indicatore di trend pulito e aggiornato riga per riga.

Il Framing Logico (ROWS BETWEEN) Mentre la clausola **ROWS** definisce una finestra basata esclusivamente sul conteggio fisico delle tuple, la clausola **RANGE** costruisce una finestra basata sul **valore logico** assunto dalla colonna di ordinamento.

Questa distinzione architetturale diventa vitale quando si analizzano serie temporali che presentano "buchi" (giorni senza eventi) o "pareggi" (eventi multipli verificatisi nello stesso istante). In questi scenari, estrarre "le ultime 3 righe" produrrebbe risultati fuorvianti e statisticamente invalidi. Utilizzando **RANGE**, il calcolo si svincola dalla struttura fisica della tabella e si aggancia al dominio del tempo o della matematica pura.

Esempio 6.19 (Analisi di Rischio: Accumulo Danni a 30 Giorni). Nel dipartimento *Pricing* di *SafeDrive*, gli attuari vogliono valutare la frequenza a breve termine dei sinistri per ricalcolare il premio delle polizze. Per ogni sinistro denunciato, il sistema deve calcolare la somma totale dei danni richiesti da quello stesso cliente **esattamente nei 30 giorni precedenti** all'evento corrente.

In Java, questo richiederebbe l'estrazione di tutta la storia del cliente e l'utilizzo di librerie come `java.time` per calcolare le durate ciclando all'indietro. In SQL, la logica temporale è nativa:

```

1 SELECT
2     id_cliente ,
3     data_apertura ,
4     codice_sinistro ,
5     danno_stimato ,
6     -- Finestra temporale logica dinamica
7     SUM(danno_stimato) OVER (
8         PARTITION BY id_cliente
9         ORDER BY data_apertura ASC
10        RANGE BETWEEN INTERVAL '30' DAY PRECEDING AND CURRENT ROW
11    ) AS accumulo_danni_30_giorni
12 FROM sinistro
13 ORDER BY id_cliente , data_apertura;
```

Il funzionamento del Query Planner (Valutazione Logica):

1. Il motore isola il cliente e ordina i sinistri cronologicamente.
2. Quando analizza la riga corrente (es. un sinistro avvenuto il 15 Novembre), la finestra logica non cerca un numero fisso di righe. Calcola invece un *offset* matematico: il confine inferiore della finestra diventa il 16 Ottobre (15 Novembre - INTERVAL '30' DAY).
3. Il motore include nella somma **tutte e sole** le righe fisiche la cui `data_apertura` ricade tra il 16 Ottobre e il 15 Novembre.
4. Se in quella finestra di 30 giorni il cliente ha avuto 0 incidenti (oltre a quello in corso), la somma coinciderà con il danno attuale. Se ne ha avuti 50, il motore li sommerà tutti. Il conteggio fisico delle righe viene del tutto ignorato a favore del vincolo temporale.

6.6.3 Funzioni di Ranking (`ROW_NUMBER()`, `RANK()`, `DENSE_RANK()`)

Un requisito estremamente frequente nelle applicazioni aziendali è la generazione di classifiche o la selezione dei primi *N* record all'interno di una categoria (es. identificare i 3 clienti con il più alto indice di rischio per regione).

Svolgere questa operazione lato backend (Java) richiede il recupero di enormi dataset, l'ordinamento in memoria e la successiva potatura manuale delle liste, con un impatto devastante sulle prestazioni. L'SQL risolve questo problema nativamente attraverso le **Funzioni di Ranking**.

Queste funzioni necessitano tassativamente della clausola `ORDER BY` all'interno di `OVER()`, poiché non ha senso calcolare una posizione senza un criterio di ordinamento prestabilito.

PostgreSQL fornisce tre funzioni principali, che differiscono esclusivamente per il modo in cui gestiscono i casi di "pareggio" (righe con lo stesso identico valore di ordinamento).

- **ROW_NUMBER()**: Assegna un numero sequenziale univoco e incrementale a ogni riga, partendo da 1. In caso di pareggio, non si cura dell'uguaglianza e assegna numeri diversi in base all'ordine fisico in cui legge i dati. Non genera mai buchi nella sequenza.
- **RANK()**: Assegna la stessa posizione alle righe in pareggio. Tuttavia, quando la sequenza riprende, **salta tante posizioni quanti sono i duplicati** precedenti (comportamento "olimpico"). Genera buchi nella numerazione.
- **DENSE_RANK()**: Simile a **RANK()**, assegna la stessa posizione ai pareggi, ma alla ripresa della sequenza **non salta alcuna posizione**, procedendo con il numero immediatamente successivo (classifica "densa"). Non genera mai buchi.

Esempio 6.20 (Analisi Antifrode: Identificazione dei Sinistri Top per Area). Il reparto Audit di *SafeDrive* vuole estrarre una classifica dei sinistri all'interno di ciascuna area geografica, ordinandoli dal danno più alto a quello più basso, per evidenziare le anomalie. Alcuni sinistri presentano lo stesso identico valore di danno stimato.

Visualizziamo il comportamento delle tre funzioni a confronto:

```

1 SELECT
2     area_geografica,
3     codice_sinistro,
4     danno_stimato,
5     ROW_NUMBER()
6         OVER (PARTITION BY area_geografica ORDER BY danno_stimato
7              DESC)
8     AS row_num,
9     RANK()
10        OVER (PARTITION BY area_geografica ORDER BY danno_stimato
11             DESC)
12    AS rnk,
13    DENSE_RANK()
14        OVER (PARTITION BY area_geografica ORDER BY danno_stimato
15             DESC)
16    AS dense_rnk
17 FROM sinistro
18 ORDER BY area_geografica, danno_stimato DESC;
```

Simulazione del comportamento con valori in pareggio: Supponiamo che nell'area 'Centro' ci siano quattro sinistri con i seguenti danni: 5000, 4000, 4000, 2000. Il motore calcolerà le colonne di ranking in questo modo:

- **Danno 5000:** ROW_NUMBER = 1, RANK = 1, DENSE_RANK = 1
- **Danno 4000 (Primo):** ROW_NUMBER = 2, RANK = 2, DENSE_RANK = 2
- **Danno 4000 (Secondo):** ROW_NUMBER = 3, RANK = 2 (pareggio), DENSE_RANK = 2 (pareggio)
- **Danno 2000:** ROW_NUMBER = 4, RANK = 4 (ha saltato il 3), DENSE_RANK = 3 (nessun buco)

Osservazione 6.4 (L'Uso Ingegneristico: Limitare i risultati con le CTE). Le funzioni di ranking non possono essere inserite direttamente nella clausola **WHERE** della stessa query (perché la **WHERE** viene eseguita prima del calcolo della finestra). Per ottenere i "Primi 3 sinistri per area", l'architettura standard impone l'uso di una CTE per materializzare il ranking, filtrando successivamente nel blocco principale:

```

1 WITH ClassificaSinistri AS (
2     SELECT area_geografica, codice_sinistro, danno_stimato,
3           DENSE_RANK()
4           OVER (PARTITION BY area_geografica ORDER BY danno_stimato
5                 DESC)
6           AS posizione
7     FROM sinistro
8 )
9 SELECT * FROM ClassificaSinistri
10 WHERE posizione <= 3; -- Estrazione pulita dei primi 3 livelli di
11 danno

```

6.6.4 Funzioni Posizionali (LEAD() e LAG())

L'algebra relazionale valuta storicamente ogni tupla (riga) in modo indipendente. Confrontare il valore di una riga corrente con quello della riga precedente (es. per calcolare un delta temporale o una variazione percentuale) ha sempre rappresentato una sfida architetturale, tipicamente risolta tramite pesanti *Self-Join* o delegando il calcolo a cicli iterativi nel backend Java.

Le funzioni posizionali `LEAD()` e `LAG()` risolvono questo problema permettendo alla riga in corso di valutazione di accedere direttamente ai dati delle righe adiacenti all'interno della partizione definita, senza alterare la struttura del risultato.

- `LAG(colonna, offset, default)`: Guarda all'indietro. Restituisce il valore della colonna nella riga precedente (o di N righe precedenti, specificando l'offset).
- `LEAD(colonna, offset, default)`: Guarda in avanti. Restituisce il valore della colonna nella riga successiva (o di N righe successive).

Esempio 6.21 (Analisi Comportamentale: Distanza Temporale tra Sinistri). Il reparto Analisi Dati di *SafeDrive* vuole monitorare la frequenza degli incidenti. Per ogni sinistro denunciato, è necessario calcolare quanti giorni sono trascorsi dal sinistro **immediatamente precedente** dello stesso cliente. Una distanza temporale sospettosamente breve (es. meno di 10 giorni) potrebbe innescare un controllo antifrode automatico.

Invece di scaricare la lista e fare sottrazioni tra date in Java, estraiamo la data precedente con `LAG()` e demandiamo la sottrazione direttamente a PostgreSQL:

```

1 SELECT
2     id_cliente,
3     codice_sinistro,
4     data_apertura,
5     -- Estraiamo la data del sinistro precedente (offset = 1)
6     LAG(data_apertura, 1) OVER (
7         PARTITION BY id_cliente
8         ORDER BY data_apertura ASC
9     ) AS data_sinistro_precedente,
10
11     -- Calcoliamo direttamente il delta in giorni (PostgreSQL
12     date math)
13     data_apertura - LAG(data_apertura, 1) OVER (
14         PARTITION BY id_cliente
15         ORDER BY data_apertura ASC

```

```
15         ) AS giorni_dal_precedente
16
17 FROM sinistro
18 ORDER BY id_cliente, data_apertura ASC;
```

Il funzionamento del Query Planner:

1. Il motore isola il cliente e ordina i sinistri cronologicamente.
2. Alla prima riga (il primissimo sinistro del cliente), non esiste un evento precedente. La funzione `LAG` restituisce l'incognita `NULL`. Di conseguenza, la sottrazione matematica (`Data - NULL`) restituisce `NULL`, che è l'unico risultato logicamente corretto per il primo evento.
3. Alla riga successiva, `LAG` estrae la data della riga 1. Il database esegue la sottrazione e restituisce il numero intero di giorni trascorsi (es. 45).

Osservazione 6.5 (Gestione dei Valori di Default Nativi). Come evidenziato nell'esempio, la prima riga di una partizione restituirà `NULL` se interrogata tramite `LAG()` (e simmetricamente l'ultima riga restituirà `NULL` con `LEAD()`). Se la logica di business in Java non tollera i valori nulli, è possibile utilizzare il terzo parametro opzionale della funzione posizionale per fornire un *fallback* immediato, evitando di dover innestare una funzione `COALESCE` esterna:

```
1 -- Se non c'è un sinistro precedente,
2 -- fingiamo sia avvenuto 1000 giorni fa
3 LAG(data_apertura, 1, CURRENT_DATE - 1000) OVER (...)
```

Questo approccio garantisce la ricezione di dati robusti e tipizzati lato backend, azzerando il rischio di `NullPointerException`.

Ottimizzazione, Prestazioni, Scalabilità

Nei capitoli precedenti abbiamo trattato il database relazionale come un'entità logica in grado di restituire i dati interrogati. Tuttavia, nell'ingegneria del software su larga scala, questa astrazione non è più sufficiente. Quando le tabelle passano da poche migliaia a decine di milioni di righe, un'interrogazione scritta in modo superficiale può causare la saturazione delle risorse e il blocco dell'intero sistema.

Questo capitolo si discosta dalla sintassi SQL per esplorare l'architettura fisica dello storage. Comprendere come il Database Management System (DBMS) organizza, indicizza e recupera i byte sul disco rigido o sull'SSD è il requisito fondamentale per poter progettare servizi backend realmente scalabili e per affrontare con successo i colloqui tecnici di livello avanzato.

7.1 L'Anatomia dello Storage

Per comprendere la necessità delle complesse strutture dati utilizzate dai database relazionali moderni, è didatticamente utile partire dall'analisi della forma più elementare possibile di memorizzazione dei dati.

Il Database Primitivo: Il Log Append-Only Immaginiamo di voler costruire il nostro DBMS personale da zero. La struttura di memorizzazione più semplice in assoluto è un normale file di testo in cui ogni riga rappresenta un record (es. un file testuale separato da virgole).

Quando un'applicazione backend invia un comando di `INSERT`, il nostro database rudimentale si limita ad aprire il file e ad accodare la nuova stringa di testo alla fine (operazione di *append*).

L'accodamento sequenziale (*Append-Only Log*) è in assoluto l'operazione di scrittura più veloce che un supporto di memoria fisico possa eseguire. Non richiedendo alcuna riorganizzazione dei dati preesistenti né complessi posizionamenti della testina del disco, il costo computazionale di un inserimento è minimo e rigorosamente costante nel tempo, denotato in notazione asintotica come $O(1)$.

Il collasso architetturale di questo modello primitivo si manifesta inesorabilmente in fase di lettura. Quando l'applicazione invia una query di estrazione (es. `SELECT * FROM log WHERE id = 12345`), il database primitivo non ha alcuna informazione su dove si trovi fisicamente quel record.

L'unica strategia algoritmica possibile per il motore è aprire il file dalla prima riga e scandirlo integralmente verso il basso finché non trova una corrispondenza. Questa operazione prende il nome di **Ricerca Lineare** (*Sequential Scan*). Se il file è cresciuto fino a contenere 10 milioni di righe, il database potrebbe dover leggere 10 milioni di stringhe per trovare l'unica tupla richiesta. L'algoritmo ha un costo computazionale proporzionale alla mole dei dati, ovvero $O(N)$. All'aumentare del database, il tempo di risposta della singola query degrada in modo lineare, portando in breve tempo il backend al collasso prestazionale (il cosiddetto *Timeout*).

Il Compromesso Fondamentale del Software Engineering Per evitare il disastroso costo della ricerca lineare $O(N)$, l'ingegneria dei dati adotta la stessa identica soluzione utilizzata nell'editoria stampata per trovare un concetto in un libro di mille pagine: si crea un **Indice**.

Un indice, all'interno di un database, è una struttura dati aggiuntiva e fisicamente separata dai dati primari. Il suo unico scopo è mantenere una mappa ordinata (una "segnaletica") che indichi al motore di ricerca l'esatto indirizzo di memoria (*offset*) di ciascun record. Invece di scansionare i dati, il DBMS interroga l'indice, trova le coordinate del dato, e lo recupera a colpo sicuro.

Sebbene l'idea appaia come la soluzione definitiva, la sua implementazione introduce nel sistema quello che Martin Kleppmann definisce il "compromesso fondamentale" (*Trade-off*) dei database. Mantenere una struttura dati aggiuntiva ha un costo architetturale severo. Sebbene un indice abbatta drasticamente le tempistiche di lettura, esso **degrada le prestazioni in scrittura**.

Ogni volta che l'applicazione esegue un'istruzione INSERT, UPDATE o DELETE, il DBMS non può più limitarsi ad accodare velocemente il dato nel file primario ($O(1)$), ma è fisicamente costretto a ri-calcolare, ri-bilanciare e aggiornare simultaneamente tutti gli indici associati a quella tabella affinché la mappa rimanga coerente.

Non esiste un database relazionale "ottimizzato per impostazione predefinita". La creazione degli indici è un'operazione manuale e rappresenta una precisa responsabilità dell'ingegnere software o del Database Administrator (DBA), il quale deve bilanciare l'infrastruttura basandosi sui reali *pattern* di interrogazione dell'applicazione.

La regola aurea della scalabilità afferma che: *"Un indice ben scelto velocizza enormemente le query di lettura, ma ogni singolo indice aggiunto rallenta proporzionalmente le operazioni di scrittura"*. Creare indici a tappeto su ogni colonna "per sicurezza" è un grave anti-pattern che rischia di paralizzare l'inserimento dei dati nel sistema.

7.2 Strutture Dati per l'Indicizzazione

Per evitare la scansione lineare, il DBMS deve organizzare gli indici tramite strutture dati avanzate. Sebbene esistano diversi approcci (come i log LSM utilizzati nei DB NoSQL), il mondo relazionale è dominato da decenni da un unico, indiscusso standard algoritmico.

7.2.1 Il motore universale relazionale: Il B-Tree (Balanced Tree)

Introdotta nei primi anni '70 e costantemente raffinata, il **B-Tree** (precisamente nella sua variante B^+ -Tree) è l'albero di ricerca bilanciato su cui poggia l'intera architettura di PostgreSQL.

A differenza degli alberi binari di ricerca studiati nei corsi teorici di algoritmi e strutture dati, il B-Tree è progettato specificamente per lavorare in simbiosi con l'hardware dei supporti di memoria fisici (Dischi Meccanici ed SSD).

1. L'organizzazione fisica sul disco: Le Pagine Il segreto del B-Tree risiede nella sua granularità. Mentre i log di testo crescono in modo indefinito, il B-Tree scompone l'intero database in blocchi di memoria a dimensione fissa, chiamati **Pagine** (o *Blocks*), tipicamente di 4 KB o 8 KB (in PostgreSQL il default è 8 KB). Il DBMS legge o scrive sul disco sempre una pagina intera alla volta.

La struttura si sviluppa in modo gerarchico e piramidale:

- **Nodo Radice (Root Node):** È la pagina iniziale dell'indice. Contiene una sequenza ordinata di chiavi e una serie di puntatori (indirizzi di memoria del disco) che rimandano alle pagine del livello inferiore.
- **Nodi Interni (Internal Nodes):** Pagine intermedie che fungono da "svincoli autostradali". Leggendo la chiave cercata, indirizzano il motore verso la pagina successiva, restringendo progressivamente il campo di ricerca.
- **Nodi Foglia (Leaf Nodes):** Sono le pagine situate alla base della piramide. Contengono le chiavi effettive e, accanto a ciascuna di esse, il puntatore fisico (*Tuple ID* o *offset*) che indica dove si trova la riga reale all'interno della tabella principale. Nelle varianti moderne, tutte le foglie sono collegate tra loro da puntatori orizzontali per velocizzare la scansione sequenziale ordinata.

Il numero di puntatori a figli che una singola pagina può ospitare prende il nome di **Branching Factor** (o *Fan-out*). Poiché una pagina di 8 KB può contenere centinaia di chiavi numeriche, il fattore di ramificazione è straordinariamente elevato (spesso superiore a 200–300).

Esempio 7.1 (Tracciamento di una ricerca puntuale nel B-Tree). Supponiamo che il backend Java esegua l'interrogazione `SELECT * FROM sinistro WHERE id_cliente = 83`. La colonna `id_cliente` è indicizzata tramite un B-Tree. Vediamo come il motore di PostgreSQL naviga fisicamente le pagine da 8 KB memorizzate sul disco per recuperare il record.

Fase 1: Ispezione del Nodo Radice (Pagina #100) Il motore carica in RAM la pagina di radice. Al suo interno trova una guida sintetica suddivisa in intervalli di chiavi:

- Chiavi < 50 → Punta alla Pagina #101
- Chiavi da 50 a 99 → Punta alla Pagina #102
- Chiavi ≥ 100 → Punta alla Pagina #103

Poiché il valore cercato è 83, il Query Planner scarta immediatamente le pagine #101 e #103 (e tutti i sotto-alberi ad esse collegati). Il motore esegue una lettura sul disco per caricare solo la **Pagina #102**.

Fase 2: Navigazione del Nodo Interno (Pagina #102) La Pagina #102 è un nodo intermedio (lo svincolo). Al suo interno la granularità aumenta e l'intervallo viene ulteriormente frammentato:

- Chiavi da 50 a 69 → Punta alla Pagina #201
- Chiavi da 70 a 89 → Punta alla Pagina #202
- Chiavi da 90 a 99 → Punta alla Pagina #203

Il valore 83 ricade nell'intervallo centrale. Il motore ignora le pagine #201 e #203 ed effettua un secondo accesso al disco per caricare la **Pagina #202**.

Fase 3: Identificazione nel Nodo Foglia (Pagina #202) La Pagina #202 è un nodo foglia, la base della piramide. Qui non ci sono più puntatori ad altre pagine dell'indice, ma le chiavi reali associate agli indirizzi fisici dei dati nella tabella (*Heap File*):

- Chiave 75 → Record memorizzato all'offset 0x0A2
- Chiave 83 → Record memorizzato all'offset 0x1F4
- Chiave 89 → Record memorizzato all'offset 0x33B

Il motore trova la corrispondenza esatta per la chiave 83 e ottiene il puntatore fisico 0x1F4.

Fase 4: Estrazione dal File dei Dati (Heap File) L'indice ha terminato il suo compito. Il database esegue l'ultimo accesso al disco puntando direttamente all'offset 0x1F4 della tabella `sinistro`, estrae la riga completa (targa, data, danno stimato) e la impacchetta nel *Result Set* da inviare sulla rete all'applicazione Java.

Grazie a un elevato *Branching Factor*, l'albero si sviluppa pochissimo in altezza (profondità) ma immensamente in larghezza. Un B-Tree a soli 4 livelli logici, con un fattore di ramificazione pari a 250, è matematicamente in grado di ospitare oltre 3.9 miliardi di record.

Cercare un ID specifico in un database da 4 miliardi di righe non richiede 4 miliardi di scansioni ($O(N)$), ma l'apertura di **sole 4 pagine sul disco**. Il costo computazionale è rigidamente logaritmico, denotato come $O(\log N)$. Le prestazioni rimangono stabili, fulminee e indipendenti dalla crescita del database.

2. Il comportamento in Scrittura: Il Page Split Il B-Tree viene definito "Bilanciato" perché l'algoritmo garantisce che tutte le pagine foglia si trovino sempre, in ogni istante di tempo, alla stessa identica distanza (profondità) dalla radice. Questo previene il degrado dell'albero.

Tuttavia, mantenere questo perfetto bilanciamento durante un'operazione di **INSERT** richiede un calcolo computazionale oneroso. Quando l'applicazione backend inserisce una nuova riga, il DBMS naviga il B-Tree per individuare la pagina foglia corretta in cui allocare la nuova chiave in ordine numerico.

A questo punto si aprono due scenari fisici:

1. **Spazio disponibile:** Se la pagina da 8 KB ha ancora byte liberi, la chiave viene semplicemente scritta nella pagina. Il costo è minimo.
2. **Pagina satura (Il Page Split):** Se la pagina è fisicamente piena, il motore non può semplicemente accodare il dato. Deve attivare l'algoritmo di **Page Split** (Scissione della Pagina): la pagina viene allocata *ex novo* sul disco, i dati vengono divisi a metà tra la vecchia e la nuova pagina, e la chiave mediana viene spinta verso l'alto nel nodo padre per aggiornare i puntatori.



Il collasso da Page Split a cascata

Se anche il nodo padre è saturo, il fenomeno del *Page Split* si propaga ricorsivamente verso l'alto, potendo arrivare a sdoppiare il nodo radice stesso e aumentando l'altezza dell'intero albero.

Durante un inserimento che causa un Page Split a cascata, il database deve effettuare molteplici operazioni di scrittura in punti diversi del disco rigido anziché un singolo accodamento rapido. Questo spiega sperimentalmente perché le **INSERT** massive rallentino drasticamente in presenza di troppi indici attivi.

7.2.2 Velocità puntuale: Gli Indici Hash

Mentre il B-Tree domina il mondo relazionale per la sua straordinaria versatilità, esiste un'architettura di indicizzazione alternativa progettata per massimizzare la velocità assoluta in scenari estremamente specifici: l'**Indice Hash**.

Basato sulla celebre struttura dati della *Hash Table* (Tabella Hash), questo indice abbandona completamente l'idea di navigare un albero strutturato pagina per pagina. Al suo posto, utilizza una **Funzione di Hash**: un algoritmo che prende in input la chiave di ricerca (es. la *matricola* di uno studente) e la trasforma istantaneamente in un indirizzo di memoria (un *offset* o un *bucket*) che punta direttamente alla posizione del record sul disco o nella RAM.

Poiché l'indirizzo viene calcolato matematicamente e non cercato fisicamente tramite salti tra le pagine, l'indice Hash evita la fase di navigazione a strati del B-Tree. Il costo computazionale per recuperare un record tramite un indice Hash è **tempo medio costante** ($O(1)$). Che il database contenga dieci righe o dieci miliardi di righe, il tempo medio per decodificare la chiave e trovare il record esatto rimane identico e pressoché istantaneo.

Se le prestazioni per le ricerche puntuali (*Point Queries*, es. `SELECT * FROM studente WHERE matricola = 10001`) sono matematicamente inarrivabili per qualsiasi altra struttura, per quale motivo i database relazionali come PostgreSQL utilizzano il B-Tree come impostazione di default?

La risposta risiede nel limite strutturale e insuperabile delle funzioni di Hash: **la distruzione dell'ordine naturale dei dati**.

Per definizione crittografica e matematica, una buona funzione di Hash distribuisce i valori in modo apparentemente casuale e uniforme per evitare collisioni di memoria. Se abbiamo due matricole consecutive come 10001 e 10002, i loro hash genereranno locazioni di memoria completamente diverse e fisicamente distanti tra loro.

Questo significa che l'indice Hash non ha la benché minima idea concettuale di cosa significhi "maggiore di", "minore di" o "vicino a". Se un'applicazione backend esegue una *Range Query* (un'interrogazione a intervalli, es. `SELECT * FROM studente WHERE dataN BETWEEN '1999-01-01' AND '2000-12-31'`), l'indice Hash risulta **completamente inutile**. Il motore del database, incapace di sfruttare l'indice per definire un perimetro logico, sarà costretto a declassare l'operazione a una letale ricerca lineare su tutto il disco (*Sequential Scan*).

Nell'ingegneria del backend moderno, definire esplicitamente degli indici Hash all'interno di un database relazionale (es. `CREATE INDEX idx_nome ON tabella USING HASH (colonna);`) è una pratica rara. Vengono presi in considerazione solo se l'architetto del software ha la certezza matematica che quella specifica colonna verrà interrogata **esclusivamente tramite operatori di uguaglianza** (=) e mai tramite operatori di disuguaglianza (>, <, BETWEEN) o direttive di ordinamento (ORDER BY).

Il paradigma Hash trova invece la sua massima e naturale espressione nei database NoSQL in-memory di tipo *Key-Value Store* (come **Redis** o **Memcached**), dove le query ad intervalli non sono strutturalmente previste dal motore di archiviazione e l'obiettivo primario è la velocità di lettura assoluta in cache.

7.3 Strategie di Indicizzazione: Clustered vs Non-Clustered Index

Conoscere il funzionamento logaritmico $O(\log N)$ di un B-Tree è solo il primo passo. L'ingegneria del database richiede di comprendere come l'albero si leghi fisicamente ai dati reali scritti sul disco. La distinzione fondamentale riguarda il contenuto delle "foglie" dell'albero: contengono i dati veri e propri o solo delle coordinate?

Quando l'applicazione backend inserisce una nuova riga, il motore relazionale deve decidere dove scriverla fisicamente sull'hard disk. Esistono due architetture principali per la gestione di questo salvataggio:

1. Il File di Heap (Heap File) e gli Indici Secondari In questa architettura (che rappresenta il comportamento **di default in PostgreSQL**), il motore del database sceglie deliberatamente di separare i "dati reali" dalle "mappe" utilizzate per cercarli, al fine di ottimizzare le scritture.

La tabella vera e propria viene salvata fisicamente in un *Heap File* (letteralmente "file a mucchio"). Si tratta di una struttura dati non ordinata: quando il backend applicativo esegue una `INSERT`, PostgreSQL non perde tempo a riordinare i gigabyte di dati già presenti sul disco rigido. Si limita ad accodare la nuova riga nel primo blocco di memoria libero disponibile. Questo garantisce inserimenti fulminei con un costo architetturale minimo ($O(1)$).

Tuttavia, estrarre un record da un mucchio disordinato richiederebbe una letale ricerca lineare ($O(N)$). Per risolvere il problema, l'indice (ad esempio quello costruito sulla Chiave Primaria) viene istanziato come un **file separato** contenente una struttura B-Tree. Questi indici che "puntano" a un file dati separato prendono il nome di **Indici Secondari (Non-Clustered Indexes)**.

Il file dell'indice è estremamente leggero: le foglie di questo albero non contengono gli attributi pesanti della riga, ma esclusivamente due elementi:

1. La chiave di ricerca ordinata.
2. Un puntatore assoluto chiamato **TID (Tuple Identifier)**, che funge da "indirizzo fisico". Il TID indica al database il numero esatto della pagina da caricare in RAM e l'offset (il byte di partenza) in cui si trova la riga all'interno del caotico *Heap File*.

Esempio 7.2 (Meccanica del Pointer Lookup nell'Heap File). Consideriamo la tabella **sinistro**, popolata da milioni di record (id, data, targa, grado di colpa, danno stimato). Quando l'applicazione esegue un'interrogazione mirata come `SELECT * FROM sinistro WHERE id_sinistro = 850`, il motore PostgreSQL non tocca immediatamente i dati reali. Esegue un'operazione in due fasi distinte:

- **Fase 1: Navigazione dell'Indice** ($O(\log N)$). Il motore attraversa rapidamente il B-Tree ordinato fino a trovare la foglia associata all'ID 850. Qui estrae il TID (es. "Blocco 45, Riga 12").
- **Fase 2: Salto verso l'Heap** ($O(1)$). Avendo l'indirizzo esatto, il motore esegue un accesso meccanico mirato, noto come *Pointer Lookup*. Salta direttamente al Blocco 45 dell'Heap File, estrae tutti i campi del sinistro e li restituisce al chiamante.

L'operazione completa risulta quindi ultra-performante, combinando la velocità di attraversamento logaritmico della mappa con l'estrazione a tempo costante dei dati grezzi dal mucchio. Tuttavia, ci sono anche degli svantaggi. Il termine **Non-Clustered** (non raggruppato) evidenzia il principale svantaggio architetturale di questo approccio: l'ordine logico della mappa (l'indice) e l'ordine fisico dei dati sul disco (l'Heap File) sono completamente slegati.

Sebbene il *Pointer Lookup* sia istantaneo per ricerche puntuali ($O(1)$), diventa un grave collo di bottiglia durante le interrogazioni a intervallo (*Range Queries*). Se nel nostro gestionale richiediamo tutti i sinistri di un intero anno, l'albero B-Tree individuerà rapidamente 10.000 TID logicamente contigui. Tuttavia, per estrarre i dati reali, il motore sarà costretto a eseguire 10.000 salti meccanici casuali (*Random Access*) verso blocchi fisici dispersi e frammentati nell'Heap File. Questa assenza di contiguità fisica rende la lettura massiva estremamente onerosa per il disco.

Tip

L'architettura a Heap File/indicizzazione non-clustered predilige le prestazioni in scrittura a discapito delle estrazioni massive.

2. Tabelle Organizzate a Indice (Index-Organized Tables / Clustered) In questa architettura (che è il default, ad esempio, nel motore InnoDB di MySQL), il file di Heap non esiste. La tabella è l'indice stesso. L'albero B-Tree viene costruito sulla Chiave Primaria e le sue foglie contengono nativamente l'intera riga dei dati (es. nome, cognome, indirizzo). Questo si definisce **Indice Clustered** (o Primario). Poiché i dati vivono direttamente nelle foglie dell'albero, la tabella è fisicamente costretta ad essere ordinata sul disco in base alla Chiave Primaria.

 Tip

L'architettura a Indice Clustered (Tabella Organizzata a Indice) massimizza le prestazioni nelle estrazioni massive grazie alla contiguità fisica dei record, ma penalizza severamente le scritture poiché costringe il motore a riordinare fisicamente i dati sul disco ad ogni inserimento.

7.3.1 Forzatura dell'Architettura: Il comando CLUSTER e il De-Clustering

Come analizzato, l'architettura nativa di PostgreSQL si basa intrinsecamente sul file di Heap (*Non-Clustered*). Tuttavia, in scenari applicativi dove le estrazioni massive (*Range Queries*) diventano il collo di bottiglia primario, è legittimo domandarsi se sia possibile forzare il motore a riorganizzare i dati in modo fisicamente contiguo, emulando i benefici di una tabella organizzata a indice.

La risposta ingegneristica è che **non è possibile creare un vero indice clustered permanente in PostgreSQL**, ma è consentito imporre un riordinamento fisico una tantum attraverso il comando **CLUSTER**.

```
1 -- Riorganizza fisicamente l'Heap File seguendo l'ordine dell'
   indice
2 CLUSTER nome_tabella USING nome_indice;
```

Quando il Database Administrator invoca questa direttiva, PostgreSQL crea una copia integrale della tabella, legge l'indice specificato (es. quello sulla data di apertura del sinistro) e riordina fisicamente tutte le righe sul disco rigido per farle coincidere con l'albero B-Tree. Al termine dell'operazione, la lettura massiva non subirà più il ritardo del *Random Access*, poiché i blocchi verranno letti sequenzialmente.

 La trappola del De-Clustering e del Table Lock

L'impiego del comando **CLUSTER** nei sistemi di produzione nasconde criticità architetturali severe:

1. **Effetto "One-Shot" e De-Clustering:** L'allineamento fisico non viene mantenuto attivamente dal motore. Non appena l'applicazione backend eseguirà nuove istruzioni di **INSERT** o **UPDATE**, PostgreSQL riprenderà ad accodare i record nel primo spazio libero dell'Heap File, distruggendo progressivamente l'ordine fisico. Per garantire prestazioni costanti, l'operazione dovrebbe essere schedulata periodicamente (es. tramite un *cron job* notturno).
2. **Access Exclusive Lock:** Durante la riorganizzazione, il database applica un blocco totale sulla tabella. L'applicazione Java non potrà né leggere né scrivere dati per l'intera durata del processo, causando inevitabilmente il collasso delle chiamate API (*Timeout*).

Motori Rigidi vs Motori Ibridi L'impossibilità di stravolgere la natura di PostgreSQL senza compromessi severi evidenzia una netta spaccatura nel panorama dei database relazionali:

- **Architetture Rigide:** Motori come PostgreSQL (vincolato all'Heap File) o MySQL/InnoDB (vincolato obbligatoriamente all'Indice Clustered) impongono il loro paradigma di memorizzazione. È responsabilità dello sviluppatore backend adattare le query e il carico di lavoro alla natura inalterabile del motore.
- **Architetture Ibride (Enterprise):** Nei sistemi commerciali di fascia alta (come *Microsoft SQL Server* o *Oracle*), la scelta dell'architettura fisica è delegata nativamente al software engineer. Tramite sintassi esplicita (es. `CREATE CLUSTERED INDEX` vs `CREATE NONCLUSTERED INDEX`), è possibile ottimizzare ogni singola tabella del dominio in base ai reali pattern di traffico previsti.

7.4 Uso degli Indici

7.4.1 Indicizzazione Automatica vs Manuale

Un errore architetturale estremamente comune nello sviluppo backend è l'errata assunzione che il Database Management System ottimizzi nativamente tutte le relazioni definite nello schema logico. Per progettare un livello di persistenza solido ed evitare il collasso in produzione, è vitale tracciare una linea netta tra ciò che il motore relazionale indicizza per la propria "sopravvivenza" e ciò che deve essere esplicitamente indicizzato dal programmatore.

1. Chiavi Primarie e Vincoli di Unicità (Indicizzazione Automatica) Tutti i principali database relazionali (inclusi PostgreSQL, MySQL e Oracle) istanziano in modo del tutto automatico e invisibile un indice (tipicamente un B-Tree) ogni volta che si definisce una colonna come `PRIMARY KEY` o vi si applica un vincolo `UNIQUE`.

Questa automazione non è una semplice "cortesia" del motore, ma un requisito computazionale inderogabile. Il vincolo di chiave primaria impone che non possano esistere due tuple con lo stesso identificatore esatto. Quando l'applicazione invia un'istruzione `INSERT`, il database deve accertarsi preventivamente che la nuova chiave non esista già. Se non disponesse di un indice ordinato da attraversare a costo logaritmico ($O(\log N)$), il motore sarebbe costretto a eseguire una disastrosa ricerca lineare ($O(N)$) sull'intero *Heap File* prima di ogni singolo inserimento. Di conseguenza, dichiarare esplicitamente un `CREATE INDEX` sulla chiave primaria in uno script DDL è un'operazione ridondante e attivamente ignorata dal DBMS.

2. Chiavi Esterne (Foreign Keys) e l'Indicizzazione Manuale Obbligatoria Il vero baratro prestazionale si nasconde nella gestione delle relazioni tra tabelle. Sebbene il database verifichi costantemente l'integrità referenziale (assicurandosi, ad esempio, che non si inserisca un figlio senza un padre valido), **il DBMS non crea mai automaticamente un indice su una Foreign Key.**

Esempio 7.3 (La trappola della Foreign Key non indicizzata). Consideriamo le entità fondamentali del dominio assicurativo sulle auto: `cliente` e `sinistro`. Supponiamo che la tabella `sinistro` possieda logicamente una Foreign Key (`id_cliente`) per tracciare a quale assicurato appartiene l'evento. Se il layer di persistenza in Java esegue un'operazione quotidiana, come il recupero dello storico sinistri di uno specifico cliente:

```
1 -- Recupero di tutti i sinistri del cliente con ID 83
```

```
2 SELECT * FROM sinistro WHERE id_cliente = 83;
```

Senza un intervento manuale dello sviluppatore, PostgreSQL si troverà sprovvisto di una mappa per la colonna `id_cliente`. Non avendo alternative architetturali, sarà fisicamente costretto a eseguire un letale *Sequential Scan*, leggendo in memoria i milioni di sinistri globali registrati sull'hardware per filtrare esclusivamente quelli associati al cliente 83.

3. Oltre le Chiavi Esterne: L'Indicizzazione guidata dai Pattern di Accesso

Sebbene l'omissione degli indici sulle Foreign Key rappresenti l'errore più sistematico, la necessità di indicizzazione manuale non si esaurisce affatto nella gestione delle relazioni. Il database è una macchina cieca: qualunque interrogazione richieda di filtrare o ordinare il dataset basandosi su una colonna sprovvista di mappa, scatenerà inevitabilmente un *Sequential Scan*.

Nell'ingegneria del software, gli indici B-Tree secondari devono essere istanziati proattivamente dall'ingegnere analizzando i **Pattern di Accesso** dell'applicazione, ovvero studiando quali query verranno eseguite più frequentemente dal backend. I tre contesti operativi più critici sono:

- **Ricerche Puntuali su Attributi di Business (Non Univoci):** Molti domini richiedono la ricerca rapida di un record tramite attributi descrittivi che non sono soggetti a vincoli di unicità. Nel contesto *SafeDrive*, se un operatore del call center cerca un cliente per *cognome* o filtra i sinistri per *città* di accadimento, senza un indice manuale su queste colonne il motore dovrà scansionare l'intero *Heap File*. (Nota architetturale: Attributi di business come il *codice_fiscale* o la *targa*, pur non essendo scelti come Chiave Primaria, nel mondo reale sono Chiavi Candidate e ricevono un vincolo *UNIQUE*. Essendo protetti da tale vincolo, godono già dell'indicizzazione automatica da parte del DBMS).
- **Estrazioni Temporali (Range Queries):** I gestionali lavorano molto frequentemente su finestre di tempo. Un *batch process* notturno scritto in Spring Boot potrebbe eseguire una query per estrarre tutte le polizze in scadenza nel mese successivo:

```
1 SELECT * FROM polizza WHERE data_scadenza BETWEEN '
    2026-07-01' AND '2026-07-31';
```

Senza un indice B-Tree dedicato alla colonna `data_scadenza`, questa interrogazione si trasforma in una scansione lineare disastrosa.

- **Ordinamento e Paginazione (Sorting):** Quando un'API REST restituisce i dati a un'interfaccia frontend, lo fa quasi sempre paginandoli e ordinandoli (es. "Mostra gli ultimi 50 sinistri aperti").

```
1 SELECT * FROM sinistro ORDER BY data_apertura DESC LIMIT
    50;
```

Se non esiste un indice pre-ordinato su `data_apertura`, il DBMS è costretto a caricare in memoria (o peggio, su disco) l'intera tabella dei sinistri, eseguire un oneroso algoritmo di ordinamento (*In-Memory Sort*) e infine restituire solo le prime 50 righe, sprecando enormi quantità di CPU.



La Selettività e il Paradosso dell'Indice Ignorato

Nell'applicare questi indici manuali, l'ingegnere deve prestare massima attenzione alla **selettività** della colonna (la percentuale di righe uniche rispetto al tota-

⚠ le). Creare un indice su una colonna di stato (es. `stato_sinistro` con valori "APERTO" o "CHIUSO") è spesso uno spreco di risorse. Se un'interrogazione richiede tutti i sinistri "CHIUSI" e questi rappresentano il 90% del database, il *Query Optimizer* calcolerà matematicamente che eseguire milioni di salti meccanici (*Pointer Lookup*) nell'albero risulta più oneroso che scansionare sequenzialmente il disco. L'indice verrà deliberatamente ignorato dal motore, risultando unicamente in un rallentamento delle operazioni di scrittura.

✓ La Regola d'Oro per gli Script DDL e Migrations

Mentre le chiavi primarie possono essere ignorate in fase di ottimizzazione esplicita, è **precisa e inderogabile responsabilità dell'ingegnere software** valutare e istanziare manualmente un Indice Secondario (B-Tree) su tutte le colonne che fungono da Chiave Esterna. Queste colonne costituiscono infatti la spina dorsale fisica su cui vengono calcolate le clausole JOIN e instradate le estrazioni gerarchiche del livello applicativo.

7.4.2 Indici Composti e la Regola del Prefisso Sinistro

Nel business reale, le interrogazioni filtrano raramente per un singolo attributo. Spesso le clausole WHERE combinano più colonne (es. `WHERE cognome = 'Rossi' AND citta = 'Roma'`). Per ottimizzare queste query, il DBMS permette di creare **Indici Composti** (*Multi-column indexes*), ovvero un singolo B-Tree che indicizza la concatenazione rigorosa di più attributi.

```
1 -- Creazione di un indice composto su due colonne in PostgreSQL
2 CREATE INDEX idx_cognome_nome ON studente (cognome, nome);
```

Il funzionamento di un indice composto è concettualmente identico all'elenco telefonico cartaceo: i dati sono ordinati **primariamente** per la prima colonna (`cognome`). A parità di cognome (es. tutti i "Rossi"), i record interni vengono ordinati **secondariamente** in base alla seconda colonna (`nome`).

⚠ La Regola del Prefisso Sinistro

L'ordine in cui si dichiarano le colonne durante la creazione di un indice composto è architetturalmente vincolante. Un indice composto è utilizzabile dal Query Planner **se e solo se** l'interrogazione filtra per la prima colonna dell'indice (o per un prefisso continuo letto rigorosamente da sinistra verso destra).

Data la presenza dell'indice `idx_cognome_nome(cognome, nome)`:

- `WHERE cognome = 'Rossi'`: **Veloce**. L'indice viene usato parzialmente. Il motore salta direttamente alla sezione dei "Rossi".
- `WHERE cognome = 'Rossi' AND nome = 'Mario'`: **Velocissimo**. L'indice viene sfruttato in modo ottimale.
- `WHERE nome = 'Mario'`: **Lento (Sequential Scan)**. L'indice è totalmente inutilizzabile. Senza conoscere prima il cognome, i nomi "Mario" sono sparsi casualmente in tutto l'elenco telefonico. Il database sarà costretto a scansionare fisicamente l'intera tabella.

✓ Design degli Indici Composti

Quando si progetta un indice multicolonna per un'applicazione backend, la colonna posizionata all'estrema sinistra (*Leading Column*) dovrebbe sempre essere l'attributo interrogato più di frequente in modo isolato, o quello con la **selettività maggiore** (l'attributo in grado di scartare matematicamente il maggior numero di righe fin dal primo filtraggio). Se la logica di business impone di cercare spesso gli utenti *esclusivamente* per nome, sarà architetturalmente obbligatorio istanziare un secondo indice separato e dedicato alla colonna **nome**.

7.4.3 Cheat Sheet: Quando creare un Indice (e quando evitarlo)

La regola aurea dell'ingegneria dei database impone che **ogni indice velocizza le letture ma penalizza le scritture**. Di conseguenza, l'indicizzazione non deve mai essere preventiva ("lo metto per sicurezza"), ma guidata dai reali pattern di accesso dell'applicazione.

Dove usare gli indici:

- **Chiavi Esterne (Foreign Keys): Obbligatorio.** Il DBMS non le indicizza in automatico. Senza un indice B-Tree sulle Foreign Keys, le operazioni di JOIN in SQL e la navigazione a oggetti dell'ORM (es. `cliente.getSinistri()` in Hibernate) causeranno disastrosi *Sequential Scans*.
- **Attributi di Ricerca Frequenti:** Colonne che compaiono costantemente nelle clausole WHERE per ricerche puntuali (es. `cognome` del cliente, `citta` del sinistro), purché non siano già protette da vincoli di unicità.
- **Colonne per Estrazioni Temporali (Range Queries):** Date o timestamp utilizzati per filtrare finestre temporali tramite operatori di disuguaglianza o BETWEEN (es. `data_scadenza_polizza`).
- **Colonne di Ordinamento (ORDER BY):** Attributi su cui vengono frequentemente ordinate o paginate le estrazioni (es. ultimi 50 sinistri per `data_apertura` DESC). L'indice evita al server l'onere computazionale di dover riordinare i dati in memoria (*In-Memory Sort*) prima di inviarli al client.

Dove NON usare gli indici:

- **Chiavi Primarie e Vincoli UNIQUE: Inutile.** Il motore relazionale genera e gestisce già automaticamente un albero B-Tree per queste colonne per poter far rispettare i vincoli di univocità. Un indice manuale sarebbe un duplicato ridondante.
- **Colonne a Bassa Selettività:** Attributi che contengono pochi valori distinti rispetto al numero totale di righe (es. un booleano `is_attivo`, il sesso, o uno `stato_sinistro` con soli valori "APERTO"/"CHIUSO"). L'ottimizzatore ignorerà volontariamente l'indice preferendo il *Sequential Scan* per evitare il costo meccanico dei *Pointer Lookups*.
- **Tabelle ad Altissima Frequenza di Scrittura:** Se una tabella funge da log append-only puro (es. registrazione della telemetria IoT della scatola nera in tempo reale) e viene interrogata solo in caso di audit, l'aggiunta di indici degraderà severamente la latenza delle INSERT a causa dei continui *Page Split* dell'albero.

- **Violazione del Prefisso Sinistro (Indici Composti):** È inutile creare un indice composto (A, B, C) se l'applicativo filtra di frequente solo per la colonna B o C. Il DBMS non potrà utilizzare l'albero.

7.5 Ispezione Fisica delle Query: I Piani di Esecuzione (PostgreSQL)

Comprendere la teoria degli indici è il fondamento dell'ingegneria del software, ma nei sistemi di produzione la realtà fisica spesso diverge dalla teoria. Il motore del database potrebbe decidere, contro ogni nostra previsione, di ignorare un indice perfettamente costruito. Per diagnosticare i colli di bottiglia e validare le nostre scelte architetturali, PostgreSQL ci fornisce lo strumento di introspezione definitivo: il comando `EXPLAIN`.

7.5.1 Gli strumenti di diagnostica: `EXPLAIN` vs `EXPLAIN ANALYZE`

Quando inviamo una query, il *Query Planner* (il cervello matematico del database) valuta diversi algoritmi possibili per estrarre i dati e sceglie quello con il costo computazionale stimato minore. Questo piano di battaglia prende il nome di **Piano di Esecuzione** (*Execution Plan*).

- **`EXPLAIN`:** Antepoendo questa parola chiave a qualsiasi interrogazione, PostgreSQL non eseguirà fisicamente la query, ma restituirà una stringa di testo contenente la *stima* teorica del piano. Mostrerà quali indici intende usare, quante righe stima di dover attraversare e quale costo computazionale teorico ha calcolato.
- **`EXPLAIN ANALYZE`:** Questa è la direttiva di profilazione reale e inconfutabile. Il motore eseguirà fisicamente la query e restituirà un report comparativo: mostrerà la stima teorica a fianco del tempo di esecuzione effettivo (in millisecondi) e del numero di righe processate sul campo.

```

1 -- Richiesta del piano di esecuzione reale per un'estrazione
2 EXPLAIN ANALYZE
3 SELECT nome, cognome FROM studente WHERE citta = 'Roma';

```

Il pericolo nascosto di `EXPLAIN ANALYZE`

Poiché `EXPLAIN ANALYZE` esegue realmente le operazioni sull'hardware per misurarne le tempistiche, utilizzarlo su istruzioni DML come `UPDATE`, `INSERT` o `DELETE` causerà l'effettiva modifica irreversibile dei dati di produzione. Per ispezionare la reale esecuzione di un'operazione di scrittura distruttiva senza alterare lo stato del database, è obbligatorio avvolgerla in una transazione isolata (`BEGIN;`) seguita da un immediato annullamento (`ROLLBACK;`).

7.5.2 Leggere un Piano di Esecuzione e identificare i colli di bottiglia

L'output di un piano di esecuzione si legge dal basso verso l'alto (eseguendo un'analisi dai nodi foglia dell'albero logico fino all'estrazione finale). Analizzando il responso, l'ingegnere software deve ricercare specifiche parole chiave per determinare lo stato di salute dell'interrogazione:

1. Sequential Scan (Seq Scan) Indica che il database ha ignorato gli indici a disposizione e sta scansionando l'intero file fisico della tabella dall'inizio alla fine (Ricerca Lineare con complessità $O(N)$). *Diagnosi Oggettiva:* Se la tabella conta poche migliaia di righe, il Query Planner opta deliberatamente per il **Seq Scan** perché risulta meccanicamente più veloce caricare l'intero blocco in RAM piuttosto che sostenere il costo di un *Pointer Lookup*. Se invece questa dicitura compare su tabelle da milioni di record, è in corso un'emergenza prestazionale causata dall'assenza di un indice idoneo o da una violazione della regola del prefisso sinistro negli indici composti.

2. Index Scan Indica che il motore sta attraversando la struttura logaritmica del B-Tree ($O(\log N)$) e sta impiegando i puntatori per saltare nel file di Heap ed estrarre i blocchi di dati reali. *Diagnosi Oggettiva:* È l'esecuzione architetturale ottimale standard. La query risulterà scalabile, con latenze minime garantite all'aumentare dei volumi.

3. Index Only Scan (L'Esecuzione Perfetta) Costituisce l'operazione computazionale più efficiente offerta dal motore relazionale. Si verifica quando il *Query Planner* constata che **tutte** le colonne richieste dalla clausola **SELECT** sono già intrinsecamente memorizzate all'interno dei nodi dell'indice stesso. *Diagnosi Oggettiva:* Il database estrae i byte direttamente dalla struttura del B-Tree e confeziona il risultato ignorando completamente l'esistenza del file della tabella principale (*Heap File*). Questo approccio abbattere drasticamente le operazioni di input/output meccanico (I/O) sul disco.

✓ Il Covering Index

Se i pattern di carico applicativo prevedono milioni di invocazioni quotidiane per una specifica estrazione (es. il recupero di nome e cognome dato un identificativo primario), l'ingegnere può forzare artificialmente un **Index Only Scan** strutturando un **Covering Index** (Indice di Copertura).

Questo si realizza istanziando le colonne di filtro nell'indice principale e aggiungendo le colonne di *payload* come appendice ai nodi. In PostgreSQL si adopera la clausola **INCLUDE**:

```
1 CREATE INDEX idx_matr_copertura ON studente (matricola)
   INCLUDE (nome, cognome);
```

Questa operazione genererà ridondanza fisica sul disco (duplicando i valori di nome e cognome nell'indice), ma ottimizzerà le prestazioni di lettura in modo assoluto e inconfutabile.

Gestione delle Transazioni e Concorrenza

Fino a questo momento, abbiamo studiato il database all'interno di una campana di vetro. Abbiamo analizzato la modellazione dei dati e l'ottimizzazione fisica degli indici supponendo implicitamente che il nostro Database Management System (DBMS) servisse un unico utente alla volta, all'interno di un sistema hardware infallibile e perfetto.

La verità cruda dell'ingegneria del software, tuttavia, è che il mondo reale è un ambiente ostile e caotico. I server subiscono cali di tensione e si spengono improvvisamente, i dischi fissi si danneggiano, i cavi di rete perdono pacchetti e, soprattutto, un backend in produzione riceve centinaia o migliaia di richieste HTTP nello stesso identico millisecondo. Se due operatori tentano di modificare i dati dello stesso sinistro contemporaneamente, o se un'operazione di pagamento fallisce a metà a causa di un crash del server, il database rischia di corrompersi in modo irrimediabile, generando informazioni parziali, incoerenti e legalmente inaccettabili.

Questo capitolo abbandona l'ottimizzazione della singola interrogazione per esplorare lo scudo protettivo primario del database relazionale: la gestione della concorrenza e delle transazioni. Comprendere questi meccanismi è il requisito essenziale per poter utilizzare correttamente framework applicativi come Spring Boot o Hibernate senza causare disastri in produzione.

8.1 Fondamenti della Transazione e Proprietà ACID

Il problema fondamentale che un DBMS deve risolvere è la frammentazione delle operazioni. Nel codice dell'applicazione backend, un processo di business (es. "Rinnovo di una polizza") rappresenta un'unica azione logica per l'utente. Per il database, tuttavia, questa singola azione logica si traduce inevitabilmente in una molteplicità di operazioni fisiche separate (molteplici INSERT e UPDATE su tabelle diverse). Se un guasto hardware interrompesse l'esecuzione a metà di questa sequenza, il sistema si ritroverebbe in uno stato ibrido e invalido.

Per scongiurare questo scenario, i database relazionali introducono il concetto di **Transazione**.

8.1.1 Cos'è una transazione: il concetto di "Tutto o Niente" (Commit e Rollback)

Una **Transazione** è definita come *un'unità logica di lavoro non divisibile*, composta da una o più istruzioni SQL, che il database deve eseguire in modo inscindibile.

Il principio architetturale alla base della transazione è la regola del **"Tutto o Niente"** (*All-or-Nothing*). Il DBMS garantisce che:

- O **tutte** le operazioni fisiche incluse nella transazione vanno a buon fine e vengono salvate permanentemente sul disco.
- O, se si verifica un singolo errore in una qualsiasi delle operazioni (o un blocco hardware), **nessuna** delle operazioni precedenti produce effetti, e il database torna esattamente allo stato in cui si trovava un istante prima dell'inizio della transazione, come se nulla fosse mai accaduto.

Per controllare questo confine invisibile, il linguaggio SQL mette a disposizione tre comandi fondamentali che il programmatore (o il framework ORM per suo conto) deve orchestrare:

1. **BEGIN** (o **START TRANSACTION**): Segna l'inizio ufficiale dell'unità di lavoro. Da questo momento, tutte le istruzioni SQL vengono temporaneamente "annotare" ma non rese definitive per il resto del sistema.
2. **COMMIT**: È il comando di successo. Dichiarare che l'unità di lavoro è terminata senza errori. Il database prende le modifiche temporanee e le "scolpisce" permanentemente sul disco rigido, rendendole visibili a tutti gli altri utenti.
3. **ROLLBACK**: È il comando di emergenza (annullamento). Se un'istruzione fallisce o il backend Java intercetta un'eccezione, il comando **ROLLBACK** distrugge tutte le modifiche pendenti scritte dopo il **BEGIN**, ripristinando l'integrità totale del sistema.

Esempio 8.1 (Il collasso senza Transazioni nel dominio SafeDrive). Consideriamo il processo di "Emissione di una nuova Polizza" per un cliente esistente nel gestionale *SafeDrive*. La logica di business impone due operazioni sequenziali:

1. Inserire il nuovo contratto nella tabella **polizza**.
2. Aggiornare il veicolo del cliente nella tabella **veicolo**, impostando l'attributo **is_assicurato = TRUE**.

Se queste due operazioni venissero eseguite fuori da una transazione e, subito dopo il passo 1, il server del database esaurisse la memoria (Crash), la prima operazione rimarrebbe salvata sul disco, ma la seconda non verrebbe mai eseguita. Il database si ritroverebbe in uno stato *incoerente*: esisterebbe un contratto di polizza attivo e pagato, ma il veicolo risulterebbe legalmente "non assicurato" nei sistemi di controllo.

La soluzione transazionale: Avvolgendo il processo in una transazione:

```
1 BEGIN;  
2 INSERT INTO polizza (id_cliente, targa, data_inizio) VALUES (83,  
  'AB123CD', CURRENT_DATE);  
3 UPDATE veicolo SET is_assicurato = TRUE WHERE targa = 'AB123CD';  
4 COMMIT;
```

In caso di crash subito dopo l'INSERT, il comando COMMIT non verrebbe mai raggiunto. Al riavvio, il *Recovery Manager* del database eseguirebbe automaticamente un ROLLBACK d'ufficio, cancellando la polizza inserita a metà e salvando l'azienda da un'incoerenza fatale dei dati.

8.1.2 Le 4 Proprietà ACID: Il Contratto del Database

Quando il livello di persistenza in Java invia un comando di COMMIT, il database relazionale non si limita a un semplice tentativo di salvataggio. Firma un vero e proprio contratto con lo sviluppatore, garantendo quattro proprietà fondamentali racchiuse nell'acronimo **ACID**. Comprendere questo acronimo è obbligatorio, in quanto rappresenta una delle domande di sbarramento più frequenti in qualsiasi colloquio tecnico per ruoli backend.

A - Atomicità (Atomicity) È l'implementazione formale della regola del "tutto o niente" vista nel paragrafo precedente. L'Atomicità garantisce che le operazioni fisiche all'interno di una transazione siano indivisibili. Se il processo di "Registrazione Sinistro e Aggiornamento Massimale" prevede cinque query UPDATE distinte, e la quinta fallisce (es. per un errore di sintassi o una violazione di vincolo), il database distrugge automaticamente

i risultati delle prime quattro. Per il sistema, è come se la transazione non fosse mai iniziata. Non esistono stati intermedi.

C - Coerenza (Consistency) La transazione deve portare il database da uno stato di validità logica a un altro stato di validità logica, senza mai violare le regole strutturali definite nello schema (*DDL*). La verità ingegneristica è che il database non sa se il tuo codice Java sta calcolando il premio della polizza in modo giusto o sbagliato; lui si preoccupa solo dell'integrità referenziale.

- Se la tua transazione tenta di inserire un **sinistro** associato a un **id_cliente** che non esiste nella tabella madre (violazione di *Foreign Key*).
- Se tenta di inserire una **targa** già esistente (violazione *UNIQUE*).
- Se tenta di impostare un **danno_stimato** negativo (violazione *CHECK constraints*).

In tutti questi casi, la Coerenza viene compromessa. Il database intercetta la violazione, rifiuta l'istruzione e fa fallire l'intera transazione eseguendo un **ROLLBACK** forzato.

I - Isolamento (Isolation) [Il fulcro per il Backend Developer] Questa è la proprietà su cui nello sviluppo backend si passa la maggior parte del tempo a progettare e fare *debug*. L'Isolamento impone che l'esecuzione simultanea di due transazioni debba produrre lo stesso identico risultato che si otterrebbe eseguendole in serie (una dopo l'altra). Se nel gestionale due periti assicurativi aprono contemporaneamente lo stesso record del **sinistro** per aggiornare la stima del danno, l'Isolamento impedisce che le due operazioni si "intreccino" sovrascrivendosi a vicenda in modo imprevedibile. Mentre l'Atomicità, la Coerenza e la Durabilità sono concetti assoluti (o ci sono o non ci sono), l'Isolamento è un compromesso configurabile: sarai tu a dover dire a Spring Boot quanto forte deve essere questo isolamento, bilanciando l'integrità assoluta con le prestazioni del server (i cosiddetti *Isolation Levels* che studieremo nei prossimi paragrafi).

D - Durabilità (Durability) Una volta che il database ha accettato il **COMMIT** e ha restituito "OK" alla tua applicazione Java, la transazione è permanente e incancellabile. Anche se un millisecondo dopo il completamento qualcuno stacca fisicamente il cavo di alimentazione del server, al riavvio i dati saranno esattamente lì. Per ottenere questo, i DBMS non scrivono direttamente i dati finali nell'Heap File (che è un'operazione lenta), ma appuntano il cambiamento su un registro sequenziale ultra-veloce chiamato **WAL (Write-Ahead Log)**. Come sviluppatore Java, la Durabilità è un problema che deleghi totalmente all'hardware e ai sistemisti; devi solo sapere che esiste e che è il motivo per cui i database relazionali non perdono dati in caso di *crash*.



Riepilogo A.C.I.D.

la spiegazione migliore dell'acronimo ACID è pratica: L'Atomicità previene le operazioni a metà; la Coerenza previene i dati illegali; l'Isolamento previene il caos multi-utente; la Durabilità previene la perdita dei dati in caso di guasto hardware.

8.1.3 Il limite del singolo statement: perché una singola query non basta

Per comprendere la reale necessità del controllo transazionale esplicito (tramite **BEGIN**, **COMMIT** e **ROLLBACK**), è necessario chiarire il comportamento predefinito del database relazionale.

La verità meccanica è che **ogni singola istruzione SQL è già, di per sé, una transazione atomica**. Questo comportamento è noto come **Auto-Commit**. Se il backend invia una singola query come `UPDATE cliente SET email = 'nuova@email.com' WHERE id = 83`, il database garantisce che questa operazione non rimarrà mai a metà: o l'email viene aggiornata completamente, o, in caso di errore di rete durante la query, l'aggiornamento fallisce senza lasciare tracce.

Se il database è già atomico per natura, perché i programmatori Java devono preoccuparsi di gestire esplicitamente le transazioni? La risposta risiede nella **frammentazione del dominio di business**.

Nel mondo reale dell'ingegneria del software, i processi aziendali (i cosiddetti *Use Cases* o *User Stories*) non si risolvono quasi mai con l'alterazione di una singola riga in una singola tabella. Una logica di business significativa coinvolge sempre l'orchestrazione di molteplici entità relazionate tra loro.

Esempio 8.2 (Il disastro del Rinnovo Polizza senza Transazione Esplicita). Analizziamo il processo di "Rinnovo e Pagamento Polizza" nel gestionale *SafeDrive*. Quando il cliente paga il premio annuale, il livello di persistenza in Java deve tradurre questo processo aziendale in due istruzioni SQL distinte:

```
1 -- 1. Registrazione dell'incasso
2 INSERT INTO pagamento (id_polizza, importo, data_pagamento)
3 VALUES (1050, 450.00, CURRENT_DATE);
4
5 -- 2. Spostamento in avanti della scadenza
6 UPDATE polizza SET data_scadenza = '2027-06-13'
7 WHERE id_polizza = 1050;
```

Se ci affidassimo all'Auto-Commit predefinito, queste verrebbero trattate come **due transazioni atomiche separate e indipendenti**. Se un calo di tensione, un *Timeout* del server o un errore di validazione (es. violazione di una *Foreign Key*) facesse crashare il sistema **esattamente un millisecondo dopo** il completamento della prima query e prima dell'inizio della seconda, ci ritroveremmo in uno scenario catastrofico:

- L'azienda ha incassato e registrato 450 euro (la prima query ha fatto Auto-Commit).
- La polizza risulta ancora scaduta (la seconda query non è mai partita).

Il cliente ha pagato ma risulta non assicurato. Dal punto di vista del business e legale, i dati sono corrotti.

Il limite del singolo statement rende obbligatorio l'intervento del programmatore. Utilizzando una transazione esplicita, il backend Java ordina al database di sopprimere l'Auto-Commit e di trattare le molteplici operazioni fisiche (`INSERT` e `UPDATE`) come un singolo blocco logico, estendendo la protezione dell'Atomicità sull'intero processo di business.

8.2 Le Anomalie della Concorrenza

Nel paragrafo precedente abbiamo stabilito che la proprietà di **Isolamento** (la 'I' di ACID) ha il compito di impedire che le transazioni simultanee si influenzino a vicenda in modo imprevedibile. Tuttavia, a differenza dell'Atomicità o della Coerenza, l'Isolamento non è una proprietà "binaria" (attiva o spenta).

Per massimizzare le prestazioni, i database relazionali permettono ai programmatori di configurare *quanto* debba essere forte questo isolamento. Abbassare il livello di isolamento

migliora drasticamente la velocità del server (perché i dati non vengono "bloccati"), ma espone il sistema al rischio di comportamenti imprevisti.

Questi comportamenti patologici, che si verificano solo ed esclusivamente sotto carico concorrente, prendono il nome formale di **Anomalie della Concorrenza**. Conoscerle è vitale per poter scegliere la strategia di *Locking* corretta nel livello applicativo Java.

8.2.1 Lost Update (L'Aggiornamento Perduto)

L'Aggiornamento Perduto (*Lost Update*) è in assoluto l'anomalia più distruttiva e frequente nello sviluppo backend. Si verifica quando due transazioni indipendenti leggono contemporaneamente lo stesso record dal database, lo modificano in memoria locale (RAM) in base al valore appena letto, e infine tentano di sovrascriverlo sul disco una dopo l'altra.

La transazione che esegue il **COMMIT** per ultima sovrascrive ciecamente i dati salvati dalla prima transazione. Il lavoro della prima transazione viene letteralmente "perduto", cancellato per sempre dalla storia del sistema, senza che il database o l'applicativo segnalino alcun errore.

Esempio 8.3 (Il disastro del Lost Update nella liquidazione dei Sinistri). Immaginiamo che il gestionale *SafeDrive* debba decurtare il massimale residuo di una polizza a seguito di due sinistri indipendenti. La tabella **polizza** contiene il record per il Cliente 83: `massimale_residuo = 10.000 €`.

Due periti assicurativi (Perito A e Perito B) lavorano contemporaneamente su due sinistri diversi del Cliente 83, su due computer diversi.

1. **T1 (Perito A):** Legge dal DB il record della polizza. Il massimale in memoria è **10.000 €**.
2. **T2 (Perito B):** Nello stesso identico millisecondo, legge dal DB il record della polizza. Il massimale in memoria è **10.000 €**.
3. **T1 (Perito A):** Il sinistro ammonta a 2.000 €. Il codice Java del Perito A calcola il nuovo residuo ($10.000 - 2.000 = 8.000$) e invia l'**UPDATE**.
4. **T1 (Perito A):** Il database esegue il **COMMIT**. Sul disco ora c'è scritto **8.000 €**.
5. **T2 (Perito B):** Il sinistro ammonta a 3.000 €. Il codice Java del Perito B, che ricordiamo aveva letto il valore iniziale, calcola il nuovo residuo ($10.000 - 3.000 = 7.000$) e invia il suo **UPDATE**.
6. **T2 (Perito B):** Il database esegue il **COMMIT**. Sovrascrive gli 8.000 € del Perito A. Sul disco ora c'è scritto **7.000 €**.

Il Risultato Architetture: L'aggiornamento del Perito A è andato perduto. L'azienda ha erogato 5.000 € di risarcimenti totali ($2.000 + 3.000$), ma il massimale è sceso solo di 3.000 €. Dal punto di vista del business, l'azienda ha letteralmente perso 2.000 € a causa di una sovrascrittura cieca (Blind Write). Nessuna eccezione è stata generata, nessuna traccia è rimasta.

8.2.2 Dirty Read (Lettura Sporca)

Se il *Lost Update* rappresenta il pericolo di sovrascrivere dati validi, la **Lettura Sporca** (*Dirty Read*) rappresenta il pericolo di fidarsi di dati "fantasma" che non sono mai stati resi definitivi.

Questa anomalia si verifica quando una transazione (**T2**) è autorizzata a leggere le modifiche effettuate da un'altra transazione concorrente (**T1**), **prima** che quest'ultima abbia eseguito il comando di **COMMIT**. Se T1, per qualsiasi motivo (un errore di validazione,

un crash del server o una logica di business non soddisfatta), esegue un **ROLLBACK**, i dati letti da T2 diventano carta straccia: T2 sta basando la propria esecuzione su informazioni che, per il database, non sono mai legalmente esistite.

Esempio 8.4 (Il paradosso del pagamento basato su dati non confermati). Analizziamo una potenziale interazione nel sistema *SafeDrive* tra il backoffice e il portale clienti rivolto al pubblico.

1. **T1 (Operatore Backoffice)**: Inizia una transazione per ricalcolare in aumento il premio della polizza 1050 a causa di un sinistro con colpa. Esegue l'**UPDATE** portando il premio da **500 € a 800 €**. L'operatore non ha ancora confermato (**nessun COMMIT**).
2. **T2 (Cliente sul Sito Web)**: Nello stesso istante, il cliente accede alla sua area personale e clicca su "Paga Rinnovo". Il backend Java avvia la transazione T2, legge il premio attuale e, a causa del basso livello di isolamento, vede il dato "sporco": **800 €**.
3. **T2 (Cliente sul Sito Web)**: L'applicativo mostra 800 € a schermo e avvia la chiamata all'API della carta di credito per incassare la cifra.
4. **T1 (Operatore Backoffice)**: Il sistema di backoffice rileva che l'aumento è illegittimo (es. il sinistro era senza colpa) e lancia un'eccezione. La transazione T1 va in **ROLLBACK**. Il premio nel database torna istantaneamente a **500 €**.

Il Risultato Architettureale: La transazione T2 ha addebitato 800 € sulla carta di credito del cliente basandosi su una "Lettura Sporca". Il database risulta perfettamente coerente (la polizza costa 500 €), ma l'azienda ha incassato una cifra illegale e inesistente, basata sull'allucinazione temporanea di un'altra transazione fallita.

Per un backend developer, la Lettura Sporca è considerata un'anomalia inaccettabile nella quasi totalità dei domini applicativi (fanno eccezione solo alcuni sistemi di reportistica statistica, dove un errore dello 0.1% sui totali è tollerato pur di non bloccare le letture). Per questo motivo, la stragrande maggioranza dei DBMS moderni impedisce questa anomalia per impostazione predefinita.

8.2.3 Non-Repeatable Read (Lettura Non Ripetibile)

Mentre la Lettura Sporca riguarda dati "falsi" mai confermati, la **Lettura Non Ripetibile** (*Non-Repeatable Read*) riguarda dati assolutamente validi e confermati, ma che **mutano sotto i tuoi occhi** durante l'esecuzione della tua logica di business.

Questa anomalia si verifica quando la tua transazione (**T1**) legge la stessa identica riga del database due volte in momenti diversi della sua esecuzione. Nel lasso di tempo che intercorre tra la prima e la seconda lettura, un'altra transazione concorrente (**T2**) modifica quella stessa riga e fa un **COMMIT** con successo. Il risultato è che la tua transazione, pur avendo richiesto lo stesso dato, ottiene due valori diversi. La "ripetibilità" della lettura è stata compromessa.

Esempio 8.5 (Il report PDF con dati incoerenti). Nel gestionale *SafeDrive*, un'operazione *batch* notturna scritta in Java deve generare un report PDF riassuntivo per un cliente *Corporate* (Flotta Aziendale).

1. **T1 (Generatore PDF)**: Inizia la transazione. Esegue una prima **SELECT** per estrarre i dati del Cliente 83 e stampare l'intestazione del documento. Il premio totale annuale risulta **5.000 €**.

2. **T2 (Backoffice):** Nello stesso istante, un operatore lavorando in un fuso orario diverso approva uno sconto per il Cliente 83, aggiornando il premio totale a **4.500 €**. L'operatore esegue regolarmente il **COMMIT**.
3. **T1 (Generatore PDF):** T1 ha continuato a elaborare il corpo del documento. Arrivato alla fine dell'elaborazione (magari pochi decimi di secondo dopo), esegue una seconda **SELECT** sul Cliente 83 per stampare il riepilogo a piè di pagina. Trova il nuovo valore confermato da T2: **4.500 €**.

Il Risultato Architettureale: Il database ha agito in modo legalmente ineccepibile (T2 ha fatto un aggiornamento valido). Tuttavia, l'applicativo Java ha generato un documento ufficiale corrotto: l'intestazione riporta "Premio: 5.000 €", mentre il riepilogo finale dello stesso documento riporta "Totale: 4.500 €". Per il business, questo è un bug critico.

Osservazione 8.1 (La differenza con il Livello di Default). A differenza della Lettura Sporca, la Lettura Non Ripetibile è **un'anomalia perfettamente legale e attiva nel livello di isolamento predefinito di PostgreSQL e SQL Server (READ COMMITTED)**. Poiché bloccare un record per impedirne la modifica in lettura paralizzerebbe le prestazioni del sistema, il DBMS accetta il compromesso: ti garantisce che leggerai sempre dati veri e confermati, ma non ti garantisce che, se li rileggi un secondo dopo, saranno rimasti identici. Se la logica di business di un'API richiede coerenza assoluta tra due letture sequenziali, spetta al programmatore innalzare esplicitamente il livello di isolamento della transazione.

8.2.4 Phantom Read (Lettura Fantasma)

La **Lettura Fantasma** (*Phantom Read*) è l'anomalia più subdola in assoluto, perché si verifica anche quando il database ha protetto con successo tutte le righe che stai leggendo.

Mentre la Lettura Non Ripetibile riguarda la modifica di record *già esistenti*, la Lettura Fantasma riguarda l'alterazione di un **insieme di risultati**. Si verifica quando la tua transazione (**T1**) esegue una **SELECT** con una clausola **WHERE** (es. "estrai tutti i sinistri di Roma") e, prima che T1 finisca, una transazione concorrente (**T2**) esegue una **INSERT** o una **DELETE** che aggiunge o rimuove una riga che soddisfa proprio quella condizione. Se T1 ripete l'interrogazione, vedrà comparire dal nulla una riga "fantasma" (o ne vedrà sparire una), sballando completamente eventuali conteggi o aggregazioni in corso.

Esempio 8.6 (Lo sconto flotta e il veicolo fantasma). Il gestionale *SafeDrive* possiede una logica aziendale (scritta in Java) per i clienti *Corporate*: se un'azienda assicura 10 o più veicoli, scatta automaticamente uno sconto "Maxi-Flotta" del 20% sul totale della fattura.

1. **T1 (Fatturazione Mensile):** Il batch notturno in Java calcola la fattura per l'Azienda trasporti "Logistica SpA". Esegue: **SELECT COUNT(*) FROM veicolo WHERE id_cliente = 83**. Il risultato è **9 veicoli**.
2. **T1 (Fatturazione Mensile):** Java valuta la condizione: $9 < 10$. Niente sconto flotta.
3. **T2 (Operatore Concessionaria):** Nello stesso istante lavorativo, un concessionario convenzionato vende un nuovo furgone a "Logistica SpA". Esegue una **INSERT INTO veicolo** associata al cliente 83 e fa **COMMIT**.
4. **T1 (Fatturazione Mensile):** Il codice Java di T1 procede a generare le righe della fattura. Esegue: **SELECT * FROM veicolo WHERE id_cliente = 83** per calcolare il prezzo di ogni singolo mezzo e sommarlo. Questa volta, il database restituisce **10 veicoli** (il nuovo furgone fantasma è appena apparso).

Il Risultato Architeturale: Il sistema fattura 10 veicoli a prezzo pieno. Il cliente va su tutte le furie perché, avendo 10 veicoli assicurati, aveva legalmente diritto allo sconto del 20% sull'intera flotta. Il report ha agito in modo schizofrenico: ha applicato la logica decisionale su un insieme di 9 elementi, ma ha eseguito la matematica su un insieme di 10 elementi.

Perché la Lettura Fantasma è così difficile da sconfiggere per un DBMS? Come vedremo in dettaglio a breve, nel caso della Lettura Non Ripetibile, il database risolve il problema mettendo un lucchetto (*Row Lock*) sulle righe che stai leggendo, impedendo agli altri di modificarle. Ma nel caso della Lettura Fantasma, l'operatore **T2** sta inserendo una riga *nuova*. **Il database non può mettere un lucchetto fisico su una riga che sul disco ancora non esiste.**

Per prevenire i fantasmi, il motore relazionale è costretto a utilizzare meccanismi estremamente pesanti, come i **Range Locks** (bloccare l'intero indice o l'intera tabella per impedire qualsiasi inserimento in quell'intervallo). Questo livello di paranoia assoluta corrisponde al livello di isolamento massimo: **SERIALIZABLE**.

Riepilogo: le anomalie della Concorrenza

- **Lost Update:** tra modifiche concorrenti, una sovrascrive tutte le altre. È l'unica anomalia in scrittura ed è evitato di default da tutti i DBMS.
- **Dirty Read:** Leggi dati che non sono mai stati confermati.
- **Non-Repeatable Read:** Un dato che hai letto cambia prima che tu abbia finito.
- **Phantom Read:** L'insieme di righe su cui stai lavorando si allarga o si restringe a tua insaputa.

8.3 I Livelli di Isolamento

Il motore relazionale non impone un approccio assolutista alla concorrenza. Lo standard SQL definisce quattro **Livelli di Isolamento** (*Isolation Levels*), ovvero quattro configurazioni che il programmatore può impostare per bilanciare l'eterno compromesso dell'ingegneria dei dati: **Integrità Assoluta vs Prestazioni/Scalabilità**.

Aumentare il livello di isolamento significa chiedere al database di utilizzare meccanismi di *Locking* sempre più aggressivi (lucchetti sulle righe o sulle tabelle). Più lucchetti ci sono, più le transazioni degli altri utenti dovranno mettersi in coda ad aspettare, causando rallentamenti applicativi (*Latenza*) o, nei casi peggiori, il blocco totale del sistema (*Deadlock*).

Il *Lost Update* (Aggiornamento Perduto) sarà omesso nella trattazione a seguire: si tratta di un'anomalia di **scrittura**, non di lettura. Qualsiasi database moderno previene la sovrascrittura distruttiva già a partire dal primo livello **READ COMMITTED**, imponendo l'uso dei *Row Lock* esclusivi ogni volta che si esegue un'istruzione di **UPDATE**.

1. READ UNCOMMITTED (Prestazioni massime, integrità zero) È il livello più basso e anarchico. Il database non applica alcun lucchetto in lettura. Le transazioni leggono i dati alla massima velocità possibile, ma sono esposte a **tutte** le anomalie, inclusa la distruttiva *Dirty Read* (Lettura Sporca). Nell'architettura backend moderna non si usa quasi mai, se non per generare report statistici di massa dove un margine di errore è tollerato pur di non bloccare il database lavorativo. (*Nota: PostgreSQL considera questo*

livello talmente insicuro che lo ignora. Se lo si imposta, PostgreSQL applica silenziosamente il livello superiore).

2. READ COMMITTED (Il default di PostgreSQL e Oracle) È il livello di compromesso per eccellenza e rappresenta il comportamento predefinito della stragrande maggioranza dei database relazionali. Una transazione può leggere solo dati che sono stati ufficialmente confermati da un **COMMIT**.

- **Cosa previene:** Sconfigge la *Dirty Read*. Non leggerai mai "allucinazioni" o dati di transazioni fallite.
- **Cosa permette:** Lascia passare le *Non-Repeatable Reads* e le *Phantom Reads*. I dati che leggi sono veri, ma possono mutare o moltiplicarsi se li rileggi un istante dopo.

3. REPEATABLE READ (Il default di MySQL/InnoDB) Questo livello alza la barriera difensiva per garantire che, una volta letto un record, esso rimanga invariato per tutta la durata della tua transazione. Il database lo ottiene mettendo un lucchetto in lettura (*Read Lock*) sulle righe interrogate, oppure (nei DBMS più moderni) scattando una vera e propria "fotografia" invisibile dei dati al momento dell'inizio della transazione (*Snapshot Isolation / MVCC*).

- **Cosa previene:** Sconfigge le *Non-Repeatable Reads*. Se leggi la polizza 1050 dieci volte nel ciclo di vita di una stessa transazione, otterrai sempre lo stesso premio, anche se un collega lo ha appena modificato.
- **Cosa permette:** Storicamente, non protegge dalle *Phantom Reads*. Il database ha "congelato" le righe esistenti, ma un collega può ancora inserirne di nuove nel tuo perimetro di ricerca.

4. SERIALIZABLE (Integrità assoluta, crollo prestazionale) È il livello di paranoia massima. Il database garantisce che il risultato dell'esecuzione concorrente sia identico all'eseguire le transazioni in serie (una rigorosamente dopo l'altra). Per ottenere questo, il motore relazionale applica aggressivi **Range Locks**: se cerchi "i clienti di Roma", il database blocca in scrittura l'intera tabella (o l'indice) per la città di Roma, impedendo a chiunque di inserire, modificare o cancellare record in quel perimetro.

- **Cosa previene:** Sconfigge tutte le anomalie, inclusa la *Phantom Read*.
- **Lo svantaggio:** Rovina letteralmente la scalabilità orizzontale. Se applicato sistematicamente in un'API con molto traffico, il backend va in *Timeout* perché le transazioni passano il 90% del tempo in coda ad aspettare che le tabelle si sbloccino.

Tabella Riassuntiva: Livelli vs Anomalie La seguente matrice rappresenta lo standard SQL universale. Memorizzarla è uno step obbligato per chiunque voglia definire l'architettura di un livello di persistenza.

Livello di Isolamento	Dirty Read	Non-Repeatable Read	Phantom Read
READ UNCOMMITTED	Possibile	Possibile	Possibile
READ COMMITTED	Impedita	Possibile	Possibile
REPEATABLE READ	Impedita	Impedita	Possibile
SERIALIZABLE	Impedita	Impedita	Impedita

8.4 Lock Pessimistico vs Lock Ottimistico

I DBMS relazionali offrono garanzie matematiche sull'isolamento dei dati, ma i loro meccanismi nativi (i *Lock* fisici sulle righe o sulle tabelle) sono strumenti a "forza bruta", pensati per transazioni che durano frazioni di secondo. Quando l'interazione coinvolge i tempi di reazione umani (es. un utente che legge un modulo a schermo, compila i campi in 5 minuti e poi preme "Salva"), il backend Java non può mantenere aperta la transazione sul database per tutto quel tempo: il server crollerebbe al terzo utente connesso a causa dei troppi lucchetti attivi.

Per gestire la concorrenza in architetture *stateless* (come le moderne API REST), l'ingegneria del software ha standardizzato due pattern architetturali diametralmente opposti da implementare a livello applicativo.

8.4.1 Lock Pessimistico: La Forza Bruta (FOR UPDATE)

L'approccio pessimistico si basa sulla totale sfiducia logica: *"Sono certo che qualcuno cercherà di modificare questo dato mentre ci sto lavorando, quindi lo blocco preventivamente per tutti"*.

A livello SQL, questo si ottiene aggiungendo la clausola `FOR UPDATE` alla fine di una normale query di lettura:

```
1 SELECT * FROM sinistro WHERE id = 42 FOR UPDATE;
```

Quando il database riceve questa istruzione, applica un *Row Lock* esclusivo e immediato sulla riga del sinistro 42. Finché la tua transazione non esegue un `COMMIT` o un `ROLLBACK`, nessun altro utente nel sistema potrà modificare o leggere (se richiede a sua volta un lock) quel record.

L'Anti-Pattern nelle API REST

Nel mondo delle applicazioni web, il Lock Pessimistico è considerato quasi sempre un *anti-pattern*. Se un operatore apre la pratica di un sinistro (bloccando la riga) e poi va a prendersi un caffè dimenticandosi la schermata aperta, o se la connessione di rete cade lasciando la transazione "appesa", l'intero ufficio sinistri si paralizza. Il Lock Pessimistico va riservato esclusivamente a operazioni di *batch processing* notturno o transazioni finanziarie totalmente automatizzate e invisibili all'utente.

8.4.2 Lock Ottimistico: L'Astuzia del Versioning (Lo Standard)

L'approccio ottimistico si basa sulla statistica: *"È altamente improbabile che due operatori modifichino lo stesso esatto record nello stesso identico millisecondo. Quindi non blocco nulla, ma prima di salvare verifico che nessuno mi abbia preceduto"*.

Non essendoci alcun lucchetto fisico, le prestazioni del database sono massimizzate. Questo meccanismo, standard assoluto nell'ecosistema Spring Boot/Hibernate, si implementa aggiungendo una colonna tecnica in quasi tutte le tabelle del database, solitamente chiamata *versione* (di tipo numerico intero).

Esempio 8.7 (La meccanica del Lock Ottimistico e la prevenzione del Lost Update). Riprendiamo l'esempio del massimale della polizza 1050, conteso tra il Perito A e il Perito B che usano il gestionale *SafeDrive*.

1. **Lettura Concorrente:** Entrambi i periti interrogano la polizza tramite l'API. Il database restituisce i dati e `versione = 1`. Le transazioni di lettura si chiudono immediatamente (il DB è libero).
2. **Primo Aggiornamento (Perito A):** Il Perito A modifica il massimale e preme "Salva". Il livello di persistenza Java invia un `UPDATE` condizionale che incrementa la versione:

```
1 UPDATE polizza SET massimale = 8000, versione = 2
2 WHERE id = 1050 AND versione = 1;
```

La condizione `versione = 1` è vera. Il database aggiorna la riga e restituisce "**1 riga aggiornata**".

3. **La Trappola per il Perito B:** Il Perito B preme "Salva" pochi secondi dopo. Java invia il suo aggiornamento basato sui dati vecchi che aveva in memoria locale:

```
1 UPDATE polizza SET massimale = 7000, versione = 2
2 WHERE id = 1050 AND versione = 1;
```

4. **Il Blocco Logico:** Il database cerca la polizza 1050 con `versione = 1`, ma non la trova più (il Perito A l'ha appena portata a 2). La query non va in errore sintattico, ma il DBMS risponde all'applicativo: "**0 righe aggiornate**".

Quando il framework ORM rileva "0 righe aggiornate" su un aggiornamento per ID primario, capisce istantaneamente di aver subito una collisione concorrente. Esegue un `ROLLBACK` di sicurezza e lancia un'eccezione interna (`OptimisticLockException` in Java). L'API REST cattura l'eccezione e restituisce un errore HTTP 409 (*Conflict*) all'utente, segnalando che il record è stato modificato da un collega e invitandolo a ricaricare la pagina con i dati aggiornati.

8.4.3 Implementazione in JPA/Hibernate: L'annotazione `@Version`

Nel mondo dello sviluppo Java Enterprise (Spring Boot), la specifica JPA (*Java Persistence API*) e la sua implementazione più famosa (Hibernate) offrono un supporto nativo, trasparente e totalmente automatizzato per il Lock Ottimistico.

Invece di dover scrivere manualmente le interrogazioni SQL condizionali per controllare la versione del record, lo sviluppatore deve semplicemente aggiungere una proprietà tecnica alla sua classe *Entity* (il modello Java che mappa la tabella) e decorarla con l'annotazione `@Version`.

Esempio 8.8 (La mappatura dell'Entità Polizza in Java). Ecco come si presenta il codice reale dell'entità *Polizza* nel progetto *SafeDrive*:

```
1 import jakarta.persistence.Entity;
2 import jakarta.persistence.Id;
3 import jakarta.persistence.Version;
4 import java.math.BigDecimal;
5
6 @Entity
7 public class Polizza {
8
9     @Id
10     private Long id;
11 }
```

```
12     private BigDecimal massimaleResiduo;  
13  
14     // Il "Lucchetto Logico" automatizzato  
15     @Version  
16     private Integer versione;  
17  
18     // ... getter e setter ...  
19 }
```

Cosa succede "dietro le quinte" (La Magia dell'ORM): Quando il codice applicativo invoca il salvataggio dell'oggetto (`polizzaRepository.save(polizza)`), Hibernate intercetta la chiamata e compie tre azioni in totale autonomia:

1. Genera la query SQL inserendo automaticamente la clausola `WHERE id = ? AND versione = ?`.
2. Incrementa automaticamente di 1 il valore del campo `versione` da salvare sul database.
3. Analizza il numero di righe modificate restituito dal driver JDBC.

Se il driver del database risponde "0 righe aggiornate" (segno inequivocabile che un'altra transazione ha modificato il record nel frattempo), Hibernate interrompe immediatamente il processo ed esegue un `ROLLBACK` della transazione corrente.

Osservazione 8.2 (La gestione dell'Eccezione nel Controller). Quando si verifica una collisione concorrente, Hibernate lancia una `OptimisticLockException` (che Spring avvolge nella sua `ObjectOptimisticLockingFailureException`). Il compito del programmatore backend è catturare questa eccezione a livello di Controller REST (spesso utilizzando un `@RestControllerAdvice` globale) e tradurla in una risposta HTTP standardizzata per il client (Front-end o App Mobile). La prassi architetturale impone di restituire lo status code **HTTP 409 Conflict**, accompagnato da un messaggio amichevole per l'utente, come: *"Attenzione: i dati che stai cercando di salvare sono stati appena modificati da un altro operatore. Ti preghiamo di ricaricare la pagina per visualizzare le ultime modifiche."*

Tip

Tipi di dato supportati: L'annotazione `@Version` supporta numeri interi (`Integer`, `Short`, `Long`) e `Timestamp`. L'uso dei tipi numerici (come `Integer`) è universalmente preferito per questioni di portabilità tra database diversi, poiché alcuni DBMS hanno risoluzioni dei millisecondi sui `Timestamp` differenti, il che potrebbe causare falsi positivi nei controlli di collisione.

8.5 La Pratica in Spring Boot: L'annotazione `@Transactional`

L'architettura moderna del backend Java delega la gestione del livello relazionale a framework potenti come Spring Data JPA e Hibernate. L'orchestrazione delle transazioni non avviene più scrivendo manualmente istruzioni SQL, ma in modo **dichiarativo**. Lo sviluppatore si limita ad apporre l'annotazione `@Transactional` sopra un metodo di business (solitamente nel *Service Layer*), delegando al framework il compito di inviare i comandi di `BEGIN`, `COMMIT` o `ROLLBACK` al momento giusto.

Tuttavia, utilizzare un costrutto astratto ignorandone il funzionamento meccanico sottostante è la causa principale dei difetti di persistenza nei sistemi Enterprise.

8.5.1 Il pattern Proxy di Spring: cosa succede "sotto il cofano"

In Java, un'annotazione (`@...`) è di per sé completamente inerte. È semplicemente un metadato, un pezzo di testo appiccicato a una classe o a un metodo; non contiene e non esegue alcun codice esplicito. Come fa allora Spring Boot a tradurre questa etichetta in un controllo ferreo sul database PostgreSQL? La risposta risiede in uno dei design pattern più importanti del framework: il **Pattern Proxy**.

Quando si avvia l'applicazione, Spring ispeziona le classi Java. Se trova una classe (o un metodo) annotata con `@Transactional`, il framework **non** restituisce al server web l'oggetto originale. Invece, interviene dinamicamente generando in memoria un "clone avvolgente", un intermediario chiamato, appunto, **Proxy**.

Esempio 8.9 (L'Anatomia di una chiamata `@Transactional`). Analizziamo il flusso logico quando un utente richiede il pagamento di un sinistro nel gestionale *SafeDrive*.

```
1 @Service
2 public class SinistroService {
3
4     @Transactional
5     public void pagaSinistro(Long id) {
6         // 1. Esegue UPDATE sul saldo
7         // 2. Esegue INSERT nello storico bonifici
8     }
9 }
```

Quando il *Controller* riceve la richiesta HTTP e invoca:

```
sinistroService.pagaSinistro(42)
```

la chiamata non finisce direttamente nel tuo codice. Viene intercettata dal **Proxy** di Spring, che esegue questa rigida sequenza meccanica invisibile:

1. **Apertura:** Il Proxy richiede una connessione al database e invia fisicamente il comando SQL `BEGIN`;
2. **Delega:** Il Proxy passa l'esecuzione al tuo vero metodo Java `pagaSinistro(42)`, che esegue la logica di business.
3. **Chiusura (Successo):** Se il tuo metodo termina regolarmente senza intoppi, il Proxy riprende il controllo e invia il comando SQL `COMMIT`;
4. **Chiusura (Emergenza):** Se il tuo metodo si rompe e lancia un'Eccezione (*Exception*), il Proxy intercetta l'errore, capisce che la logica è fallita e invia istantaneamente un `ROLLBACK`; annullando qualsiasi modifica a metà.

La Trappola Mortale della Self-Invocation

Comprendere l'esistenza del Proxy è vitale per evitare il bug transazionale più insidioso nello sviluppo Spring: la *Self-Invocation* (Auto-invocazione).

Se all'interno della classe `SinistroService` scrivi un metodo secondario (privo di annotazione) che **al suo interno** chiama il metodo `pagaSinistro()`, **la transazione sul database non verrà mai aperta**. Questo avviene perché stai effettuando una chiamata interna all'istanza dell'oggetto (equivalente a `this.pagaSinistro()`). Facendo ciò, la chiamata rimane confinata dentro l'oggetto reale e bypassa completamente l'involucro esterno (il Proxy) che Spring aveva preparato. Senza il passaggio dal Proxy, i comandi `BEGIN` e `COMMIT` non vengono

⚠ emessi. Il metodo verrà eseguito in puro e pericoloso *Auto-Commit*: ogni singola riga SQL sarà scolpita sul disco immediatamente, e al primo errore il sistema si ritroverà corrotto, senza alcuna possibilità di fare Rollback.

Regola Architettuale d'Oro: L'annotazione `@Transactional` ha effetto **solo ed esclusivamente** se il metodo viene invocato dall'esterno della classe (es. da un Controller o da un Service di dominio differente).

8.5.2 La trappola del Rollback: Eccezioni Checked vs Unchecked

Abbiamo visto che il Proxy di Spring intercetta qualsiasi eccezione sollevata dal tuo metodo annotato con `@Transactional`. Tuttavia, il comportamento decisionale del Proxy non è uniforme, ma si basa su una rigida convenzione ereditata dai primi anni di vita di Java: la differenza tra eccezioni *Checked* e *Unchecked*.

Il comportamento predefinito (e spesso fatale per i neofiti) di Spring Boot è il seguente:

- **ROLLBACK automatico:** Avviene **solo** se il metodo lancia una **Unchecked Exception** (ovvero classi che estendono `RuntimeException` o errori di sistema non recuperabili).
- **COMMIT letale:** Se il metodo lancia una **Checked Exception** (ovvero classi che estendono la classe base `Exception` ma non `RuntimeException`, come un'eccezione personalizzata di business o una `IOException`), **Spring non esegue il Rollback. Esegue un COMMIT.**

La giustificazione storica di questo comportamento è che le eccezioni *Checked* erano considerate "situazioni anomale ma recuperabili" (es. "File non trovato, chiedi all'utente un altro file"). Spring assume quindi che, nonostante l'errore, tu voglia preservare sul database il lavoro fatto fino a quel momento.

Esempio 8.10 (Il disastro del COMMIT su un'eccezione di Business). Immaginiamo che il gestionale *SafeDrive* abbia una regola di business: non si può rinnovare una polizza se il cliente non ha allegato il documento d'identità aggiornato. Il programmatore Junior crea un'eccezione personalizzata `DocumentoMancanteException` *extends* `Exception` (una *Checked Exception*).

```
1 @Transactional
2 public void rinnovaPolizza(Long idPolizza) throws
   DocumentoMancanteException {
3     // 1. Il DB esegue un UPDATE: Sposta in avanti la scadenza (
       SUCCESSO)
4     polizzaRepository.estendiScadenza(idPolizza);
5
6     // 2. Controllo di Business
7     if (!clienteHaDocumentoValido(idPolizza)) {
8         // Questa e' una Checked Exception!
9         throw new DocumentoMancanteException("Documento scaduto")
           ;
10    }
11
12    // 3. Generazione del PDF contrattuale (non verra' mai
       raggiunto)
13 }
```


Cosa succede in produzione: L'utente invoca il metodo. La riga 1 viene eseguita con successo. La riga 2 fallisce e lancia l'eccezione. Il Proxy di Spring la intercetta, analizza l'albero genealogico della classe e scopre che è una *Checked Exception*. Di conseguenza, il Proxy **esegue il COMMIT** della transazione. Risultato: la polizza è stata rinnovata legalmente sul database, ma il processo si è interrotto a metà, il documento contrattuale non è mai stato generato e l'eccezione è stata restituita all'utente. Il database è logicamente corrotto.

Come risolvere la trappola: Le due vie Architettureali Per impedire questo comportamento, l'ingegneria del backend moderno adotta due possibili difese:

1. **La via moderna (Preferita):** Creare eccezioni di business personalizzate facendole estendere sempre e solo `RuntimeException`. In questo modo, Spring eseguirà sempre il Rollback di default.
2. **La via esplicita:** Modificare il parametro dell'annotazione costringendo il Proxy a ribaltare la sua logica storica e a eseguire il Rollback per qualsiasi anomalia, usando la sintassi: `@Transactional(rollbackFor = Exception.class)`.

8.5.3 Propagazione delle Transazioni: REQUIRED vs REQUIRES_NEW

Nelle architetture Enterprise modulari, è frequente che un metodo transazionale (il "Chiamante") invochi un altro metodo transazionale (il "Chiamato"), magari situato in una classe *Service* differente. La **Propagazione** definisce il comportamento del Proxy di Spring quando si verifica questa sovrapposizione logica.

1. Propagation.REQUIRED (Il comportamento di Default) Se non specifichi nulla, Spring applica automaticamente il livello `REQUIRED`. La regola di questo livello è semplice: *"Se esiste già una transazione fisica aperta, unisciti a quella. Altrimenti, creane una nuova"*.

- **Meccanica:** Il metodo Chiamato non apre una nuova connessione al database, ma si "aggancia" alla transazione del Chiamante. Esiste **una sola transazione fisica** con un solo `COMMIT` finale.
- **Il Rischio Architettureale:** Poiché condividono lo stesso spazio fisico, se il metodo Chiamato fallisce e lancia un'eccezione, "sporca" irrimediabilmente l'intera transazione contrassegnandola come *Rollback-Only*. Anche se il Chiamante catturasse l'eccezione con un `try-catch` nel tentativo di salvarsi e proseguire, al momento del `COMMIT` finale Spring lancerà un errore fatale (`UnexpectedRollbackException`) e annullerà tutto.

2. Propagation.REQUIRES_NEW (L'Isolamento Indipendente) Questo livello si imposta esplicitamente con l'annotazione:

```
@Transactional(propagation = Propagation.REQUIRES_NEW)
```

La regola impone: *"Sospendi qualsiasi transazione in corso e aprine sempre una fisicamente nuova e indipendente"*.

- **Meccanica:** Quando il Chiamante invoca il Chiamato, il Proxy di Spring "mette in pausa" la connessione del Chiamante, richiede una **seconda connessione separata** al database, ed esegue il metodo Chiamato.

- **Il Vantaggio Architettureale:** Le due transazioni sono fisicamente isolate. Se il metodo Chiamato fallisce e fa ROLLBACK, il Chiamante può catturare l'eccezione e continuare a lavorare, eseguendo infine il suo COMMIT con successo.

Esempio 8.11 (L'uso vitale di REQUIRES_NEW: Il Log di Audit). Nel gestionale *SafeDrive*, per motivi legali, ogni tentativo di pagamento di un sinistro (sia che vada a buon fine, sia che fallisca) deve essere registrato in una tabella di *Audit Log*.

```
1 @Service
2 public class AuditService {
3     // Deve salvare il log a prescindere da cosa succeda fuori!
4     @Transactional(propagation = Propagation.REQUIRES_NEW)
5     public void salvaLog(String messaggio) {
6         auditRepository.save(new Log(messaggio));
7     }
8 }
9
10 @Service
11 public class SinistroService {
12     @Autowired private AuditService auditService;
13
14     @Transactional // Default: REQUIRED
15     public void tentaPagamento(Long idSinistro) {
16         auditService.salvaLog("Inizio pagamento per sinistro " +
17                               idSinistro);
18
19         // ... logica complessa che potrebbe fallire lanciando un
20         // eccezione ...
21     }
22 }
```

Perché REQUIRES_NEW è l'unica soluzione: Se l'`AuditService` usasse il default (`REQUIRED`), si aggancerebbe alla transazione di `tentaPagamento`. Se il pagamento fallisse (es. fondi insufficienti), il conseguente ROLLBACK cancellerebbe dal database non solo il tentativo di pagamento, ma anche il Log appena inserito! L'azienda perderebbe le tracce legali dell'operazione fallita. Utilizzando `REQUIRES_NEW`, l'inserimento del log avviene in una "bolla" indipendente: viene *committato* sul disco istantaneamente, prima ancora che il metodo del pagamento inizi a fare i suoi calcoli a rischio fallimento.